

Cryptography Notes (Depth Notes)

Feistel, SPN, DES/2DES/3DES, and Finite Fields

(Reference-focused on William Stallings)

Prepared by **Er. Arjun Neupane**

January 11, 2026

Contents

1	Confusion and Diffusion (Important Concepts)	4
1.1	Meaning and purpose (core idea)	4
1.2	Confusion	4
1.2.1	Effect of changing a key bit (avalanche idea)	4
1.3	Diffusion	4
1.3.1	Effect of changing a plaintext bit	4
1.4	“Vagueness” and “Redundancy” interpretation (as in your note)	5
1.5	Stream cipher vs Block cipher (confusion and diffusion)	5
1.6	One-line summary (very exam friendly)	5
2	Feistel Cipher Structure	5
2.1	Why do we need block cipher structures?	5
2.2	Feistel network idea (the main trick)	6
2.3	Feistel diagram (data flow)	7
2.4	What the diagram is showing	7
2.5	One Feistel round (the mathematics behind each round)	8
2.6	Why decryption is possible even if F is NOT invertible (core Feistel property)	9
2.7	Decryption process (prove mathematically with the same structure)	9
2.8	Why the decryption diagram starts with swapped halves	10
2.9	Round-by-round explanation	10
2.10	Algorithm	10
2.11	Most important exam lines (15 marks conclusion)	11
2.12	Why Feistel is reversible (key point for exams)	11
2.13	Design ingredients of the round function F	11
2.14	Feistel vs non-Feistel block cipher	12
3	Substitution-Permutation Network (SPN)	12
3.1	Core idea	12
3.2	SPN diagram (simple)	12
3.3	Feistel vs SPN (exam comparison)	12
4	DES (Data Encryption Standard)	13
4.1	DES overview (Stallings-style summary)	13
4.2	DES high-level structure	13
5	General Depiction of DES Encryption Process (Short Notes)	13
5.1	Block input and initial permutation	13
5.2	Round processing (16 rounds)	13
5.3	Key schedule connection shown in the figure	14
5.4	Final swap and inverse permutation	14
5.5	Why expansion and S-boxes are used	14
6	DES Key Generation Process (Round Key Generator)	15

7	Single Round of DES Algorithm (10 Marks Notes with Diagram Explanation)	17
7.1	Overview of what happens in one DES round	17
7.2	Round equations (core mathematics)	18
7.3	Round function F in detail (step-by-step as shown in the diagram)	18
7.4	How the diagram produces L_i and R_i	19
7.5	Round key K_i generation link shown on the right side	19
7.6	Why a single round still keeps invertibility	20
8	Double DES and Triple DES	20
8.1	Motivation: why not single DES today?	20
8.2	Double DES (2DES)	20
8.3	Meet-in-the-middle attack (why 2DES is not strong enough)	20
8.4	Triple DES (3DES)	20
8.4.1	EDE mode (most common)	21
8.4.2	Keying options	21
9	Finite Fields Background for Cryptography	21
9.1	Why algebra (groups/rings/fields) matters in cryptography	21
9.2	Groups	21
9.3	Rings	22
9.4	Fields	22
10	Modular Arithmetic (Core for Cryptography)	22
10.1	Congruence and modulo	22
10.2	Arithmetic rules (important)	23
10.3	Modular inverse	23
11	Euclidean Algorithm and Extended Euclidean Algorithm	23
11.1	Euclidean Algorithm (GCD)	23
11.2	Extended Euclidean Algorithm (EEA)	23
12	Galois Fields: $GF(p)$ and $GF(2^n)$	24
12.1	$GF(p)$: finite field of prime order	24
12.2	$GF(2^n)$: binary extension fields (used in AES)	25
12.3	Polynomial representation	25
12.4	Addition in $GF(2^n)$ (XOR)	25
12.5	Multiplication in $GF(2^n)$ (polynomial + modular reduction)	25
12.6	Polynomial arithmetic over $GF(2)$: key rules	26
13	Quick Revision Tables	27
13.1	Feistel vs SPN (short table)	27
13.2	DES / 2DES / 3DES (short table)	27
14	Practice Questions (Exam Style)	27
15	References (Stallings-based)	28

1 Confusion and Diffusion (Important Concepts)

1.1 Meaning and purpose (core idea)

In modern cryptography, two fundamental design goals are **confusion** and **diffusion**. These terms were introduced by Claude Shannon and are emphasized in William Stallings to explain how block ciphers achieve security by hiding statistical structure.

Classroom Note

Easy classroom statement:

Confusion hides the relationship between **ciphertext** and **key**.

Diffusion hides the relationship between **ciphertext** and **plaintext**.

1.2 Confusion

Confusion means: even if an attacker knows how the system works, the relationship between the **key** and the **ciphertext** should be highly complex and unclear. This is usually achieved using **substitution** (S-boxes or nonlinear mapping).

1.2.1 Effect of changing a key bit (avalanche idea)

In a good cipher, if we change only **one bit of the secret key**, then **many or most bits of the ciphertext** should change. This makes it very hard for an attacker to guess how the key affects the output.

Exam-Focused Points (Write in Answers)

Write in exam:

In confusion, a small change in **key** causes a large unpredictable change in **ciphertext**. This masks the ciphertext–key relationship.

1.3 Diffusion

Diffusion means: the influence of a single plaintext bit should spread over many ciphertext bits. In other words, the ciphertext should not reveal patterns or redundancy of the plaintext. This is usually achieved using **permutation** and **mixing** operations (like P-box, shifting, linear mixing).

1.3.1 Effect of changing a plaintext bit

In a good cipher, if we change only **one bit of the plaintext**, then **many or most bits of the ciphertext** should change. This helps remove statistical patterns and redundancy from the plaintext.

Exam-Focused Points (Write in Answers)

Write in exam:

In diffusion, a small change in **plaintext** causes many bits of **ciphertext** to change. This masks the ciphertext–plaintext relationship.

1.4 “Vagueness” and “Redundancy” interpretation (as in your note)

Sometimes students explain confusion and diffusion using simple words:

- **Confusion** increases **vagueness/complexity** of how the key affects ciphertext.
- **Diffusion** spreads plaintext information so that plaintext **redundancy/patterns** are not visible in ciphertext.

Classroom Note

Important correction for classroom:

Diffusion does *not* “increase redundancy” in ciphertext.

Rather, diffusion **spreads** the plaintext redundancy so that it becomes **hidden** and does not appear as obvious patterns.

1.5 Stream cipher vs Block cipher (confusion and diffusion)

- **Stream ciphers:** mainly aim for **confusion** (key should unpredictably affect the keystream and ciphertext).
- **Block ciphers:** aim for both **confusion and diffusion** through repeated rounds of substitution and permutation/mixing.

Exam-Focused Points (Write in Answers)

Short 5-mark statement:

Both stream and block ciphers use **confusion**.

Block ciphers (especially SPN/Feistel designs) are designed to strongly provide **diffusion** as well.

1.6 One-line summary (very exam friendly)

Confusion: hides ciphertext–key relationship (mainly substitution/nonlinearity).

Diffusion: hides ciphertext–plaintext relationship by spreading plaintext bits (mainly permutation/mixing).

Avalanche: 1-bit change in key or plaintext should change many ciphertext bits.

2 Feistel Cipher Structure

2.1 Why do we need block cipher structures?

A modern block cipher must achieve two core security goals described in classic cryptography:

- **Confusion:** hide the relationship between the ciphertext and the key.
- **Diffusion:** spread the influence of each plaintext bit over many ciphertext bits.

In William Stallings' explanation, these properties are achieved through **repeated rounds** that combine:

- **substitution** (nonlinear operations, often using S-boxes),
- **permutation/linear mixing** (bit shuffles, XOR mixing),
- and **key mixing** (XOR with round keys).

2.2 Feistel network idea (the main trick)

The Feistel structure is extremely important because it allows **encryption and decryption using the same round structure** (only subkeys are used in reverse order). This was one of the most practical advantages behind DES.

Definition 2.1 (Feistel Round). *Let the input block be split into two halves:*

$$L_{i-1}, R_{i-1}$$

A Feistel round produces:

$$L_i = R_{i-1}, \quad R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

where F is the round function and K_i is the round key.

Classroom Note

Classroom explanation:

“In each round, the right half is processed by a function F and then mixed (XOR) with the left half. After that, we swap halves. Because of the swap and XOR, we can reverse the process easily during decryption.”

2.3 Feistel diagram (data flow)

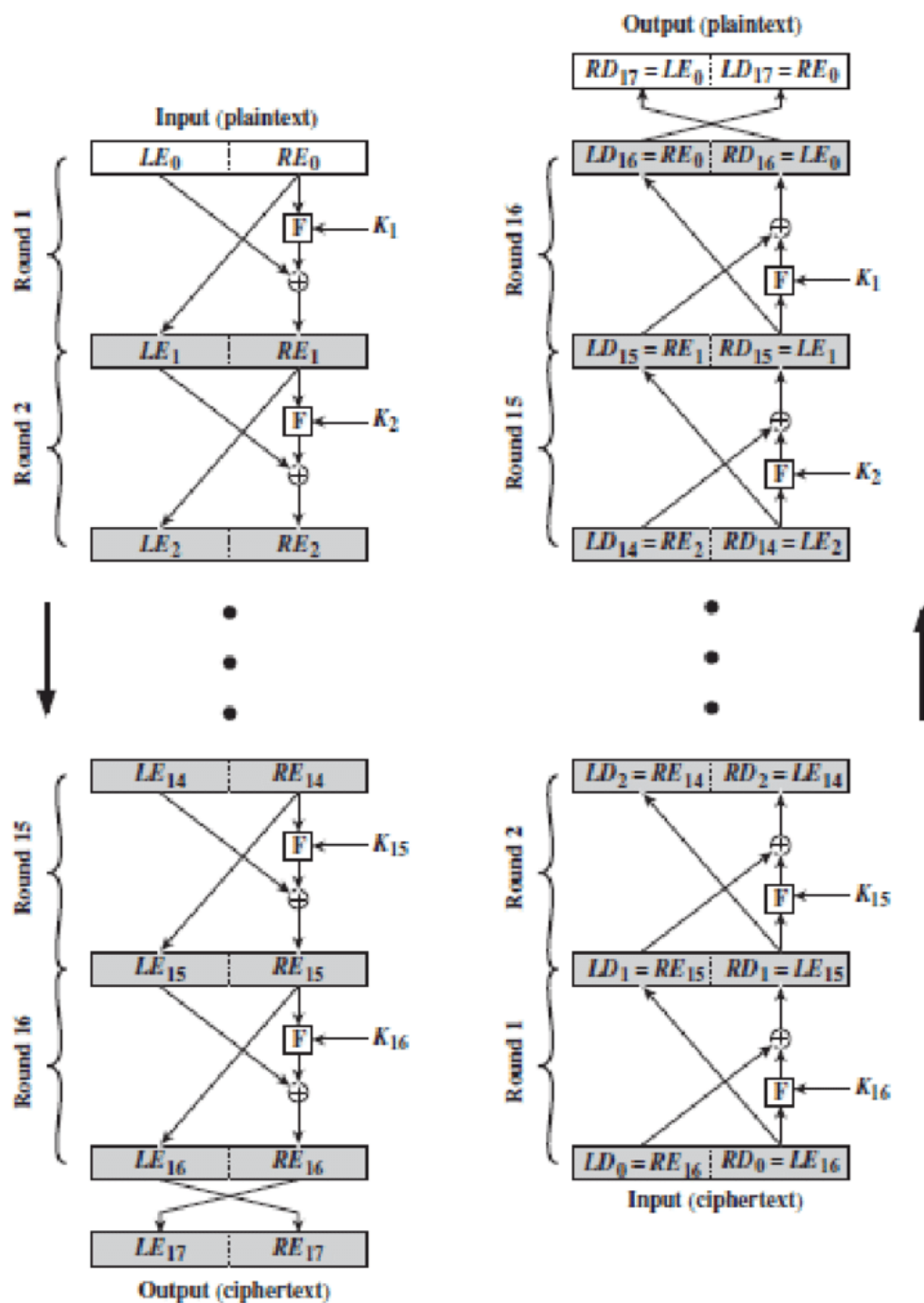


Figure 1: Feistel Cipher Encryption and Decryption

2.4 What the diagram is showing

Figure-1 shows the same Feistel structure used for both:

- **Encryption (left side):** plaintext $\rightarrow$ ciphertext using round keys $K_1, K_2, \dots, K_{16}$
- **Decryption (right side):** ciphertext $\rightarrow$ plaintext using the **same round function** but keys in reverse order $K_{16}, K_{15}, \dots, K_1$

The block is split into two halves at every stage:

$$\text{Input block} = L_0 \parallel R_0$$

In the figure, encryption halves are labeled as:

$$LE_0, RE_0, LE_1, RE_1, \dots, LE_{16}, RE_{16}$$

and decryption halves are labeled as:

$$LD_0, RD_0, LD_1, RD_1, \dots, LD_{16}, RD_{16}$$

2.5 One Feistel round (the mathematics behind each round)

A Feistel round takes the previous halves (L_{i-1}, R_{i-1}) and produces (L_i, R_i) using a round function F and a round key K_i .

Encryption round equations

For round $i = 1, 2, \dots, 16$:

$$L_i = R_{i-1} \tag{1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i) \tag{2}$$

Where:

- $\oplus$ denotes XOR (bitwise addition mod 2)
- $F(\cdot)$ is the round function (it may include expansion, substitution (S-box), permutation, etc., depending on the cipher)
- K_i is the round subkey for round i

How the figure matches these equations

In the left side (encryption):

- The **crossed lines** show the swap: $L_i \leftarrow R_{i-1}$
- The small $\oplus$ circle shows: $R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$
- The F box uses the right half and round key K_i

2.6 Why decryption is possible even if F is NOT invertible (core Feistel property)

Feistel design is powerful because the round function F does not need to be invertible. Decryption works because XOR is reversible:

$$A \oplus B \oplus B = A$$

From the encryption equations:

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

If we know R_i and also know $F(R_{i-1}, K_i)$, then we can recover L_{i-1} by XOR again:

$$L_{i-1} = R_i \oplus F(R_{i-1}, K_i)$$

Also, from $L_i = R_{i-1}$ we directly get:

$$R_{i-1} = L_i$$

2.7 Decryption process (prove mathematically with the same structure)

Assume after encryption we have (L_i, R_i) . We want to recover (L_{i-1}, R_{i-1}) .

From (1):

$$R_{i-1} = L_i$$

From (2):

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

Substitute $R_{i-1} = L_i$:

$$R_i = L_{i-1} \oplus F(L_i, K_i)$$

Now XOR both sides with $F(L_i, K_i)$:

$$R_i \oplus F(L_i, K_i) = L_{i-1}$$

Therefore, the **decryption equations** are:

$$R_{i-1} = L_i \tag{3}$$

$$L_{i-1} = R_i \oplus F(L_i, K_i) \tag{4}$$

Key observation (matches the right side of the figure)

To decrypt the full cipher:

- we apply the same round structure,
- but we feed keys in reverse order: $K_{16}, K_{15}, \dots, K_1$.

That is exactly why on the right side the first decryption round uses K_{16} , then K_{15} , and so on.

2.8 Why the decryption diagram starts with swapped halves

In many Feistel ciphers (including DES), after the last round there is a **final swap** before output. That is why the encryption output is drawn as a swapped pair.

So, if encryption ends at (LE_{16}, RE_{16}) , the ciphertext output is commonly:

$$C = RE_{16} \parallel LE_{16}$$

Now look at the right side (decryption input). It shows:

$$LD_0 = RE_{16}, \quad RD_0 = LE_{16}$$

This is simply the ciphertext halves being assigned as the starting halves for decryption.

2.9 Round-by-round explanation

Encryption (left)

1. Start: (LE_0, RE_0) from plaintext split.

2. Round 1 with K_1 :

$$LE_1 = RE_0, \quad RE_1 = LE_0 \oplus F(RE_0, K_1)$$

3. Round 2 with K_2 :

$$LE_2 = RE_1, \quad RE_2 = LE_1 \oplus F(RE_1, K_2)$$

4. Continue similarly until round 16 with K_{16} .

5. Output (often with final swap): ciphertext $\approx RE_{16} \parallel LE_{16}$ (as shown).

Decryption (right)

1. Input ciphertext halves: $(LD_0, RD_0) = (RE_{16}, LE_{16})$.

2. Decryption round 1 uses K_{16} (reverse order):

$$LD_1 = RD_0, \quad RD_1 = LD_0 \oplus F(RD_0, K_{16})$$

3. Next uses K_{15} , and so on.

4. After 16 decryption rounds, we recover the original halves (LE_0, RE_0) (up to the final swap shown in the diagram).

2.10 Algorithm

Feistel Encryption

1. Input: plaintext block P , round keys $K_1, \dots, K_{16}$.

2. Split: $P = L_0 \parallel R_0$.

3. For $i = 1$ to 16:

$$L_i = R_{i-1}, \quad R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

4. Output (with final swap): $C = R_{16} \parallel L_{16}$.

Feistel Decryption

1. Input: ciphertext block C , round keys $K_{16}, \dots, K_1$.
2. Split: $C = R_{16} \parallel L_{16}$ (so set $L_0 = R_{16}$, $R_0 = L_{16}$ as shown).
3. For $i = 1$ to 16 (keys reversed):

$$L_i = R_{i-1}, \quad R_i = L_{i-1} \oplus F(R_{i-1}, K_{17-i})$$

4. Output (with final swap): recover plaintext $P = R_{16} \parallel L_{16}$ (equivalently $L_0 \parallel R_0$).

2.11 Most important exam lines (15 marks conclusion)

- A Feistel cipher updates halves using: $L_i = R_{i-1}$ and $R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$.
- Decryption is possible without inverting F because XOR is reversible.
- Decryption uses the same structure with subkeys in reverse order: $K_{16}, \dots, K_1$.
- The swaps drawn in the figure explain why ciphertext halves are fed reversed at the start of decryption.

2.12 Why Feistel is reversible (key point for exams)

Because XOR has the property:

$$A \oplus B \oplus B = A$$

So in decryption, we can recover L_{i-1} as:

$$L_{i-1} = R_i \oplus F(L_i, K_i)$$

and $R_{i-1} = L_i$ (from the swap).

Exam-Focused Points (Write in Answers)

Write these points for 10/15 marks:

- Feistel splits block into halves and uses XOR mixing with F .
- Decryption uses the **same structure** with keys in reverse order.
- XOR reversibility is the mathematical reason.
- Multiple rounds achieve confusion + diffusion.

2.13 Design ingredients of the round function F

Typical ingredients (as described in Stallings for DES-like ciphers):

- Expansion/permutation of bits (make right half larger or rearranged)
- XOR with round key
- Substitution through S-boxes (nonlinearity)
- Permutation/straight P-box (diffusion)

2.14 Feistel vs non-Feistel block cipher

Feistel is a **structure**. Many ciphers use other designs (e.g., SPN/AES). Feistel's key advantage is that invertibility does not require F to be invertible.

3 Substitution-Permutation Network (SPN)

3.1 Core idea

An SPN block cipher builds security by repeating:

- 1) **Substitution layer (S-boxes)** for confusion (nonlinearity)
- 2) **Permutation / linear diffusion layer** to spread bits (diffusion)
- 3) **Round key mixing** (usually XOR)

Definition 3.1 (SPN Round (generic form)). *A typical SPN round can be written as:*

$$\text{State} \leftarrow P(S(\text{State} \oplus K_i))$$

where S is substitution, P is permutation/linear mixing, and K_i is round key.

3.2 SPN diagram (simple)



3.3 Feistel vs SPN (exam comparison)

- **Feistel:** half-block swapping, XOR with F , decryption same structure.
- **SPN:** full-block transformations, substitution + permutation layers; inverse operations used in decryption.

Exam-Focused Points (Write in Answers)

Typical 5 marks: Differentiate Feistel and SPN

- Feistel does not require F to be invertible; SPN requires invertible layers for decryption.
- Feistel processes half-block each round; SPN processes full state each round.
- Both aim for confusion and diffusion by multiple rounds.

4 DES (Data Encryption Standard)

4.1 DES overview (Stallings-style summary)

DES is:

- a block cipher with block size 64 bits
- a Feistel cipher with 16 rounds
- uses an effective key length of 56 bits (64-bit input key includes 8 parity bits)

4.2 DES high-level structure

DES takes plaintext block M :

- 1) Apply **Initial Permutation (IP)**
- 2) Split into L_0 and R_0 (32 bits each)
- 3) Apply 16 Feistel rounds
- 4) Swap halves (after round 16)
- 5) Apply **Final Permutation (FP = IP⁻¹)**

5 General Depiction of DES Encryption Process (Short Notes)

5.1 Block input and initial permutation

DES is a 64-bit block cipher. It takes a 64-bit plaintext block as input and first applies the initial permutation (IP), which is a fixed bit-reordering operation. The output of IP is still 64 bits. After this permutation, the block is treated as two 32-bit halves, commonly denoted by L_0 and R_0 .

5.2 Round processing (16 rounds)

DES performs 16 Feistel rounds. In each round i , a 48-bit round key K_i is used. The round update follows the Feistel equations:

$$L_i = R_{i-1}, \quad R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

Here, F is the DES round function that mixes the right half with the round key using expansion, XOR, substitution (S-boxes), and permutation. After each round, the output remains 64 bits in total (two 32-bit halves).

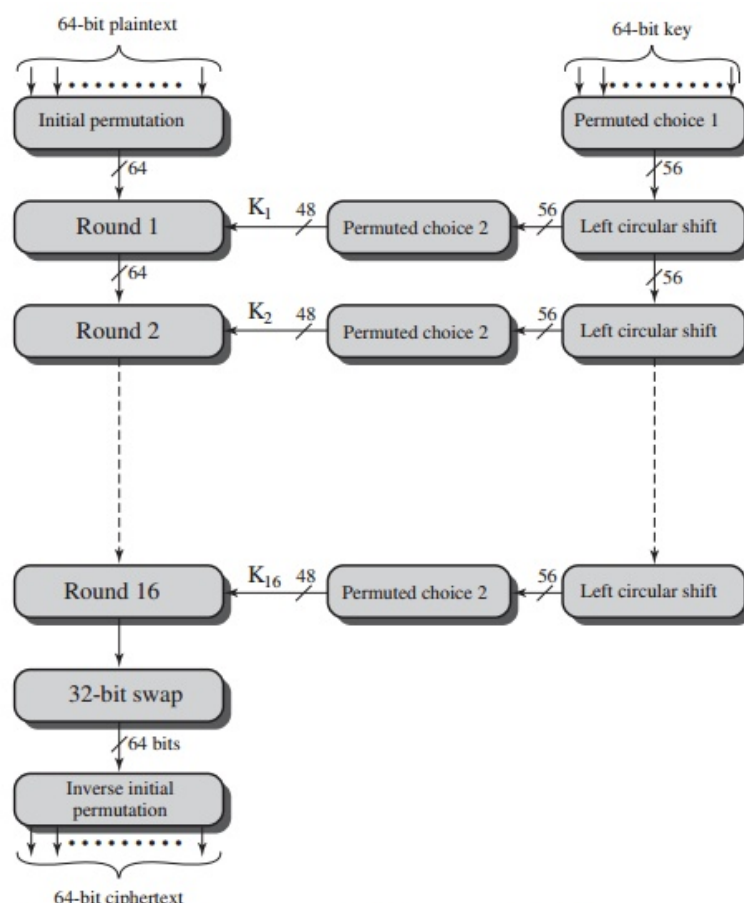


Figure 3.5 General Depiction of DES Encryption Algorithm

Figure 2: Des general structure

5.3 Key schedule connection shown in the figure

The figure also shows how the 48-bit round keys are generated from the 64-bit key. The 64-bit key first passes through permuted choice 1 (PC-1), producing 56 bits (parity bits removed and bits permuted). Then, for each round, the 56-bit value is circularly left shifted (on two 28-bit halves), and permuted choice 2 (PC-2) compresses and permutes it to produce a 48-bit round key K_i . This process repeats to generate K_1 through K_{16} .

5.4 Final swap and inverse permutation

After round 16, DES performs a 32-bit swap of the two halves, so the final pair becomes (R_{16}, L_{16}) . Then the inverse initial permutation (IP^{-1}) is applied to produce the final 64-bit ciphertext block. Decryption uses the same structure but applies the round keys in reverse order from K_{16} down to K_1 .

5.5 Why expansion and S-boxes are used

- Expansion makes 32-bit input match 48-bit key mixing.
- Repeated bits in expansion help diffusion and ensure boundary bits influence multiple S-boxes.

- S-boxes introduce **nonlinearity** (confusion); they are the main defense against linear/differential cryptanalysis (as discussed in modern analysis).

6 DES Key Generation Process (Round Key Generator)

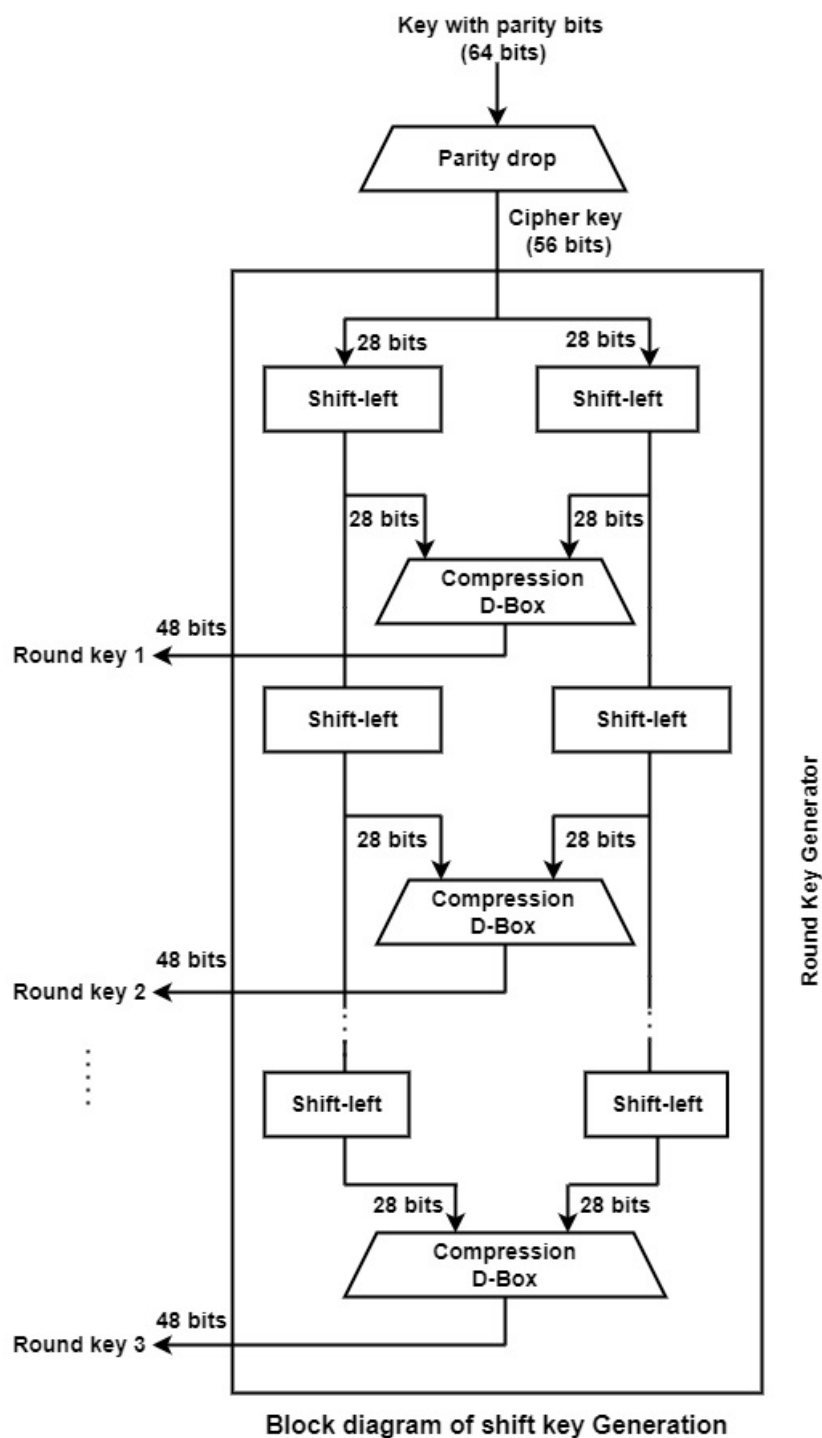


Figure 3: DES KEY generation process

Data Encryption Standard (DES) is a **block cipher** that operates on **64-bit blocks** of data. In encryption, DES takes a **64-bit plaintext block** and transforms it into a **64-bit ciphertext block**. In decryption, DES takes the **64-bit ciphertext block** and converts it back into the original **64-bit plaintext block**. DES is a **symmetric key algorithm**, which means the **same secret key** is used for both encryption and decryption. Although a DES key is given as **64 bits**, only **56 bits** are actually used as the effective cipher key, while the remaining **8 bits are parity bits**. In DES, the overall algorithm structure is fixed and publicly known; the security comes from keeping the **actual key** secret between the sender and the receiver.

DES uses **16 rounds** of processing, and an important feature of DES is that it does not use the same key bits directly in every round. Instead, it generates **sixteen different round keys**, one for each round. Each round key is **48 bits long**. Therefore, the purpose of the DES key generation process (also called the **key schedule** or **round key generator**) is to produce sixteen **48-bit subkeys**:

$$K_1, K_2, K_3, \dots, K_{16}$$

from the single original **56-bit effective key**. These round keys are used inside the DES round function to provide **confusion** and to make the ciphertext strongly dependent on the secret key.

The key generation process begins with the 64-bit key with parity bits. The 64-bit input key contains 8 parity bits placed at positions 8, 16, 24, 32, 40, 48, 56, and 64. These parity bits are discarded because they are not used as real key bits. This first stage is called parity drop. In the parity drop stage, DES applies a fixed permutation (commonly called Permuted Choice 1 or PC-1), which not only removes the parity bits but also reorders the remaining bits according to a predefined table. As a result of this parity drop and permutation, we obtain a 56-bit cipher key:

$$K^+ = PC1(K)$$

This 56-bit key K^+ is the main input to the round key generator, as shown in Figure ??.

After obtaining the 56-bit key, DES divides it into two equal halves of 28 bits each. The left half is denoted by C_0 and the right half is denoted by D_0 :

$$K^+ = C_0 \parallel D_0$$

where C_0 contains the first 28 bits and D_0 contains the last 28 bits. The reason DES splits the key into two halves is because in every round, both halves will be shifted (rotated) and then combined again. This repeated shifting ensures that different sets of key bits appear in different rounds, so the round keys are not identical and not trivially related.

In each round i (from 1 to 16), DES performs a circular left shift on both halves C_{i-1} and D_{i-1} to obtain C_i and D_i . This operation is shown as Shift-left blocks in Figure ??. A circular left shift means the leftmost bit that goes out from the left end is wrapped around and inserted back at the right end, so no key bit is lost. Mathematically, for each round:

$$C_i = LS_i(C_{i-1}), \quad D_i = LS_i(D_{i-1})$$

where LS_i indicates the number of left shifts in round i . DES uses a specific shift schedule. In rounds 1, 2, 9, and 16 the shift is 1 bit, and in all other rounds the shift is 2 bits. Therefore, the shift schedule is:

$$\{1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 1\}$$

This shifting schedule is chosen so that the key bits move through different positions across the 16 rounds, producing strong variation among round keys and avoiding simple repetitive patterns.

After shifting, the two 28-bit halves are **combined** again to form a 56-bit intermediate value:

$$C_i \parallel D_i$$

However, DES round keys must be **48 bits**, not 56 bits. So DES applies another fixed permutation and selection step, known as **Permuted Choice 2 (PC-2)** or the **compression D-box**. This compression stage is shown as the **Compression D-Box** blocks in Figure ???. The compression D-box takes the 56 combined bits and **selects 48 of them** (and also permutes them) to produce the round key:

$$K_i = PC2(C_i \parallel D_i)$$

Thus, each round key K_i is exactly **48 bits** and is used only in that particular round of DES.

By repeating the same procedure for all 16 rounds, the round key generator produces sixteen unique keys:

$$K_1, K_2, K_3, \dots, K_{16}$$

These keys are then used in the DES encryption rounds in the order $K_1 \rightarrow K_{16}$. During decryption, DES uses the same round function but applies the subkeys in the reverse order $K_{16} \rightarrow K_1$. This is possible because DES is based on a Feistel structure, and the Feistel design allows decryption to be done by reversing the sequence of round keys.

Historically, DES originated from IBM's research in the late 1960s, where IBM developed a cipher known as LUCIFER. In the 1970s, the design was modified and standardized into DES with several innovations, including the 16-round structure, the key schedule (parity drop, shifts, compression), and carefully designed substitution boxes. Over time, because computing power increased, the effective 56-bit key length became vulnerable to brute-force attacks. Therefore, modern improvements and replacements were introduced, such as using longer keys (for example, algorithms with 128-bit keys) and using multi-pass designs such as Triple DES (3DES), which applies DES multiple times with multiple keys to increase security.

7 Single Round of DES Algorithm (10 Marks Notes with Diagram Explanation)

7.1 Overview of what happens in one DES round

DES is a 16-round Feistel block cipher. In each round, the 64-bit data block is split into two 32-bit halves. The left half and right half before round i are denoted by L_{i-1} and R_{i-1} . The output halves after round i are L_i and R_i . The main operations in one round are swapping, applying the round function F to the right half with the round key K_i , and XOR mixing with the left half. The diagram in Figure 4 shows both the data path (left side) and the key schedule portion that produces K_i (right side).

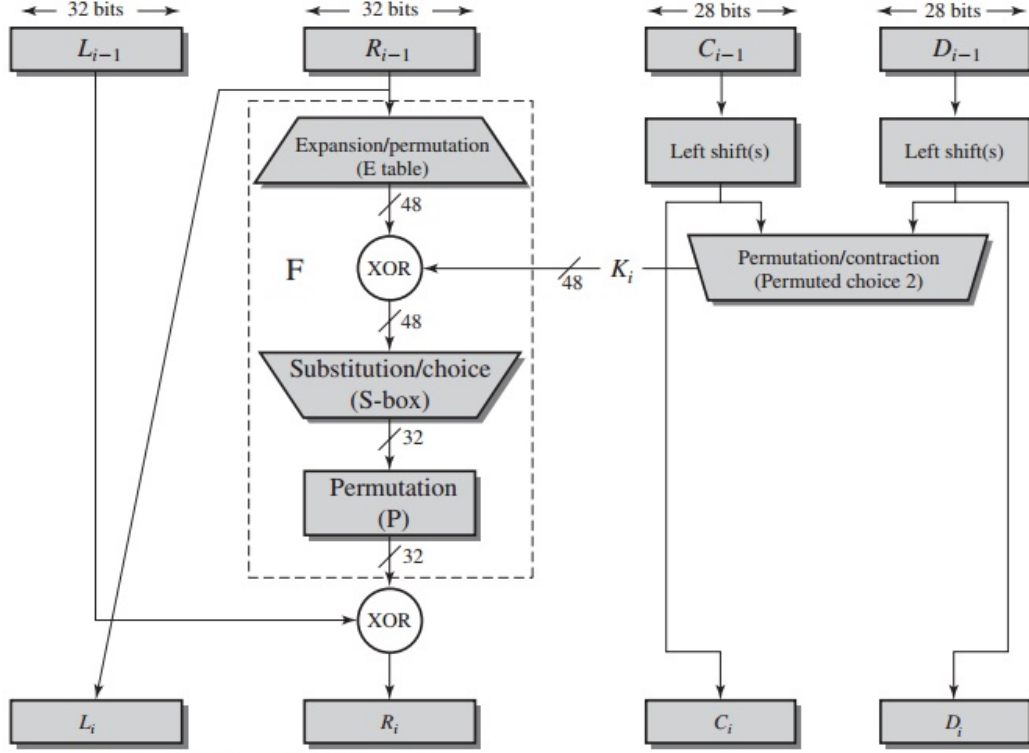


Figure 3.6 Single Round of DES Algorithm

Figure 4: Single round of DES: round function F and round-key generation link.

7.2 Round equations (core mathematics)

A single DES round follows the Feistel equations:

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

The first equation means the next left half is simply the previous right half. The second equation means the next right half is formed by XORing the previous left half with the output of the round function F .

7.3 Round function F in detail (step-by-step as shown in the diagram)

The round function F takes two inputs: a 32-bit value R_{i-1} and a 48-bit round key K_i . It produces a 32-bit output. The function contains four main stages: expansion/permutation, XOR with the round key, substitution using S-boxes, and permutation using P-box.

First, R_{i-1} (32 bits) is passed through the expansion/permutation E table. This expansion converts 32 bits into 48 bits by reordering bits and repeating some boundary bits. The main reason for expansion is to match the 48-bit round key length and to ensure diffusion by letting some input bits influence multiple S-box inputs:

$$E(R_{i-1}) : 32 \rightarrow 48$$

Second, the expanded 48-bit result is XORed with the 48-bit round key K_i :

$$B = E(R_{i-1}) \oplus K_i$$

The output B is still 48 bits.

Third, the 48-bit value B is divided into eight 6-bit blocks:

$$B = B_1 \parallel B_2 \parallel \cdots \parallel B_8$$

Each B_j (6 bits) is applied to one S-box S_j . Each S-box maps 6 bits to 4 bits, so after all eight S-box substitutions, the output becomes 32 bits:

$$S(B) : 48 \rightarrow 32$$

In DES, S-boxes provide nonlinearity and are the main source of confusion, meaning they make the relationship between key bits and ciphertext bits complex.

Fourth, the 32-bit output from the S-box layer is permuted using the P permutation (also called the straight permutation). This rearranges bits to spread the influence of each S-box output across positions that will be used in different S-boxes in the next round, supporting diffusion:

$$P(\cdot) : 32 \rightarrow 32$$

Combining these stages, the DES round function is written compactly as:

$$F(R_{i-1}, K_i) = P(S(E(R_{i-1}) \oplus K_i))$$

7.4 How the diagram produces L_i and R_i

Figure 4 shows that R_{i-1} enters the dashed box (the F function). The output of F is a 32-bit value that is XORed with L_{i-1} . The XOR circle near the bottom of the dashed box represents:

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

The line from R_{i-1} to L_i represents the Feistel swap:

$$L_i = R_{i-1}$$

Therefore the output of the round is the pair (L_i, R_i) .

7.5 Round key K_i generation link shown on the right side

The diagram also shows the key schedule elements for generating the round key K_i . The 56-bit key state is split into two 28-bit halves, denoted by C_{i-1} and D_{i-1} . Each round performs left circular shift(s) on both halves to obtain C_i and D_i . Then the permuted choice 2 block (permutation/contraction) selects and permutes 48 bits out of the combined 56 bits:

$$K_i = PC2(C_i \parallel D_i)$$

This 48-bit key K_i is fed into the XOR inside the F function.

7.6 Why a single round still keeps invertibility

Even if F is complex and not invertible, the whole round is invertible due to the Feistel design. XOR is reversible and the swap is reversible, so decryption can be done by running the same round structure with keys in reverse order. This is why DES can decrypt using the same internal design.

8 Double DES and Triple DES

8.1 Motivation: why not single DES today?

DES key length is 56 bits. Brute-force search became feasible with advances in computing power. So stronger variants were introduced.

8.2 Double DES (2DES)

Double DES means encrypt twice with two keys:

$$C = E_{K_2}(E_{K_1}(P))$$

Naively, one might think security becomes $2^{56} \times 2^{56} = 2^{112}$, but it does not.

8.3 Meet-in-the-middle attack (why 2DES is not strong enough)

Given P and C :

$$X = E_{K_1}(P), \quad C = E_{K_2}(X)$$

Attack idea:

- Compute $E_{K_1}(P)$ for all 2^{56} keys and store results.
- Compute $D_{K_2}(C)$ for all 2^{56} keys and look for matches.

This reduces complexity to about:

$$\approx 2^{57} \text{ operations (time) and } 2^{56} \text{ storage}$$

So 2DES is not recommended.

Exam-Focused Points (Write in Answers)

Write for 10 marks: 2DES is vulnerable due to meet-in-the-middle, giving about 2^{57} complexity, not 2^{112} .

8.4 Triple DES (3DES)

3DES was designed to strengthen DES while keeping DES hardware compatibility.

8.4.1 EDE mode (most common)

$$C = E_{K_3}(D_{K_2}(E_{K_1}(P)))$$

Why EDE?

- If $K_1 = K_2 = K_3$, then 3DES becomes single DES (backward compatibility).
- Decryption is symmetric:

$$P = D_{K_1}(E_{K_2}(D_{K_3}(C)))$$

8.4.2 Keying options

- **3-key 3DES:** K_1, K_2, K_3 independent (strongest)
- **2-key 3DES:** $K_1 = K_3$, K_2 independent (common in practice historically)

Classroom Note

Classroom explanation:

“3DES is like applying DES three times in a smart way so that old DES systems can still interoperate, but security becomes much stronger than single DES.”

9 Finite Fields Background for Cryptography

9.1 Why algebra (groups/rings/fields) matters in cryptography

Many cryptographic systems depend on arithmetic structures where operations behave predictably:

- RSA uses modular arithmetic over integers.
- AES uses arithmetic in $GF(2^8)$ (a finite field).
- ECC uses groups over finite fields.

So we must understand **groups, rings, and fields**.

9.2 Groups

Definition 9.1 (Group). A set G with a binary operation $\circ$ is a group if:

- Closure:** $a, b \in G \Rightarrow a \circ b \in G$
- Associativity:** $(a \circ b) \circ c = a \circ (b \circ c)$
- Identity:** $\exists e \in G$ s.t. $a \circ e = e \circ a = a$
- Inverse:** $\forall a \in G, \exists a^{-1} \in G$ s.t. $a \circ a^{-1} = e$

If also $a \circ b = b \circ a$, then group is **abelian (commutative)**.

Worked Example

Example: $(\mathbb{Z}, +)$ is an abelian group.
Identity is 0, inverse of a is $-a$.

9.3 Rings

Definition 9.2 (Ring). A set R with two operations $(+, \cdot)$ is a ring if:

- $(R, +)$ is an abelian group.
- Multiplication $\cdot$ is associative.
- Distributive laws hold:

$$a \cdot (b + c) = a \cdot b + a \cdot c, \quad (a + b) \cdot c = a \cdot c + b \cdot c$$

Worked Example

$\mathbb{Z}$ with $(+, \times)$ is a ring.

9.4 Fields

Definition 9.3 (Field). A ring $(F, +, \cdot)$ is a field if:

- $(F, +)$ is an abelian group
- $(F \setminus \{0\}, \cdot)$ is an abelian group

So every nonzero element has a multiplicative inverse.

Worked Example

Examples: $\mathbb{Q}$, $\mathbb{R}$, and finite fields like $GF(p)$ where p is prime.

Exam-Focused Points (Write in Answers)**One-line memory:**

Group = 1 operation + inverses; Ring = 2 operations (add group + multiply associative); Field = ring + multiplicative inverses for all nonzero.

10 Modular Arithmetic (Core for Cryptography)

10.1 Congruence and modulo

We say:

$$a \equiv b \pmod{n}$$

if n divides $(a - b)$.

10.2 Arithmetic rules (important)

If $a \equiv b \pmod{n}$ and $c \equiv d \pmod{n}$ then:

$$a + c \equiv b + d \pmod{n}$$

$$a \cdot c \equiv b \cdot d \pmod{n}$$

10.3 Modular inverse

An integer a has an inverse modulo n if:

$$\exists a^{-1} \text{ such that } a \cdot a^{-1} \equiv 1 \pmod{n}$$

This inverse exists **iff**:

$$\gcd(a, n) = 1$$

11 Euclidean Algorithm and Extended Euclidean Algorithm

11.1 Euclidean Algorithm (GCD)

Theorem 11.1. For integers $a > b > 0$:

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

Worked Example

Find $\gcd(232, 124)$:

$$232 = 124 \cdot 1 + 108$$

$$124 = 108 \cdot 1 + 16$$

$$108 = 16 \cdot 6 + 12$$

$$16 = 12 \cdot 1 + 4$$

$$12 = 4 \cdot 3 + 0$$

So $\gcd(232, 124) = 4$.

11.2 Extended Euclidean Algorithm (EEA)

EEA finds integers x, y such that:

$$\gcd(a, b) = ax + by$$

This is the foundation for finding modular inverse.

Worked Example

Find inverse of 17 modulo 43. We want $17x \equiv 1 \pmod{43}$.

Euclid:

$$43 = 17 \cdot 2 + 9$$

$$17 = 9 \cdot 1 + 8$$

$$9 = 8 \cdot 1 + 1$$

$$8 = 1 \cdot 8 + 0$$

So gcd is 1.

Back-substitute:

$$1 = 9 - 8 \cdot 1$$

But $8 = 17 - 9 \cdot 1$, so:

$$1 = 9 - (17 - 9) = 2 \cdot 9 - 17$$

And $9 = 43 - 17 \cdot 2$:

$$1 = 2(43 - 17 \cdot 2) - 17 = 2 \cdot 43 - 5 \cdot 17$$

Thus:

$$-5 \cdot 17 \equiv 1 \pmod{43}$$

So:

$$17^{-1} \equiv -5 \equiv 38 \pmod{43}$$

Answer: inverse is 38.

Exam-Focused Points (Write in Answers)

Very important for exams:

- Inverse of $a \bmod n$ exists only if $\gcd(a, n) = 1$.
- Use Extended Euclid to express $1 = ax + ny$.
- Then $x \pmod{n}$ is the inverse of a .

12 Galois Fields: $GF(p)$ and $GF(2^n)$ **12.1 $GF(p)$: finite field of prime order**

If p is prime, then:

$$GF(p) = \{0, 1, 2, \dots, p-1\}$$

with addition and multiplication performed modulo p .

Classroom Note

Key reason p must be prime:

If p is prime, every nonzero element has a multiplicative inverse modulo p , so it

forms a field.

Worked Example

In $GF(7)$, find inverse of 3: We need $3x \equiv 1 \pmod{7}$. Try: $3 \cdot 5 = 15 \equiv 1 \pmod{7}$, so $3^{-1} = 5$ in $GF(7)$.

12.2 $GF(2^n)$: binary extension fields (used in AES)

In $GF(2^n)$:

- Elements are polynomials of degree $< n$ with coefficients in $\{0, 1\}$.
- Addition is XOR of coefficients.
- Multiplication is polynomial multiplication modulo an **irreducible polynomial** of degree n .

12.3 Polynomial representation

An element in $GF(2^n)$ looks like:

$$a(x) = a_{n-1}x^{n-1} + \cdots + a_1x + a_0, \quad a_i \in \{0, 1\}$$

Example in $GF(2^8)$:

$$a(x) = x^7 + x^3 + x + 1$$

This corresponds to the byte:

$$10001011_2$$

12.4 Addition in $GF(2^n)$ (XOR)

$$(a_7 \cdots a_0) + (b_7 \cdots b_0) = (a_7 \oplus b_7 \cdots a_0 \oplus b_0)$$

Worked Example

In $GF(2^8)$:

$$10110010_2 + 01100101_2 = 11010111_2$$

(just XOR).

12.5 Multiplication in $GF(2^n)$ (polynomial + modular reduction)

Steps:

- 1) Multiply as polynomials over $GF(2)$ (coefficients mod 2).
- 2) Reduce modulo an irreducible polynomial $m(x)$ of degree n .

12.6 Polynomial arithmetic over GF(2): key rules

Because coefficients are in $\{0, 1\}$:

$$1 + 1 = 0, \quad 1 + 0 = 1, \quad 0 + 0 = 0$$

So subtraction equals addition:

$$a(x) - b(x) = a(x) + b(x)$$

Worked Example

Let us work in $GF(2^4)$ with irreducible polynomial:

$$m(x) = x^4 + x + 1$$

Multiply:

$$a(x) = x^3 + x + 1 \quad (\text{binary } 1011)$$

$$b(x) = x^2 + 1 \quad (\text{binary } 0101)$$

Multiply:

$$a(x)b(x) = (x^3 + x + 1)(x^2 + 1) = x^5 + x^3 + x^3 + x + x^2 + 1$$

Combine like terms (mod 2):

$$x^5 + (x^3 + x^3) + x^2 + x + 1 = x^5 + x^2 + x + 1$$

Now reduce mod $m(x) = x^4 + x + 1$. Since:

$$x^4 \equiv x + 1 \pmod{m(x)}$$

Multiply both sides by x :

$$x^5 \equiv x(x + 1) = x^2 + x$$

So:

$$x^5 + x^2 + x + 1 \equiv (x^2 + x) + x^2 + x + 1 \equiv 1$$

Thus product is 1 in this field.

13 Quick Revision Tables

13.1 Feistel vs SPN (short table)

Feature	Feistel	SPN
Round input	Split: left/right halves	Full state/block
Main mixing	XOR with F and swap	Substitution + permutation layers
Invertibility	F need not be invertible	layers must be invertible for decryption
Decryption	Same structure, keys reversed	Uses inverse S/P layers + keys reversed
Example	DES	AES (SPN-like)

13.2 DES / 2DES / 3DES (short table)

Scheme	Keys	Important note
DES	56-bit effective	16-round Feistel, 64-bit block
Double DES	K_1, K_2	Meet-in-the-middle reduces strength (not 2^{112})
Triple DES (EDE)	K_1, K_2, K_3	Stronger, backward compatible when $K_1 = K_2 = K_3$

14 Practice Questions (Exam Style)

Very common 5 marks

1. Define Feistel cipher structure and write round equations.
2. Differentiate Feistel network and SPN.
3. Explain DES round function F (E, XOR, S-box, P).
4. What is meet-in-the-middle attack? Explain for Double DES.
5. Define group, ring, field with one example each.

Very common 10/15 marks

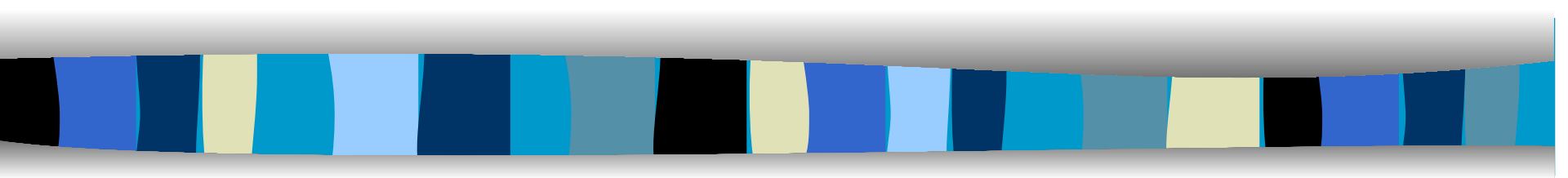
1. Explain DES structure with diagram and describe F function in detail.
2. Explain DES key schedule: PC-1, shifts, PC-2 and why circular shifts are used.
3. Explain $GF(p)$ and $GF(2^n)$ with examples of addition and multiplication.
4. Use Extended Euclidean Algorithm to find modular inverse and explain why it works.

15 References (Stallings-based)

References

- [1] William Stallings, *Cryptography and Network Security: Principles and Practice*, Pearson (latest available edition in your library/department).
- [2] NIST (historical standard), *Data Encryption Standard (DES)* / FIPS publications (DES details and tables).

Abstract Algebra



Modulo arithmetic, Group, Ring, Field

Set of Integers

- The set of integers, denoted by $\mathbb{Z}$, contains all integral numbers (with no fraction) from negative infinity to positive infinity
- $\mathbb{Z} = \{ \dots, -3, -2, -1, 0, 1, 2, 3, \dots \}$

Add:	$5 + 9 = 14$	$(-5) + 9 = 4$	$5 + (-9) = -4$	$(-5) + (-9) = -14$
Subtract:	$5 - 9 = -4$	$(-5) - 9 = -14$	$5 - (-9) = 14$	$(-5) - (-9) = +4$
Multiply:	$5 \times 9 = 45$	$(-5) \times 9 = -45$	$5 \times (-9) = -45$	$(-5) \times (-9) = 45$

Integer Division

- If we divide a by n , we can get q and r .
- The relationship between these four integers can be shown as

- $a = q \cdot n + r$

$$\begin{array}{r}
 n \longrightarrow 11 \quad \left| \begin{array}{r}
 23 \longleftarrow q \\
 \hline
 255 \longleftarrow a \\
 22 \\
 \hline
 35 \\
 33 \\
 \hline
 2 \longleftarrow r
 \end{array} \right.
 \end{array}$$

- 2) -6 (3
- 6
- -12

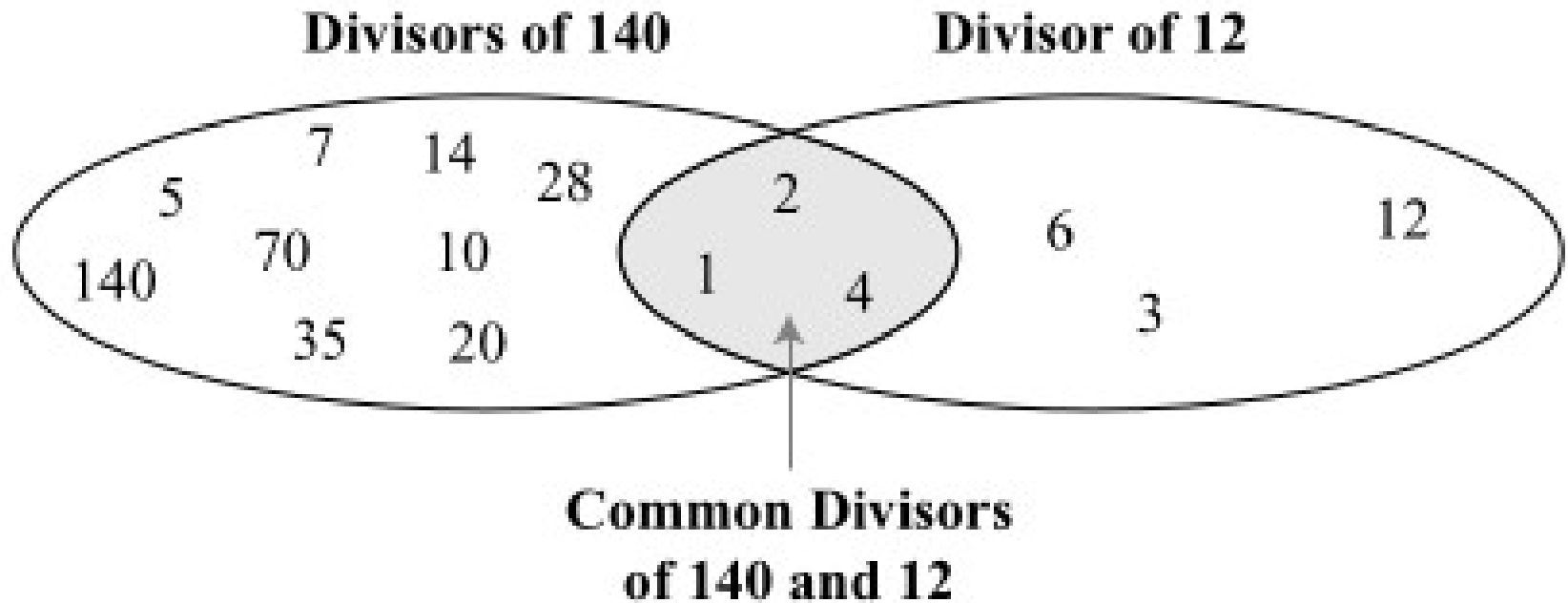
Divisibility

- In cryptography, we take
- $n > 0$
- $r \geq 0$
- a is divisible by n if $r = 0$.
- $a = nq$
- We say n divides a .
- Written as $n|a$
- $4|8$ $12|24$

Greatest Common Divisor

- The greatest common divisor of a and b is the largest integer that divides both a and b .
- We denote with $\gcd(a,b)$
- $\gcd(0, 0) = 0$
- Two positive integers may have many common divisors, but only one greatest common divisor.
- For example, the common divisors of 12 and 140 are 1, 2, and 4.
- However, the greatest common divisor is 4.

Greatest Common Divisor



Euclidean Algorithm

- Finding the greatest common divisor (gcd) of two positive integers by listing all common divisors is not practical when the two integers are large.
- Fortunately, more than 2000 years ago a mathematician named Euclid developed an algorithm that can find the greatest common divisor of two positive integers.
- The Euclidean algorithm is based on the following two facts
- **Fact 1:** $\text{gcd}(a, 0) = a$
- **Fact 2:** $\text{gcd}(a, b) = \text{gcd}(b, r)$, where r is the remainder of dividing a by b

Euclidean Algorithm

- $\gcd(36, 10) = \gcd(10, 6) = \gcd(6, 4) = \gcd(4, 2) = \gcd(2, 0) = 2$

q	r_1	r_2	r
1	2740	1760	980
1	1760	980	780
1	980	780	200
3	780	200	180
1	200	180	20
9	180	20	0
	20	0	

Euclidean Algorithm

q	r_1	r_2	r
0	25	60	25
2	60	25	10
2	25	10	5
2	10	5	0
	5	0	

Extended Euclidean Algorithm

- The greatest common divisor between two integers a and b can be expressed as the linear combination of two integers s and t .
- i. e. $\gcd(a,b) = sa + tb$
- These two integers can be found using Extended Euclidean Algorithm.
- This algorithm not only finds gcd but also two integers s and t .
- It is similar to Euclidean algorithm but it uses additional variables $s1, s2, s, t1, t2$ and t .
- $s1, s2$ are initialized to $1, 0$

Extended Euclidean Algorithm

q	a	n	r	s1	s2	$s=s1-s2*q$	t1	t2	$t=t1-t2*q$
1	252	198	54	1	0	1	0	1	-1
3	198	54	36	0	1	-3	1	-1	4
1	54	36	18	1	-3	4	-1	4	-5
2	36	18	0	-3	4	-11	4	-5	14
	18	0		4	-11	4	-5	14	-5

Linear Diophantine Equations

- One of the application of extended euclidean algorithm is to find the solutions to diophantine equations.
- Diophantine equations of two variables is $ax + by = c$.
- We need to find integer values for x and y that satisfy the equation.
- This type of equation has either no solution or an infinite number of solutions. Let $d = \gcd(a, b)$.
- If $d \mid c$, then we have an infinite number of solutions.
- One of them is called the particular solution.
- Other solutions are generic.

Linear Diophantine Equations

- **Steps for particular solution**
- Reduce the equation $ax + by = c$ to $a_1x + b_1y = c_1$ by dividing both sides by d .
- Here $a_1 = a/d$, $b_1 = b/d$, $c_1 = c/d$
- Get the values of s and t using a and b such that $a_1s + b_1t = 1$ using the extended Euclidean algorithm.
- The particular solution can be found:
- $x_0 = c_1s$ and $y_0 = c_1t$
- General solutions: $x = x_0 + k b_1$ and
- $y = y_0 - k a_1$ where k is an integer

Linear Diophantine Equations

- Find the particular and general solutions to the equation $21x + 14y = 35$.
- We have $d = \gcd(21, 14) = 7$.
- Since $7 \mid 35$, the equation has an infinite number of solutions.
- We can divide both sides by 7 to find the equation $3x + 2y = 5$.
- We get $a_1=3$, $b_1=2$ and $c_1=5$

Linear Diophantine Equations

- Now, we use extended Euclidean algorithm on $a_1(3)$ and $b_1(2)$ to find s and t such that $a_1s + b_1t = 1$.

q	a	n	r	s1	s2	s=s1-s2*q	t1	t2	t=t1-t2*q
1	3	2	1	1	0	1	0	1	-1
2	2	1	0	0	1	-2	1	-1	3
	1	0		1	-2	1	-1	3	-1

- From the table, we can see that $s=1$ and $t=-1$
- Particular Solution is $c/d=35/7=5$
- $x_0 = (c/d)s=5 \times 1 = 5$ and $y_0 = (c/d)t=5 \times (-1) = -5$

Linear Diophantine Equations

- Now, we use extended Euclidean algorithm on $a_1(3)$ and $b_1(2)$ to find s and t such that $a_1s + b_1t = 1$. We have $s = 1$ and $t = -1$.
- Particular Solution is $c/d=35/7=5$
- $x_0 = (c/d)s=5 \times 1 = 5$ and $y_0 = (c/d)t=5 \times (-1) = -5$
- $(5, -5), (7, -8), (9, -11) \dots$ are some of the solutions
- $x=x_0+kb_1 = 5+1*2=7$
- $y=y_0+ka_1=-5-3=-8$

Modulo Arithmetic

- If a is an integer and n is a positive integer, we define $a \bmod n$ to be the remainder when a is divided by n .
- For integer a , we can write the equation
- $a = q * n + r$
- This equation needs two input, a and n .
- It has two output q and r .
- In modulo arithmetic, the important output is the remainder r .
- We are not interest in quotient q .

Modulo Arithmetic

- In modulo arithmetic, we use modulo operator mod to find the remainder.
- $r = a \bmod n$
- Which means r is residue or remainder when a is divided by n .

$$a = qn + r \quad 0 \leq r < n; q = \lfloor a/n \rfloor$$

$$a = \lfloor a/n \rfloor \times n + (a \bmod n)$$

$$11 \bmod 7 = 4; \quad -11 \bmod 7 = 3$$

Modulo Arithmetic

- $27 \bmod 5 = 2$ or -3
- $36 \bmod 12 = 0$ or -12
- $-18 \bmod 14$
- $q = \text{floor}(-18/14) = -1$
- $a = qn + r$ or $-18 = -1 \cdot 14 + r$ or $r = -4$ or 12
- $-7 \bmod 10$
- $q = \text{floor}(-7/10) = 0$
- $-7 = 0 \cdot 10 + r$ or $r = -7$ or 3

Set of Residue

- The result of modulo operation with modulus n is always between 0 and $n-1$.
- $a \bmod n$ is always a nonnegative integer less than n .
- This set of nonnegative integers less than n is called set of residues
- It is represented by Z_n
- $Z_7 = \{0, 1, 2, 3, 4, 5, 6\}$
- $Z_n = \{0, 1, 2, \dots, n-1\}$

Class of Residue

- For each member of set of residue, we can have class of residue.
- For e.g. if b is member of set of residue, its class of residue is represented with $[b]$
- The class of residue $[b]$ is the set of all integers when divided by n gives b as remainder.
- For e.g. $n=5$
- Set of residue is $\{0, 1, 2, 3, 4\}$
- Residue classes are $[0], [1], [2], [3], [4]$ which are given below

Class of Residue

$$[0] = \{\dots, -15, -10, -5, 0, 5, 10, 15, \dots\}$$

$$[1] = \{\dots, -14, -9, -4, 1, 6, 11, 16, \dots\}$$

$$[2] = \{\dots, -13, -8, -3, 2, 7, 12, 17, \dots\}$$

$$[3] = \{\dots, -12, -7, -2, 3, 8, 13, 18, \dots\}$$

$$[4] = \{\dots, -11, -6, -1, 4, 9, 14, 19, \dots\}$$

Operation in set of Residue

Perform the following operations (the inputs come from $\mathbf{Z}_n$):

- a. Add 7 to 14 in $\mathbf{Z}_{15}$.
- b. Subtract 11 from 7 in $\mathbf{Z}_{13}$.
- c. Multiply 11 by 7 in $\mathbf{Z}_{20}$.

Solution

The following shows the two steps involved in each case:

$$(14 + 7) \bmod 15 \rightarrow (21) \bmod 15 = 6$$

$$(7 - 11) \bmod 13 \rightarrow (-4) \bmod 13 = 9$$

$$(7 \times 11) \bmod 20 \rightarrow (77) \bmod 20 = 17$$

Modulus property

First Property: $(a + b) \bmod n = [(a \bmod n) + (b \bmod n)] \bmod n$

Second Property: $(a - b) \bmod n = [(a \bmod n) - (b \bmod n)] \bmod n$

Third Property: $(a \times b) \bmod n = [(a \bmod n) \times (b \bmod n)] \bmod n$

$10^n \bmod x = (10 \bmod x)^n \bmod x$ Applying the third property n times.

Modulus property

$$11 \bmod 8 = 3; 15 \bmod 8 = 7$$

$$[(11 \bmod 8) + (15 \bmod 8)] \bmod 8 = 10 \bmod 8 = 2$$

$$(11 + 15) \bmod 8 = 26 \bmod 8 = 2$$

$$[(11 \bmod 8) - (15 \bmod 8)] \bmod 8 = -4 \bmod 8 = 4$$

$$(11 - 15) \bmod 8 = -4 \bmod 8 = 4$$

$$[(11 \bmod 8) \times (15 \bmod 8)] \bmod 8 = 21 \bmod 8 = 5$$

$$(11 \times 15) \bmod 8 = 165 \bmod 8 = 5$$

Modulus property

Property	Expression
Commutative Laws	$(w + x) \bmod n = (x + w) \bmod n$ $(w \times x) \bmod n = (x \times w) \bmod n$
Associative Laws	$[(w + x) + y] \bmod n = [w + (x + y)] \bmod n$ $[(w \times x) \times y] \bmod n = [w \times (x \times y)] \bmod n$
Distributive Law	$[w \times (x + y)] \bmod n = [(w \times x) + (w \times y)] \bmod n$
Identities	$(0 + w) \bmod n = w \bmod n$ $(1 \times w) \bmod n = w \bmod n$
Additive Inverse $(-w)$	For each $w \in Z_n$, there exists a z such that $w + z \equiv 0 \bmod n$

Calculating exponent modulo n

- $2^3 \bmod 5$
- $=3$
- $2^3 \bmod 5$
- $-3^3 \bmod 5$
- $-27 \bmod 5$
- $-2 \bmod 5$
- 3

Calculating exponent modulo n

- $9^{38} \bmod 5$
- $= (9 \bmod 5)^{38} \bmod 5$
- $= 4^{38} \bmod 5$
- $= -1^{38} \bmod 5$
- $= 1 \bmod 5$
- $= 1$

Calculating exponent modulo n

- $125^{107} \bmod 187$
- Binary of 107 is 1101011 that is 64, 32, 8, 2, 1
- $=125^{64} 125^{32} 125^8 125^2 125 \bmod 187$
- Lets calculate $125^2 \bmod 187 = 104$
- $125^4 \bmod 187 = 104^2 \bmod 187 = 157$
- $125^8 \bmod 187 = (125^4)^2 \bmod 187$
- $= 157^2 \bmod 187 = 152$
- $125^{16} \bmod 187 = (125^8)^2 \bmod 187$
- $= 152^2 \bmod 187 = 103$

Calculating exponent modulo n

- $125^{32} \bmod 187 = (125^{16})^2 \bmod 187$
- $= 103^2 \bmod 187 = 137$
- $125^{64} \bmod 187 = (125^{32})^2 \bmod 187$
- $= 137^2 \bmod 187 = 69$
- Finally
- $125^{64} 125^{32} 125^8 125^2 125 \bmod 187$
- $69 \times 137 \times 152 \times 104 \times 125 \bmod 187$
- $= [(69 \times 137 \bmod 187) (152 \times 104 \bmod 187)] 125 \bmod$

Calculating exponent modulo n

- $=[(69 \times 137 \bmod 187) (152 \times 104 \bmod 187)]125 \bmod 187$
- $=103 \times 100 \times 125 \bmod 187$
- $=15 \times 125 \bmod 187$
- $=5$

Calculating exponent modulo n

- $a^m \bmod n$
- is calculated as given below
- $z = 1$
- Calculate binary form b of m
- for $i = k$ down to 0 do
 - $z = z^2 \bmod n$
 - if $b_i = 1$ then $z = z \times a \bmod n$
- So final answer is z .

Calculating exponent modulo n

- $125^{107} \bmod 187$, here, $a=125$, $m=107$, $n=187$

i	b _m of m	$z=z^2 \bmod n$	$z=z*a \bmod n$
6	1	$1^2 \bmod 187=1$	$1*125 \bmod 187=125$
5	1	$125^2 \bmod 187=104$	$104*125 \bmod 187=97$
4	0	$97^2 \bmod 187=59$	59
3	1	$59^2 \bmod 187=115$	$115 * 125 \bmod 187=163$
2	0	$163^2 \bmod 187=15$	15
1	1	$15^2 \bmod 187=38$	$38*125 \bmod 187=75$
0	1	$75^2 \bmod 187=15$	$15*125 \bmod 187=5$

Calculating exponent modulo n

- $9^{38} \bmod 5$, $n=5$, $m=38$, $a=9$

i	b_i of m	$z = z^2 \bmod n$	$z = z * a \bmod n$
5	1	$1^2 \bmod 5 = 1$	$1 * 9 \bmod 5 = 4$
4	0	$4^2 \bmod 5 = 1$	1
3	0	$1^2 \bmod 5 = 1$	1
2	1	$1^2 \bmod 5 = 1$	$1 * 9 \bmod 5 = 4$
1	1	$4^2 \bmod 5 = 1$	$1 * 9 \bmod 5 = 4$
0	0	$4^2 \bmod 5 = 1$	1

Calculating exponent modulo n

- $9726^{3533} \bmod 11413 = 5761$

i	b_i	z
11	1	$1^2 \times 9726 = 9726$
10	1	$9726^2 \times 9726 = 2659$
9	0	$2659^2 = 5634$
8	1	$5634^2 \times 9726 = 9167$
7	1	$9167^2 \times 9726 = 4958$
6	1	$4958^2 \times 9726 = 7783$
5	0	$7783^2 = 6298$
4	0	$6298^2 = 4629$
3	1	$4629^2 \times 9726 = 10185$
2	1	$10185^2 \times 9726 = 105$
1	0	$105^2 = 11025$
0	1	$11025^2 \times 9726 = 5761$

Congruence

- Two integers a and b are said to be **congruent modulo n** ,
- if $(a \bmod n) = (b \bmod n)$.
- Also, if $n \mid (a-b)$ and $n \mid (b-a)$, then a and b are congruent modulo n .
- This is written as $a \equiv b \pmod{n}$
- $73 \equiv 4 \pmod{23}$;
- $21 \equiv 31 \pmod{10}$
- $21 \equiv -9 \pmod{10}$

Properties of Congruence

1. $a \equiv b \pmod{n}$ if $n \mid (a - b)$.
2. $a \equiv b \pmod{n}$ implies $b \equiv a \pmod{n}$.
3. $a \equiv b \pmod{n}$ and $b \equiv c \pmod{n}$ imply $a \equiv c \pmod{n}$.

$23 \equiv 8 \pmod{5}$	because	$23 - 8 = 15 = 5 \times 3$
$-11 \equiv 5 \pmod{8}$	because	$-11 - 5 = -16 = 8 \times (-2)$
$81 \equiv 0 \pmod{27}$	because	$81 - 0 = 81 = 27 \times 3$

Additive Inverse

- If result of addition of two integers a and b modulo n is additive identity(0) then a and b are called additive inverses of each other.
- $a+b \equiv 0 \pmod{n}$
- $0+0 \equiv 0 \pmod{10}$
- $1+9 \equiv 0 \pmod{10}$
- $2+8 \equiv 0 \pmod{10}$

Multiplicative Inverse

- Two integers a and b are multiplicative inverses of each other modulo n if
- $ab \equiv 1 \pmod{n}$
- a has multiplicative inverse modulo n only if a and n are co-prime or relatively prime.
- 8 has no multiplicative inverse modulo 10
- Since $\gcd(8,10) = 2$

Multiplicative Inverse

- Find multiplicative inverse of 3 modulo 10.
- $3 * 1 \bmod 10 = 3$
- $3 * 2 \bmod 10 = 6$
- $3 * 3 \bmod 10 = 9$
- $3 * 4 \bmod 10 = 2$
- $3 * 5 \bmod 10 = 5$
- $3 * 6 \bmod 10 = 8$
- $3 * 7 \bmod 10 = 1$
- So, 7 is multiplicative inverse of 3 modulo 10.

Multiplicative Inverse

- We can find multiplicative using extended euclidean algorithm.
- It will calculate $\gcd(n,b)$ to show that multiplicative inverse for $b \bmod n$ exists.
- For this we will use similar table as done in extended euclidean algorithm.
- we are only interested in t not in s .
- The last value of t_1 will be multiplicative inverse of b .
- We use a as n and n as b .

Multiplicative Inverse

- Find multiplicative inverse of 11 mod 26

q	n	b	r	t1	t2	t=t1-t2*q
2	26	11	4	0	1	-2
2	11	4	3	1	-2	5
1	4	3	1	-2	5	-7
3	3	1	0	5	-7	26
	1	0		-7	26	-7

- As we can see that $\gcd(26, 11) = 1$
- So, multiplicative inverse of 11 mod 26 exists.
- Its $-7 \bmod 26 = 19$
- We see that $19 * 11 \bmod 26 = 209 \bmod 26 = 1$

Multiplicative Inverse

- Find multiplicative inverse of 23 mod 100

q	n	b	r	t1	t2	t=t1-t2*q
4	100	23	8	0	1	-4
2	23	8	7	1	-4	9
1	8	7	1	-4	9	-13
7	7	1	0	9	-13	100
	1	0		-13	100	-13

- As we can see that $\gcd(100, 23) = 1$
- So, multiplicative inverse of 23 mod 100 exists.
- Its $-13 \bmod 100 = 87$
- We see that $87 * 23 \bmod 100 = 2001 \bmod 100 = 1$

Multiplicative Inverse

Example 2.27

Find the inverse of 12 in $\mathbf{Z}_{26}$.

Solution

We use a table similar to the one we used before, with $r_1 = 26$ and $r_2 = 12$.

q	r_1	r_2	r	t_1	t_2	t
2	26	12	2	0	1	-2
6	12	2	0	1	-2	13
	2	0		-2	13	

The $\gcd(26, 12) = 2 \neq 1$, which means there is no multiplicative inverse for 12 in $\mathbf{Z}_{26}$.

Addition and Multiplication Table

Addition and multiplication tables for $\mathbf{Z}_{10}$

	0	1	2	3	4	5	6	7	8	9
0	0	1	2	3	4	5	6	7	8	9
1	1	2	3	4	5	6	7	8	9	0
2	2	3	4	5	6	7	8	9	0	1
3	3	4	5	6	7	8	9	0	1	2
4	4	5	6	7	8	9	0	1	2	3
5	5	6	7	8	9	0	1	2	3	4
6	6	7	8	9	0	1	2	3	4	5
7	7	8	9	0	1	2	3	4	5	6
8	8	9	0	1	2	3	4	5	6	7
9	9	0	1	2	3	4	5	6	7	8

Addition Table in $\mathbf{Z}_{10}$

	0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6	7	8	9
2	0	2	4	6	8	0	2	4	6	8
3	0	3	6	9	2	5	8	1	4	7
4	0	4	8	2	6	0	4	8	2	6
5	0	5	0	5	0	5	0	5	0	5
6	0	6	2	8	4	0	6	2	8	4
7	0	7	4	1	8	0	2	9	6	3
8	0	8	6	4	2	0	8	6	4	2
9	0	9	8	7	6	5	4	3	2	1

Multiplication Table in $\mathbf{Z}_{10}$

Z_n and Z_n^*

- Z_n^* is subset of Z_n
- It includes the elements that have multiplicative inverse in Z_n

Some Z_n and Z_n^ sets*

$$Z_6 = \{0, 1, 2, 3, 4, 5\}$$

$$Z_6^* = \{1, 5\}$$

$$Z_7 = \{0, 1, 2, 3, 4, 5, 6\}$$

$$Z_7^* = \{1, 2, 3, 4, 5, 6\}$$

$$Z_{10} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

$$Z_{10}^* = \{1, 3, 7, 9\}$$

Z_p and Z_{p^*}

- Z_p is same to Z_n except p is prime.
- Z_{p^*} is subset of Z_p which includes elements in Z_p that has both additive and multiplicative inverse in Z_p .
- It means it includes all the elements from 1 to $p-1$
- 0 has no multiplicative inverse.
- $Z_{13} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12\}$
- $Z_{13^*} = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12\}$

Linear Congruence

- One of the application of congruence is to solve the equations.
- Let us consider the equation of the form
- $ax \equiv b \pmod{n}$.
- An equation of this type might have no solution or a limited number of solutions.
- Let $d = \gcd(a, n)$.
- If d does not divide b , there is no solution.
- If $d \mid b$, there are d solutions.

Linear Congruence

- The general solutions are $x = x_0 + k(n / d)$ for $k = 0, 1, \dots, (d - 1)$.
- Solve the equation $14x \equiv 12 \pmod{18}$.
- $d = \gcd(14 \text{ and } 18) = 2$. Since 2 divides 12, we have exactly two solutions
- $14x \equiv 12 \pmod{18} \rightarrow 7x \equiv 6 \pmod{9}$
- $x \equiv 6(7^{-1}) \pmod{9}$
- $x_0 = (6 \times 7^{-1}) \pmod{9} = (6 \times 4) \pmod{9} = 6$
- $x_1 = x_0 + 1 \times (18/2) = 15$

Figure 2.27 *Set of linear equations*

$$\begin{array}{ccccccc} a_{11}x_1 & + & a_{12}x_2 & + & \dots & + & a_{1n}x_n & \equiv & b_1 \\ a_{21}x_1 & + & a_{22}x_2 & + & \dots & + & a_{2n}x_n & \equiv & b_2 \\ \vdots & & \vdots & & & & \vdots & & \vdots \\ a_{n1}x_1 & + & a_{n2}x_2 & + & \dots & + & a_{nn}x_n & \equiv & b_n \end{array}$$

a. Equations

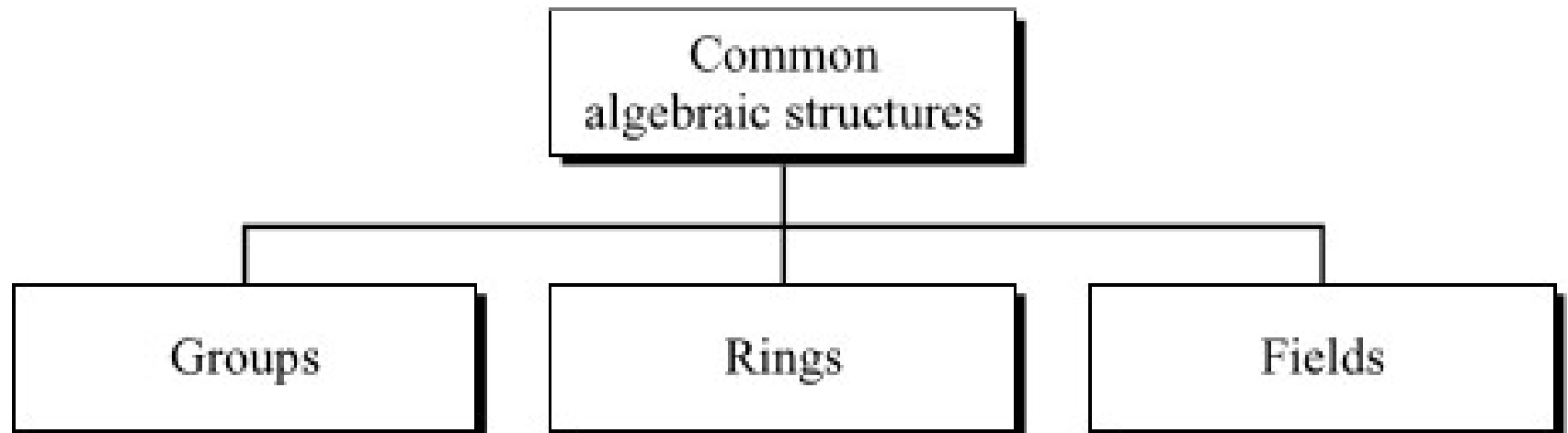
$$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \equiv \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix} \qquad \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \equiv \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{bmatrix}^{-1} \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

b. Interpretation

c. Solution

Algebraic Structures

- The non empty set together with one or more binary operations on the elements of the set that satisfies certain properties are called algebraic structures.
- $\langle G, \$ \rangle$ is a structure where G is non empty set and $\$$ is operation defined on elements of G .



Group

- An algebraic structure $\langle G, \bullet \rangle$ is said to be group if the binary operation $\bullet$ satisfies following properties for all elements a, b, c of G .
- **Associative law** $(a \bullet b) \bullet c = a \bullet (b \bullet c)$
- **Identity element e exists** $a \bullet e = e \bullet a = a$
- **For each element a in G , inverse element a^{-1} exists.** $a \bullet a^{-1} = e$
- **Closure law** $c = a \bullet b$
- Here a, b, c, e are elements of the group G .

Group

- Group can be also defined as the set of elements that is closed with respect to four different operation.
- Closed means the result of the operation is also in the set
- Closed property is related with closure law.

Group

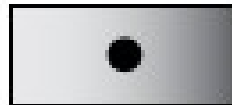
Properties

1. Closure
2. Associativity
3. Commutativity (See note)
4. Existence of identity
5. Existence of inverse

*Note:
The third property needs
to be satisfied only for a
commutative group.*

$\{a, b, c, \dots\}$

Set



Operation

Group

Group

- The set of residue integers with the addition operator, $G = \langle \mathbb{Z}_8, + \rangle$, is a group
- **Closure is satisfied.** The result of multiplying two integers in $\mathbb{Z}_8$ is another integer in $\mathbb{Z}_8$.
- **Associativity is satisfied.**
- $4 + (3 + 2) = (4 + 3) + 2$.
- **Identify element is 0.**
- $3 + 0 = 0 + 3 = 3$.
- **Inverse element of 4 is 4**
- $4 + 4 \bmod 8 = 0$ (inverse of 4 is 4)

Exponentiation of a element in group

- Let us consider a group $\langle G, \bullet \rangle$
- For any element g in G , the power of exponent of g is defined as below.
- $g^0 = e$
- $g^1 = g$
- $g^{-1} =$ inverse of g under $\bullet$
- For any integer $k \geq 2$, power is repetitive application of group operation as shown below
- $g^k = g \bullet g^{k-1}$

Exponentiation of a element in group

- For any integer $k < -1$,
- $g^{-k} = g^{-1} \cdot g^{-(k-1)}$
- $g^m \cdot g^n = g^{m+n}$
- $(g^m)^n = g^{mn}$

Exponentiation of a element in group

- Let us consider a group $G = \langle \mathbb{Z}_6, + \rangle$
- So, elements of $G = \{0, 1, 2, 3, 4, 5\}$
- Let us find the 6 power of element 2
- $2^6 = (2+2+2+2+2+2) \bmod 6 = 2$
- $3^7 = (3+3+3+3+3+3+3) \bmod 6 = 3$
- $3^0 = 0$ (identity element)

Abelian Group

- An abelian group, is a group $\langle G, \bullet \rangle$ in which elements satisfies the commutativity with respect to binary operation $\bullet$.
- Let a and b be elements of the the group.
- If $a \bullet b = b \bullet a$ then G is a abelian group.
- It is also known as commutative group.

Abelian Group

- The group $G = \langle \mathbb{Z}_8, + \rangle$ is an Abelian
- $4+5=5+4$

Abelian Group

- Show that a group $G = \langle \mathbb{Z}_n^*, \times \rangle$ is an Abelian group

Finite Group

- A group is called a **finite group** if the set has a finite number of elements; otherwise, it is an **infinite group**.
- The **order** of the group is equal to the number of elements in the group.
- Some of infinite groups are.
- the group of real numbers (with addition),
- the group of integers (with addition),
- the group of rational numbers (with addition)
- The **order** of infinite group is also infinite.

Subgroup

- A subset H of a group G is a subgroup of G if H itself is a group with respect to the operation on G .
- In other words, if $G = \langle S, \cdot \rangle$ is a group, $H = \langle T, \cdot \rangle$ is a group under the same operation
- Where, T is a nonempty subset of S
- If a and b are members of both groups, then $c = a \cdot b$ is also a member of both groups.
- Both group share the same identity element.

Subgroup

- If a is a member of both groups, the inverse of a is also a member of both groups.
- The group made of the identity element of G , $H = \langle \{e\}, \bullet \rangle$, is a subgroup of G .
- Each group is a subgroup of itself.

Subgroup

- Check if the group $H = \langle \mathbb{Z}_{10}, + \rangle$ a subgroup of the group $G = \langle \mathbb{Z}_{12}, + \rangle$?
- H is not subgroup of G because H has addition operation modulo 10
- Where as G has addition operation modulo 12.
- So, operation is different.

Subgroup

- Let us consider a group, $G = \langle \mathbb{Z}_4, + \rangle$
- One of its subgroups, $H = \langle \{0, 2\}, + \pmod{4} \rangle$
- $\{0, 2\}$ is subset of $\mathbb{Z}_4$.
- Identity element is 0 which is contained in both groups.

Cyclic group

- A group $\langle G, \bullet \rangle$ is a cyclic group if there exists a element g in G such that every element in G can be generated with different powers of g .
- Powers of g means repetitive application of group operation $\bullet$ on g .
- g is said to be generator.
- A cyclic group could have multiple generators.

Cyclic group

- Let us consider a group $G = \langle \mathbb{Z}_6, + \rangle$
- Here group operation is addition modulo 6.
- $\mathbb{Z}_6 = \{0, 1, 2, 3, 4, 5\}$
- $0^0 = 0$
- $0^1 = 0$
- $0^2 = (0+0) \bmod 6 = 0$
- So, 0 can't be a generator.
- Lets try 1 as generator

Cyclic group

- Let us consider a group $G = \langle \mathbb{Z}_6, + \rangle$
- Here group operation is addition modulo 6.
- $\mathbb{Z}_6 = \{0, 1, 2, 3, 4, 5\}$
- Lets try 1 as generator
- $1^1 = 1$
- $1^2 = (1+1) \bmod 6 = 2$
- $1^3 = (1+1+1) \bmod 6 = 3$
- $1^4 = (1+1+1+1) \bmod 6 = 4$
- $1^5 = (1+1+1+1+1) \bmod 6 = 5$
- $1^6 = (1+1+1+1+1+1) \bmod 6 = 0$

Cyclic subgroup

- The subgroup of a cyclic group is cyclic.
- Let us consider a cyclic group $G = \langle \mathbb{Z}_6, + \rangle$
- Here group operation is addition modulo 6.
- Its subgroup $\langle \{0\}, + \rangle$, $\langle \{0, 2, 4\}, + \rangle$ $\langle \{0, 3\}, + \rangle$
- Lets consider $\langle \{0, 2, 4\}, + \rangle$
- Let us use 2 as generator
- $2^1 = 2$
- $2^2 = (2 + 2) \bmod 6 = 4$
- $2^3 = (2 + 2 + 2) \bmod 6 = 0$
- So, its a cyclic group with generator 2.

Order of element of a group

- The order of an element a in a group, $\text{ord}(a)$, is the smallest positive integer n such that $a^n = e$.
- The definition can be paraphrased: the order of an element is the order of the cyclic group it generates.
- $G = \langle \mathbb{Z}_6, + \rangle$
- $\text{Ord}(0) = 0^1 = 0$ (identity element)
- $\text{Ord}(1) = 1^1 = 1$ (not identity)
- $\text{Ord}(1) = 1^2 = (1+1) \bmod 6 = 2$ (not identity)
- $\text{Ord}(1) = 1^6 = (1+1+1+1+1+1) \bmod 6 = 0$ (identity)

Ring

- A set R with two binary operations addition (+) and multiplication ($\times$) denoted as $\langle R, +, \times \rangle$ is a ring if following hold
- $\langle R, + \rangle$ is a abelian group.
- For $\times$ operation, it must satisfy **closure**, **associativity**.
- **Closure**: For a, b of R , $a \times b$ must be element of R
- **Associativity**: for elements a, b, c of R ,
- $(a \times b) \times c = a \times (b \times c)$.
- Second operator can be distributed over first
- $a \times (b + c) = (a \times b) + (a \times c)$

Commutative ring

{a,b,.....}	
Operation +	Operation x
1. Closure 2. Associativity 3. Commutativity 4. Existence of Identity element 5. Existence of inverse	1. Closure 2. Associativity
Distribution x over +	

Commutative Ring

- A ring $\langle R, +, \times \rangle$ is commutative ring if multiplication ($\times$) operation satisfies additional commutative property.

Commutative ring

{a,b,.....}	
Operation +	Operation x
1. Closure 2. Associativity 3. Commutativity 4. Existence of Identity element 5. Existence of inverse	1. Closure 2. Associativity 3. Commutativity
Distribution x over +	

Field

- A field, denoted by $F = \langle \{...\}, +, \times \rangle$ is a commutative ring in which the second operation satisfies all five properties defined for the first operation except that the identity of the addition (+) operation has no inverse with respect to multiplication ($\times$).
- $\langle \mathbb{R}, +, \times \rangle$ is a field.
- $\langle \mathbb{Q}, +, \times \rangle$ is a field.
- $\langle \mathbb{Z}_7, +, \times \rangle$ is a field

Field

{a,b,.....}	
Operation +	Operation x
1. Closure 2. Associativity 3. Commutativity 4. Existence of identity element 5. Existence of inverse	1. Closure 2. Associativity 3. Commutativity 4. Existence of identity element 5. Existence of inverse
Distribution x over +	

Finite Field

- A finite field has a finite number of elements,
- They are very extensively used in cryptography.
- Let us consider a field $\langle \mathbb{Z}_7, +, \times \rangle$ under the operations multiplication and addition.
- Set is $\mathbb{Z}_7 = \{0, 1, 2, 3, 4, 5, 6\}$
- The first operation is addition modulo 7
- Second operation is multiplication modulo 7.

Galois Fields

- The Galois Field is denoted by $GF(p^n)$ or $GF(p)$ where p is prime and $n=1$.
- Mathematician Galois showed that the order of $GF(p)$ is p .
- So, Galois Field is a finite field.
- Its set is $Z_p = \{0, 1, 2, \dots, p-1\}$
- The first operation is addition modulo p .
- The Second Operation is multiplication modulo p .
- Galois Fields $GF(p)$ are also known as prime field
- They are very extensively used in cryptography.

GF(p) Field

GF(2)

{0, 1}	+	·
--------	---	---

+	0	1
0	0	1
1	1	0

Addition

·	0	1
0	0	0
1	0	1

Multiplication

a	0	1
$-a$	1	0

a	0	1
a^{-1}	—	1

Inverses

GF(5)

{0, 1, 2, 3, 4}	+	×
-----------------	---	---

+	0	1	2	3	4
0	0	1	2	3	4
1	1	2	3	4	0
2	2	3	4	0	1
3	3	4	0	1	2
4	4	0	1	2	3

Addition

×	0	1	2	3	4
0	0	0	0	0	0
1	0	1	2	3	4
2	0	2	4	1	3
3	0	3	1	4	2
4	0	4	3	2	1

Multiplication

Additive inverse

a	0	1	2	3	4
$-a$	0	4	3	2	1

a	0	1	2	3	4
a^{-1}	—	1	3	2	4

Multiplicative inverse

Galois Fields

- Since, set of $GF(p)$ is set of integers, Both multiplication and addition modulo p satisfies closure, associative, commutative, and existence of identity.
- What we need to check is existence of inverse for each element.
- And it is shown by Galois that for prime p , every element from 0 to $p-1$ has inverse element for both operations.
- Let us inspect for field $GF(7)$ using addition and multiplication table.

Finite Field

+	0	1	2	3	4	5	6
0	0	1	2	3	4	5	6
1	1	2	3	4	5	6	0
2	2	3	4	5	6	0	1
3	3	4	5	6	0	1	2
4	4	5	6	0	1	2	3
5	5	6	0	1	2	3	4
6	6	0	1	2	3	4	5

x	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6
2	0	2	4	6	1	3	5
3	0	3	6	2	5	1	4
4	0	4	1	5	2	6	3
5	0	5	3	1	6	4	2
6	0	6	5	4	3	2	1

- The addition modulo 7 table shows that all elements have additive inverse.
- The multiplication modulo 7 table shows that all elements have multiplicative inverse except 0
- So, $\langle \mathbb{Z}_7, +, \times \rangle$ is a field.

Finite Field

- However $\langle \mathbb{Z}_8, +, \times \rangle$ is not a field as all elements of $\mathbb{Z}_8$ do not have multiplicative inverse.

$\times$	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6	7
2	0	2	4	6	0	2	4	6
3	0	3	6	1	4	7	2	5
4	0	4	0	4	0	4	0	4
5	0	5	2	7	4	1	6	3
6	0	6	4	2	0	6	4	2
7	0	7	6	5	4	3	2	1

(b) Multiplication modulo 8

Polynomial

- A polynomial of degree $n-1 \geq 0$ is an expression of the form
- $f(x) = a_{n-1}x^{n-1} + \dots + a_1x^1 + a_0x^0$
- a_i is coefficient of term x^i
- This polynomial can represent n bit binary number.
- The power of x define the position of the bit in the n -bit word.
- The coefficients are actual bits.

Polynomial

- We can denote set of coefficients with S .
- A zero-degree polynomial is called a constant polynomial.
- An n th-degree polynomial is said to be a monic polynomial if $a_n = 1$.

Polynomial Arithmetic

- Polynomial arithmetic deals with the addition, subtraction, multiplication of polynomials.
- Polynomial arithmetic can be done in three different ways.
- **Ordinary polynomial arithmetic**
- **Polynomial arithmetic over a field $\langle S, +, \times \rangle$** where S is some set to which coefficients of polynomial belongs.
- **Polynomial modulo arithmetic $\langle S, +, \times \rangle$** where S is set of polynomials.

Ordinary polynomial Arithmetic

- Polynomial arithmetic operations are defined in a natural way as defined in algebra.
- We can perform addition, multiplication, division, and subtraction of polynomials.

Ordinary Polynomial Arithmetic

As an example, let $f(x) = x^3 + x^2 + 2$ and $g(x) = x^2 - x + 1$, where S is the set of integers. Then

$$f(x) + g(x) = x^3 + 2x^2 - x + 3$$

$$f(x) - g(x) = x^3 + x + 1$$

$$f(x) \times g(x) = x^5 + 3x^2 - 2x + 2$$

Ordinary Polynomial Arithmetic

$$\begin{array}{r} x^3 + x^2 \quad + 2 \\ + (x^2 - x + 1) \\ \hline x^3 + 2x^2 - x + 3 \end{array}$$

(a) Addition

$$\begin{array}{r} x^3 + x^2 \quad + 2 \\ - (x^2 - x + 1) \\ \hline x^3 \quad + x + 1 \end{array}$$

(b) Subtraction

$$\begin{array}{r} x^3 + x^2 \quad + 2 \\ \times (x^2 - x + 1) \\ \hline x^3 + x^2 \quad + 2 \\ - x^4 - x^3 \quad - 2x \\ \hline x^5 + x^4 \quad + 2x^2 \\ \hline x^5 \quad + 3x^2 - 2x + 2 \end{array}$$

(c) Multiplication

$$\begin{array}{r} x^2 - x + 1 \overline{) x^3 + x^2 \quad + 2} \\ \underline{x^3 + x^2 \quad + x} \\ 2x^2 - x + 2 \\ \underline{2x^2 - 2x + 2} \\ x \end{array}$$

(d) Division

Polynomial Arithmetic over Ring

- We can perform polynomial arithmetic over a Ring $\langle S, +, \times \rangle$.
- S is the set to which coefficient belongs.
- All the operations addition, subtraction, multiplication and division may not be supported by the ring.
- However, All the operations are supported over Galois Field $GF(P)$

Polynomial Arithmetic GF(p)

- Given two polynomials $f(x) = (x^7 + x^5 + x^4 + x^3 + x + 1)$ and $g(x) = (x^3 + x + 1)$
- Let us perform polynomial arithmetic over a field GF(2)
- compute $f(x) + g(x)$; $f(x) - g(x)$; $f(x) * g(x)$; and $f(x)/g(x)$. Note that $g(x)|f(x)$.
- The GF(2) consists of coefficient $\{0,1\}$
- In this field addition and subtraction is XOR operation
- Multiplication is modulo 2

Polynomial Arithmetic over GF(2)

$$\begin{array}{r} x^7 \quad + x^5 + x^4 + x^3 \quad + x + 1 \\ \quad \quad \quad + (x^3 \quad + x + 1) \\ \hline x^7 \quad + x^5 + x^4 \end{array}$$

(a) Addition

$$\begin{array}{r} x^7 \quad + x^5 + x^4 + x^3 \quad + x + 1 \\ \quad \quad \quad - (x^3 \quad + x + 1) \\ \hline x^7 \quad + x^5 + x^4 \end{array}$$

Polynomial Arithmetic over GF(2)

$$\begin{array}{r}
 \begin{array}{r}
 x^7 \quad + x^5 + x^4 + x^3 \quad + x + 1 \\
 \times (x^3 \quad + x + 1) \\
 \hline
 x^7 \quad + x^5 + x^4 + x^3 \quad + x + 1 \\
 x^8 \quad + x^6 + x^5 + x^4 \quad + x^2 + x \\
 x^{10} \quad + x^8 + x^7 + x^6 \quad + x^4 + x^3 \\
 \hline
 x^{10} \quad \quad \quad + x^4 \quad + x^2 \quad + 1
 \end{array}
 \end{array}$$

(c) Multiplication

$$\begin{array}{r}
 \begin{array}{r}
 x^4 + 1 \\
 \hline
 x^3 + x + 1 \overline{) x^7 \quad + x^5 + x^4 + x^3 \quad + x + 1} \\
 \underline{x^7 \quad + x^5 + x^4} \\
 x^3 \quad + x + 1 \\
 \underline{x^3 \quad + x + 1} \\
 0
 \end{array}
 \end{array}$$

(d) Division

Perform following polynomial operations over GF(7)

- Let $f(x) = 8x^2 + 3x + 2$ and $g(x) = 2x + 1$ over the GF(7)
- Compute $f(x)+g(x)$, $f(x)-g(x)$, $f(x) * g(x)$ and $f(x)/g(x)$
- $f(x) + g(x) = 8x^2 + 3x + 2 + 2x + 1 = x^2 + 5x + 3$
- $f(x) - g(x) = 8x^2 + 3x + 2 - 2x - 1 = x^2 + x + 1$

Perform following polynomial operations over GF(7)

- Let $f(x) = 8x^2 + 3x + 2$ and $g(x) = 2x + 1$
- $f(x) * g(x) = (8x^2 + 3x + 2)(2x + 1)$
- $= 2x^3 + 6x^2 + 4x + 8x^2 + 3x + 2$
- $= 2x^3 + 2$

Perform following polynomial operations over GF(7)

2x+1	$\begin{array}{r} 8x^2 + 3x + 2 \\ 8x^2 + 4x \\ \hline \end{array}$	4x+3
	$\begin{array}{r} 6x + 2 \\ 6x + 3 \\ \hline \end{array}$	
	6	

GCD of polynomials over field

- Let $c(x)$ be gcd of two polynomials $a(x)$ and $b(x)$
- Then $c(x)$ has highest degree among all divisors of $a(x)$ and $b(x)$.
- $c(x)|a(x)$ and $c(x)|b(x)$
- $\gcd(a(x), b(x)) = \gcd(b(x), a(x) \bmod b(x))$
- So, we can use Euclidean algorithm to compute gcd of two polynomials.

GCD of polynomials over field

Euclidean Algorithm for Polynomials	
Calculate	Which satisfies
$r_1(x) = a(x) \bmod b(x)$	$a(x) = q_1(x)b(x) + r_1(x)$
$r_2(x) = b(x) \bmod r_1(x)$	$b(x) = q_2(x)r_1(x) + r_2(x)$
$r_3(x) = r_1(x) \bmod r_2(x)$	$r_1(x) = q_3(x)r_2(x) + r_3(x)$
• • •	• • •
$r_n(x) = r_{n-2}(x) \bmod r_{n-1}(x)$	$r_{n-2}(x) = q_n(x)r_{n-1}(x) + r_n(x)$
$r_{n+1}(x) = r_{n-1}(x) \bmod r_n(x) = 0$	$r_{n-1}(x) = q_{n+1}(x)r_n(x) + 0$ $d(x) = \gcd(a(x), b(x)) = r_n(x)$

GCD of polynomials over GF(2)

Find $\gcd[a(x), b(x)]$ for $a(x) = x^6 + x^5 + x^4 + x^3 + x^2 + x + 1$ and $b(x) = x^4 + x^2 + x + 1$. First, we divide $a(x)$ by $b(x)$:

$$\begin{array}{r}
 x^2 + x \\
 x^4 + x^2 + x + 1 \overline{) x^6 + x^5 + x^4 + x^3 + x^2 + x + 1} \\
 \underline{x^6 + x^4 + x^3 + x^2} \\
 x^5 + x + 1 \\
 \underline{x^5 + x^3 + x^2 + x} \\
 x^3 + x^2 + 1
 \end{array}$$

This yields $r_1(x) = x^3 + x^2 + 1$ and $q_1(x) = x^2 + x$.

Then, we divide $b(x)$ by $r_1(x)$.

$$\begin{array}{r}
 x + 1 \\
 x^3 + x^2 + 1 \overline{) x^4 + x^2 + x + 1} \\
 \underline{x^4 + x^3 + x} \\
 x^3 + x^2 + 1 \\
 \underline{x^3 + x^2 + 1} \\
 0
 \end{array}$$

This yields $r_2(x) = 0$ and $q_2(x) = x + 1$.

Therefore, $\gcd[a(x), b(x)] = r_1(x) = x^3 + x^2 + 1$.

Problems with Galois Field

- In cryptography, the fields of form $GF(p^n)$ is extensively used.
- But $GF(p^n)$ is only field when $n=1$.
- For Example $GF(2^3)$ or $GF(8)$ is not a field.
- We can make it clear using the addition and multiplication table.
- The first operation addition modulo 8 satisfies all 5 properties.
- However, the second operation multiplication modulo 8 does not satisfy inverse property.

Problem with Galois Field

Table 4.2 Arithmetic Modulo 8

+	0	1	2	3	4	5	6	7
0	0	1	2	3	4	5	6	7
1	1	2	3	4	5	6	7	0
2	2	3	4	5	6	7	0	1
3	3	4	5	6	7	0	1	2
4	4	5	6	7	0	1	2	3
5	5	6	7	0	1	2	3	4
6	6	7	0	1	2	3	4	5
7	7	0	1	2	3	4	5	6

(a) Addition modulo 8

Problem with Galois Field

- All integers in the set Z_8 does not have multiplicative inverse modulo 8 as shown below

$\times$	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6	7
2	0	2	4	6	0	2	4	6
3	0	3	6	1	4	7	2	5
4	0	4	0	4	0	4	0	4
5	0	5	2	7	4	1	6	3
6	0	6	4	2	0	6	4	2
7	0	7	6	5	4	3	2	1

(b) Multiplication modulo 8

Problem with Galois Field

- Moreover, the elements are not uniform under multiplication modulo 8.

$\times$	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6	7
2	0	2	4	6	0	2	4	6
3	0	3	6	1	4	7	2	5
4	0	4	0	4	0	4	0	4
5	0	5	2	7	4	1	6	3
6	0	6	4	2	0	6	4	2
7	0	7	6	5	4	3	2	1

(b) Multiplication modulo 8

One solution

- Such unevenly mapping makes cryptography weaker.
- One thing we can do is we can select p as the largest prime number less than 2^n .
- But, this is inefficient because we cannot use the integers from p to $2^n - 1$.
- For example, if $n = 4$, the largest prime less than 16 is 13.
- This means that we cannot use integers 13, 14, and 15.

Another Solution

- As we know $GF(p^n)$ is not a field under normal integer modulo.
- In order to make $GF(p^n)$, we define two refinements to create $GF(p^n)$
- Create set of polynomials that $GF(p^n)$ can represent.
- $GF(p^n)$ can represent polynomials of max degree $n-1$
- So, n is number of bits used to represent the coefficients of polynomial.
- Multiplication is polynomial modulo operation

Extended Galois Field

- $GF(p^n)$ is an extended galois field define over set of polynomials of max degree $n-1$ of the form
- $a_{n-1}x^{n-1} + \dots + a_1x^1 + a_0x^0$
- Where p is a prime, n is positive integer.
- Set of coefficient is **$\{0, 1, \dots, p-1\}$**
- Addition and multiplication operation is **modulo p**
- Addition of polynomials is normal.
- Multiplication of polynomials is modulo some **irreducible polynomial** of degree n .

Irreducible Polynomial

- A polynomial $f(x)$ is called **irreducible** if and only if $f(x)$ cannot be expressed as a product of two polynomials both of degree lower than that of $f(x)$.
- $f(x) = x^3 + x + 1$ is irreducible.
- $f(x) = x^4 + 1$ is reducible, because $x^4 + 1 = (x + 1)(x^3 + x^2 + x + 1)$.
- By analogy to integers, an irreducible polynomial is also called a prime polynomial.

Irreducible Polynomial

Table 4.4 *List of irreducible polynomials*

<i>Degree</i>	<i>Irreducible Polynomials</i>
1	$(x + 1), (x)$
2	$(x^2 + x + 1)$
3	$(x^3 + x^2 + 1), (x^3 + x + 1)$
4	$(x^4 + x^3 + x^2 + x + 1), (x^4 + x^3 + 1), (x^4 + x + 1)$
5	$(x^5 + x^2 + 1), (x^5 + x^3 + x^2 + x + 1), (x^5 + x^4 + x^3 + x + 1),$ $(x^5 + x^4 + x^3 + x^2 + 1), (x^5 + x^4 + x^2 + x + 1)$

Creating set of polynomials

- We need to create set of polynomials for the field $GF(p^n)$
- To create set of polynomials, first define the form of polynomial
- $a_{n-1}x^{n-1} + \dots + a_1x^1 + a_0x^0$
- Now, substitute possible values of coefficients to the polynomial.
- Let us create a polynomial for a field where $p=2$ and $n=2$.

Extended GF(2^2)

- So max possible degree of polynomials is $n-1=1$
- So, polynomials are of the form $a_1x^1+a_0$
- The set of coefficient are $\{0,1\}$ since $p=2$.
- Substituting possible values of coefficient

a0	a1	polynomial
0	0	0
0	1	1
1	0	x
1	1	x+1

Extended GF(2^2)

- So, the set of polynomials is
- $\{0, 1, x, x+1\}$
- In binary $\{00, 01, 10, 11\}$
- In decimal $\{0, 1, 2, 3\}$

Extended GF(2²)

- We xor the coefficients for addition.
- We can perform addition modulo 2 for coefficients.
- All the elements of the set have additive inverse.
- All the elements are uniform.

Polynomial table

+	0	1	x	1+x
0	0	1	x	1+x
1	1	0	1+x	x
x	x	1+x	0	1
1+x	1+x	x	1	0

Additive Table in Binary

+	00	01	10	11
00	00	01	10	11
01	01	00	11	10
10	10	11	00	01
11	11	10	01	00

Extended GF(2^2)

- For multiplicative table, remember that multiplication may result in the polynomials of degree more than $(n-1)$
- In such case, we need to perform polynomial modulo with some **irreducible polynomial** of degree n

Extended GF(2²)

- For field GF(2²), we use irreducible polynomial x^2+x+1
- Whenever degree of polynomial after multiplication is more than 1 (n-1), we perform modulo polynomial x^2+x+1

Polynomial table

x	0	1	x	1+x
0	0	0	0	0
1	0	1	x	1+x
x	0	x	1+x	1
1+x	0	1+x	1	x

Multiplicative Table in Binary

x	00	01	10	11
00	00	00	00	00
01	00	01	10	11
10	00	10	11	01
11	00	11	01	10

Extended $GF(2^2)$

- This shows that $\langle \{0,1,x,1+x\}, +, \cdot \rangle$ is a field under modular arithmetic with irreducible polynomial x^2+x+1
- So $GF(2^2)$ supports polynomial arithmetic modulo x^2+x+1

Extended GF(2^3)

		000	001	010	011	100	101	110	111
	+	0	1	2	3	4	5	6	7
000	0	0	1	2	3	4	5	6	7
001	1	1	0	3	2	5	4	7	6
010	2	2	3	0	1	6	7	4	5
011	3	3	2	1	0	7	6	5	4
100	4	4	5	6	7	0	1	2	3
101	5	5	4	7	6	1	0	3	2
110	6	6	7	4	5	2	3	0	1
111	7	7	6	5	4	3	2	1	0

(a) Addition

Extended GF(2^3)

		000	001	010	011	100	101	110	111
	$\times$	0	1	2	3	4	5	6	7
000	0	0	0	0	0	0	0	0	0
001	1	0	1	2	3	4	5	6	7
010	2	0	2	4	6	3	1	7	5
011	3	0	3	6	5	7	4	1	2
100	4	0	4	3	7	6	2	5	1
101	5	0	5	1	4	2	7	3	6
110	6	0	6	7	1	5	3	2	4
111	7	0	7	5	2	1	6	4	3

(b) Multiplication

Extended Galois Field

- Add $(x^5 + x^2 + x)$ and $(x^3 + x^2 + 1)$ in $GF(2^8)$.

$$\begin{array}{rcl} 0x^7 + 0x^6 + 1x^5 + 0x^4 + 0x^3 + 1x^2 + 1x^1 + 0x^0 & \oplus & \\ 0x^7 + 0x^6 + 0x^5 + 0x^4 + 1x^3 + 1x^2 + 0x^1 + 1x^0 & & \\ \hline 0x^7 + 0x^6 + 1x^5 + 0x^4 + 1x^3 + 0x^2 + 1x^1 + 1x^0 & \rightarrow & x^5 + x^3 + x + 1 \end{array}$$

Extended Galois Field

- Multiply polynomials $(x^5 + x^2 + x)$ and $(x^7 + x^4 + x^3 + x^2 + x)$ in $GF(2^8)$ with irreducible polynomial $(x^8 + x^4 + x^3 + x + 1)$.

$$P_1 \otimes P_2 = x^5(x^7 + x^4 + x^3 + x^2 + x) + x^2(x^7 + x^4 + x^3 + x^2 + x) + x(x^7 + x^4 + x^3 + x^2 + x)$$

$$P_1 \otimes P_2 = x^{12} + x^9 + x^8 + x^7 + x^6 + x^9 + x^6 + x^5 + x^4 + x^3 + x^8 + x^5 + x^4 + x^3 + x^2$$

$$P_1 \otimes P_2 = (x^{12} + x^7 + x^2) \bmod (x^8 + x^4 + x^3 + x + 1) = x^5 + x^3 + x^2 + x + 1$$

Operation in binary form

- For polynomials of form $GF(2^n)$, we can easily perform operation in binary form.
- For this, convert the polynomials in binary form.
- The addition and subtraction operation is XOR operation.
- Add $(x^5 + x^2 + x)$ and $(x^3 + x^2 + 1)$ in $GF(2^8)$.
- 00100110
- 00001101
- 00101011
- This is the polynomial $x^5 + x^3 + x + 1$

Multiplication in binary form

- Convert the highest degree polynomials $f(x)$ in binary form $b_{n-1}b_{n-2}\dots b_1b_0$
- Multiply it with polynomials of by 2, 4, 8 upto $n-1$
- The multiplication simply

$$x \times f(x) = \begin{cases} (b_6b_5b_4b_3b_2b_1b_00) & \text{if } b_7 = 0 \\ (b_6b_5b_4b_3b_2b_1b_00) \oplus (00011011) & \text{if } b_7 = 1 \end{cases}$$

- Now, convert $g(x)$ in binary form.
- Express $g(x)$ in xor form.
- Distribute $f(x)$ to xor of $g(x)$.

Extended Galois Field

- Multiplication in binary form

<i>Powers</i>	<i>Shift-Left Operation</i>	<i>Exclusive-Or</i>
$x^0 \otimes P_2$		10011110
$x^1 \otimes P_2$	00111100	$(00111100) \oplus (00011010) = \underline{\underline{00100111}}$
$x^2 \otimes P_2$	01001110	<u>01001110</u>
$x^3 \otimes P_2$	10011100	10011100
$x^4 \otimes P_2$	00111000	$(00111000) \oplus (00011010) = 00100011$
$x^5 \otimes P_2$	01000110	<u>01000110</u>
$P_1 \otimes P_2 = (00100111) \oplus (01001110) \oplus (01000110) = 00101111$		

Multiplication in binary form

- Multiply polynomials $(x^5 + x^2 + x)$ and $(x^7 + x^4 + x^3 + x^2 + x)$ in $GF(2^8)$ with irreducible polynomial $(x^8 + x^4 + x^3 + x + 1)$.

Powers	One poly	Multi	irreduc	result
00000001	10011110	10011110		
00000010	10011110	00111100	00011011	00100111
00000100	00100111	01001110		
00001000	01001110	10011100		
00010000	10011100	00111000	00011011	00100011
00100000	00100011	01000110		
01000000	01000110	10001100		
10000000	10001100	00011000	00011011	00000011

Multiplication in binary form

- Multiply polynomials $(x^5 + x^2 + x)$ and $(x^7 + x^4 + x^3 + x^2 + x)$ in $GF(2^8)$ with irreducible polynomial $(x^8 + x^4 + x^3 + x + 1)$.
- 00100110×10011110
- $= 10011110 (00100000 + 00000100 + 00000010)$
- $= 10011110 \times 00100000 + 10011110 \times 00000100 + 10011110 \times 00000010$
- $= 01000110 + 01001110 + 00100111$
- $= 00101111$
- $= x^5 + x^3 + x^2 + x + 1$

01000110
01001110
00100111
00101111

Multiplication in binary form

In an earlier example, we showed that for $f(x) = x^6 + x^4 + x^2 + x + 1$, $g(x) = x^7 + x + 1$, and $m(x) = x^8 + x^4 + x^3 + x + 1$, we have $f(x) \times g(x) \bmod m(x) = x^7 + x^6 + 1$. Redoing this in binary arithmetic, we need to compute $(01010111) \times (10000011)$. First, we determine the results of multiplication by powers of x :

$$(01010111) \times (00000010) = (10101110)$$

$$(01010111) \times (00000100) = (01011100) \oplus (00011011) = (01000111)$$

$$(01010111) \times (00001000) = (10001110)$$

$$(01010111) \times (00010000) = (00011100) \oplus (00011011) = (00000111)$$

$$(01010111) \times (00100000) = (00001110)$$

$$(01010111) \times (01000000) = (00011100)$$

$$(01010111) \times (10000000) = (00111000)$$

So,

$$\begin{aligned}(01010111) \times (10000011) &= (01010111) \times [(00000001) \oplus (00000010) \oplus (10000000)] \\ &= (01010111) \oplus (10101110) \oplus (00111000) = (11000001)\end{aligned}$$

which is equivalent to $x^7 + x^6 + 1$.

Multiplicative inverse

- In $GF(2^4)$, find the inverse of x^2+1 modulo x^4+x+1
- This shows that $\gcd(x^4+x+1, x^2+1)$ is 1.
- Multiplicative inverse is x^3+x+1

$q(x)$	$a(x)$	$b(x)$	$r(x)$	$t1(x)$	$t2(x)$	$t(x)=t1(x)-t2(x)*q(x)$
X^2+1	x^4+x+1	X^2+1	x	0	1	X^2+1
x	X^2+1	x	1	1	X^2+1	x^3+x+1
x	x	1	0	X^2+1	x^3+x+1	0
	1	0		x^3+x+1	0	

00011000

00011011

00000011

Construct field with generator

- We can easily define the elements of $GF(2^n)$ field using a generator.
- Every $GF(2^n)$ field has a generator g .
- If the irreducible polynomial is $f(x)$, then $f(g)=0$
- The elements of the field can be generated as
- $\{0, g, g, g^2, \dots, g^N\}$, where $N = 2^n - 2$

Construct field with generator

- We have seen that elements of a field $GF(2^2)$ are
- $\{0, 1, x, x+1\}$
- Let us generate these elements using generator g .
- The irreducible used is $f(x)=x^2+x+1$
- $N=2^2-2=4$
- Now, $f(g)=0$ or $g^2+g+1=0$ or $g^2=-g-1$
- Since subtraction is same to addition modulo 2
- $g^2=g+1$

Construct field with generator

- $g^3 = gg^2 = (g+1)g = g^2 + g = g+1+g = 1$
- $g^4 = gg^3 = g \cdot 1 = g$
- $g^5 = gg^4 = g(g) = g^2 = g+1$
- We can see that it generate all the elements of the field except 0.
- So, elements are
- $\{0, 1, g, g+1\}$

Construct field with generator

- Let us generate elements of another field $GF(2^3)$
- We know the elements are $\{0, 1, x, x+1, x^2, x^2+1, x^2+x, x^2+x+1\}$
- Let us use the irreducible polynomial $f(x)=x^3+x+1$
- Let the generator element be g
- $f(g)=g^3+x+1=0$
- Or $g^3=-x-1$
- since subtraction and addition modulo 2 is same
- Or $g^3=x+1$

Construct field with generator

- Now we will calculate next power
- $g^4 = g(g^3) = g(g+1) = g^2 + g$
- $g^5 = g(g^4) = g(g^2 + g) = g^3 + g^2 = g + 1 + g^2$
- $g^6 = g(g^5) = g(g^2 + g + 1) = g^3 + g^2 + g = g + 1 + g^2 + g = g^2 + 1$
- $g^7 = g(g^6) = g(g^2 + 1) = g^3 + 1 = g + 1 + 1 = 1 = g^0$
- So, elements of the field $GF(2^3)$ are
- $\{0, g^0, g^1, g^2, g^3, g^4, g^5, g^6\}$
- $\{0, 1, g, g^2, g+1, g^2+g, g^2+g+1, g^2+1\}$

Symmetric Ciphers

IDEA, AES

IDEA

- The International Data Encryption Algorithm (IDEA) is a symmetric-key, block cipher.
- It was published in 1991 by Lai, Massey, and Murphy .
- IDEA is a modification of the Proposed Encryption Standard (PES) that was published in 1990 by Lai and Massy.
- PES was designed as a replacement for the Data Encryption Standard (DES).

IDEA

- The algorithm was modified and published in 1991 after Biham and Shamir described the technique of differential cryptanalysis.
- The new algorithm was called the Improved Proposed Encryption Standard (IPES).
- Its name changed to IDEA in 1992.
- Today, IDEA is obsolete, but its algorithm can be a useful teaching tool to help students bridge the gap between DES, and AES.
- DES uses XOR but no algebraic operations.

IDEA

- IDEA operates with 64-bit blocks with 128-bit key
- The 64 bit plain text is divided into 4 sub-blocks, each of 16 bits in size
- The 128 bit key is used to generate 52 sub keys of length 16 bits.
- Number of identical rounds is 8 and in each round 6 keys are used.
- Like this 48 keys are used and at last round (also known as half-round) another 4 keys

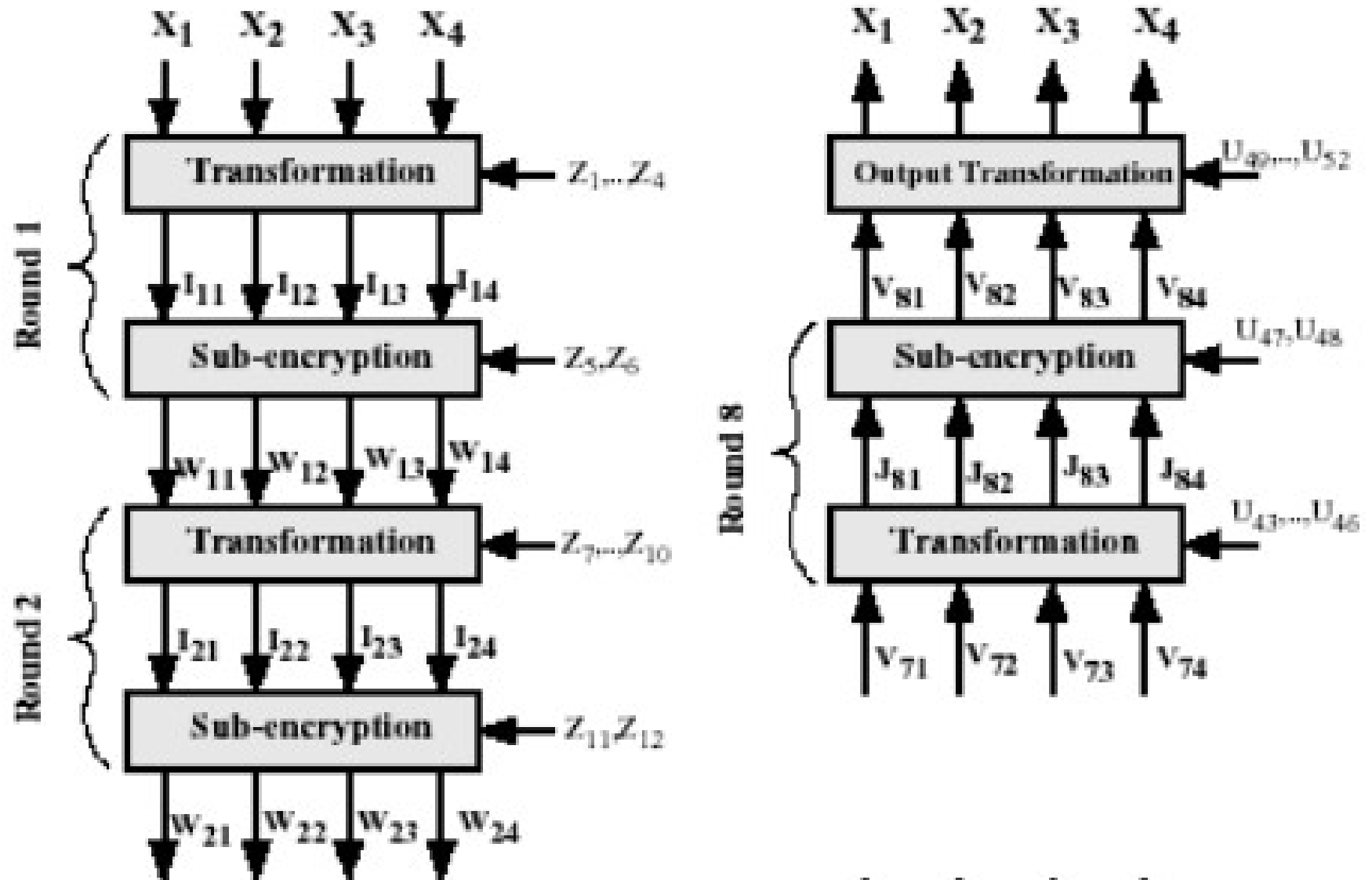
IDEA

- IDEA derives much of its security by mixing polynomial modular operations, Bitwise XOR
- The algorithm structure has been chosen such that when different key sub-blocks are used, the encryption process is identical to the decryption process

IDEA: Algorithm

- 52 keys of 16bit are generated from 128 bit initial key.
- Plaintext is divided into blocks of 64 bits.
- 64bit block is divided into 4 sub-blocks of 16bit.
- Round operation is performed for 8 rounds on sub blocks.
- Output transformation is performed to get the ciphertext.

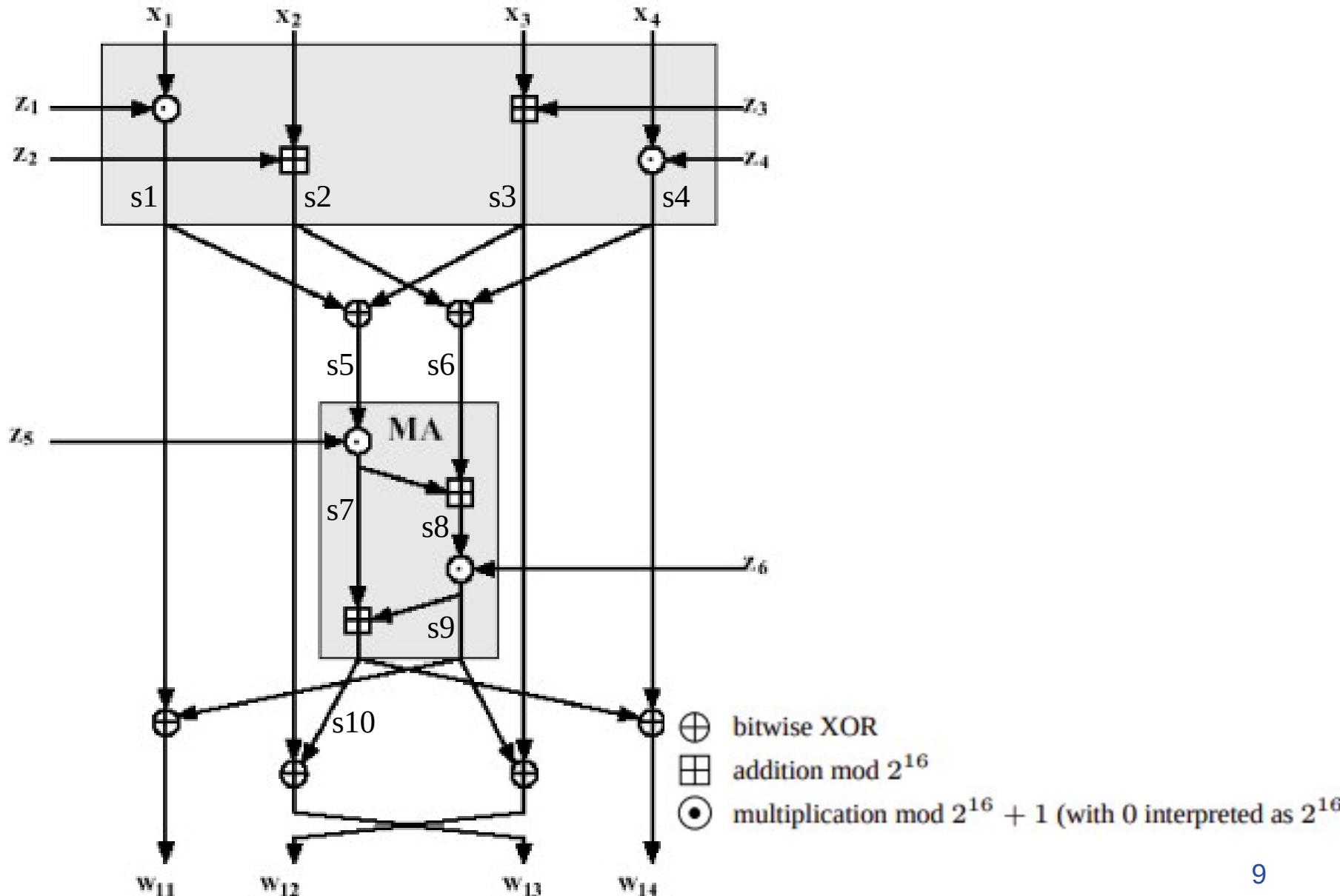
IDEA:



IDEA: Subkey generation

- Altogether 52 16-bit subkeys are generated from 128-bit encryption key.
- The first eight subkeys, labeled Z1, Z2, Z3, Z4, Z5, Z6, Z7, Z8 are taken directly from the key.
- Z1 being equal to the first 16 bits, Z2 corresponding to the next 16 bits, and so on.
- Then a circular left shift of 25 bit positions is applied to the key, and the next eight subkeys are extracted.
- This procedure is repeated until all 52 subkeys are generated.

IDEA: Round Operation



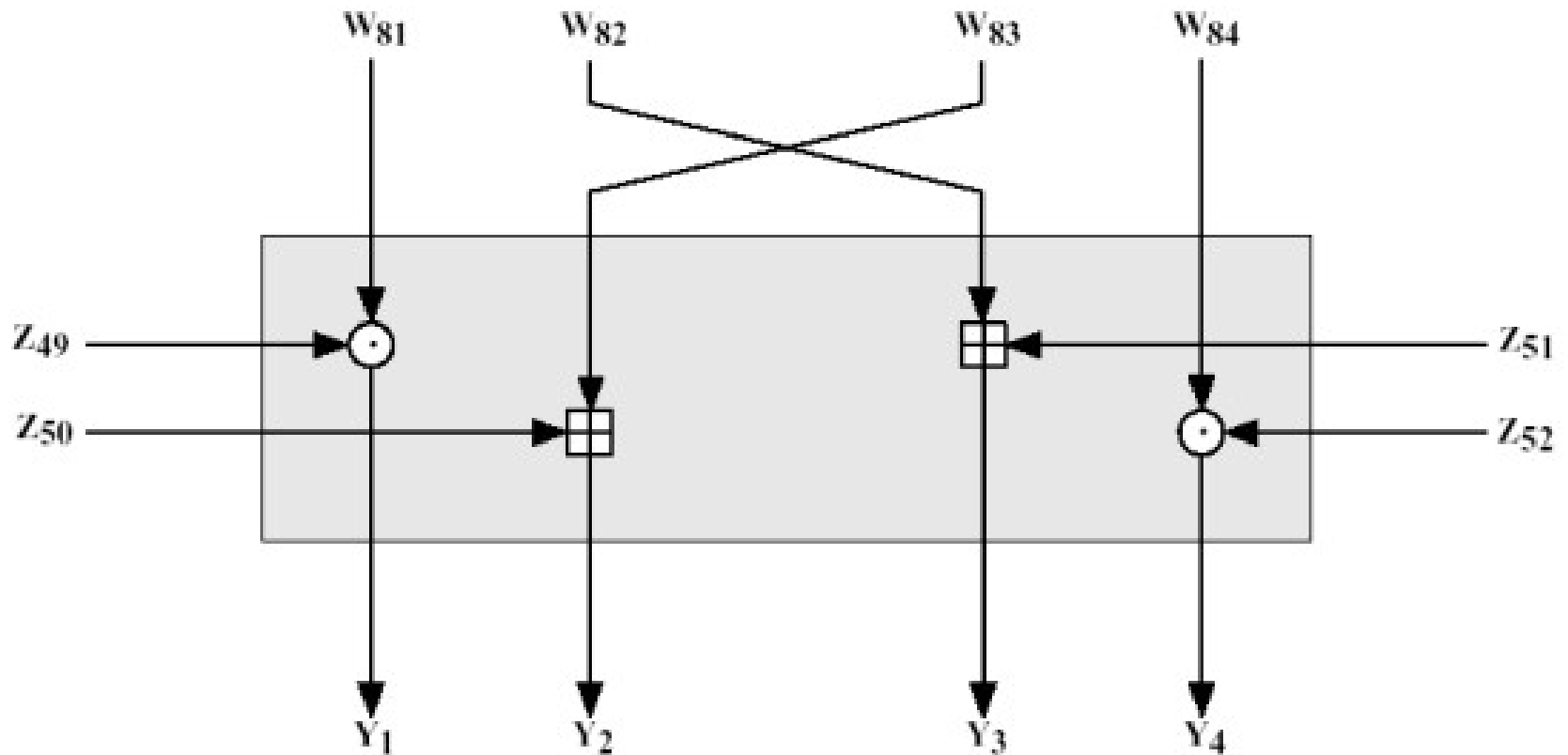
IDEA: round operation

- $s1 = x1 \times z1 \bmod 2^{16} + 1$
- $s2 = x2 + z2 \bmod 2^{16}$
- $s3 = x3 + z3 \bmod 2^{16}$
- $s4 = x4 \times z4 \bmod 2^{16} + 1$
- $s5 = s1 \text{ xor } s3$
- $s6 = s2 \text{ xor } s4$
- $s7 = s5 \times z5$
- $s8 = s6 + s7$

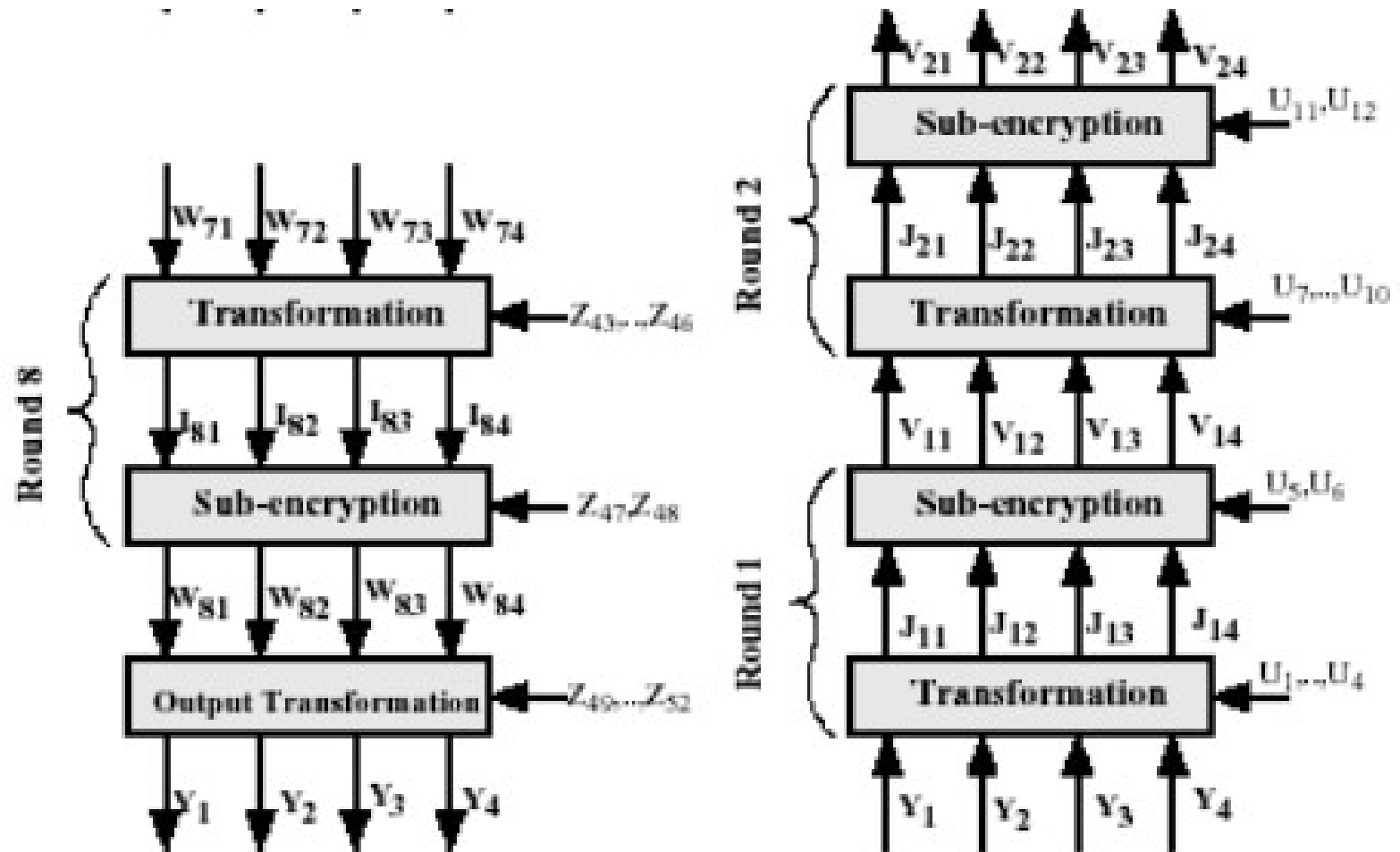
IDEA: round operation

- $s_9 = s_8 \times z_6 \bmod 2^{16} + 1$
- $s_{10} = s_9 + s_7 \bmod 2^{16}$
- $w_{11} = s_1 \text{ xor } s_9$
- $w_{12} = s_3 \text{ xor } s_9$
- $w_{13} = s_2 \text{ xor } s_{10}$
- $w_{14} = s_4 \text{ xor } s_{10}$

IDEA: Output Transformation



IDEA:



IDEA: Simplified

- Simple version of IDEA is using 16 bit plaintext and 32 bit key to generate 16 bit ciphertext.
- The plaintext is divided into 4 blocks of 4bits.
- The addition is modulo 2^{16}
- Multiplication is modulo $2^4 + 1$
- All operations are same to original version.
- For key generation left circular shift by 6 bits is performed.
- Total 28 subkeys are generated for 5 rounds

IDEA: Simplified Example

- Let us take a 32-bit key
1101 1100 0110 1111 0011 1111 0101 1001
- Left shift by 6 bits
- 0001 1011 1100 1111 1101 0110 0111 0111
- 1111 0011 1111 0101 1001 1101 1100 0110
- 1111 1101 0110 0111 0111 0001 1011 1100
- Now, 28 keys are

The 32-bit key is 1101 1100 0110 1111 0011 1111 0101 1001.

	Z_1	Z_2	Z_3	Z_4	Z_5	Z_6
Round 1	1101	1100	0110	1111	0011	1111
Round 2	0101	1001★	0001	1011	1100	1111
Round 3	1101	0110	0111	0111★	1111	0011
Round 4	1111	0101	1001	1101	1100	0110★
Round 5	1111	1101	0110	0111		

IDEA: round operation

- Let the plaintext be 1001110010101100
- The 4 blocks of plaintext are
 $x_1=1001$ $x_2=1100$ $x_3=1010$ $x_4=1100$
- $s_1=x_1 \times z_1 = 1001 \times 1101 = 1111$
- $s_2=x_1 + z_1=1100 + 1100 = 1000$
- $s_3=x_3 + z_3 = 1010 + 0110 = 0000$
- $s_4=x_4 \times z_4 = 1100 \times 1111 = 1010$
- $s_5=s_1 \text{ xor } s_3 = 1111 \text{ xor } 0000 = 1111$
- $s_6=s_2 \text{ xor } s_4 = 1000 \text{ xor } 1010 = 0010$

IDEA: round operation

- $s_7 = s_5 \times z_5 = 1111 \times 0011 = 1011$
- $s_8 = s_6 + s_7 = 0010 + 1011 = 1101$
- $s_9 = s_8 \times z_6 = 1101 \times 1111 = 1000$
- $s_{10} = s_9 + s_7 = 1000 + 1011 = 0011$
- $w_{11} = s_1 \text{ xor } s_9 = 1111 \text{ xor } 1000 = 0111$
- $w_{12} = s_3 \text{ xor } s_9 = 0000 \text{ xor } 1000 = 1000$
- $w_{13} = s_2 \text{ xor } s_{10} = 1000 \text{ xor } 0011 = 1011$
- $w_{14} = s_4 \text{ xor } s_{10} = 1010 \text{ xor } 0011 = 1001$

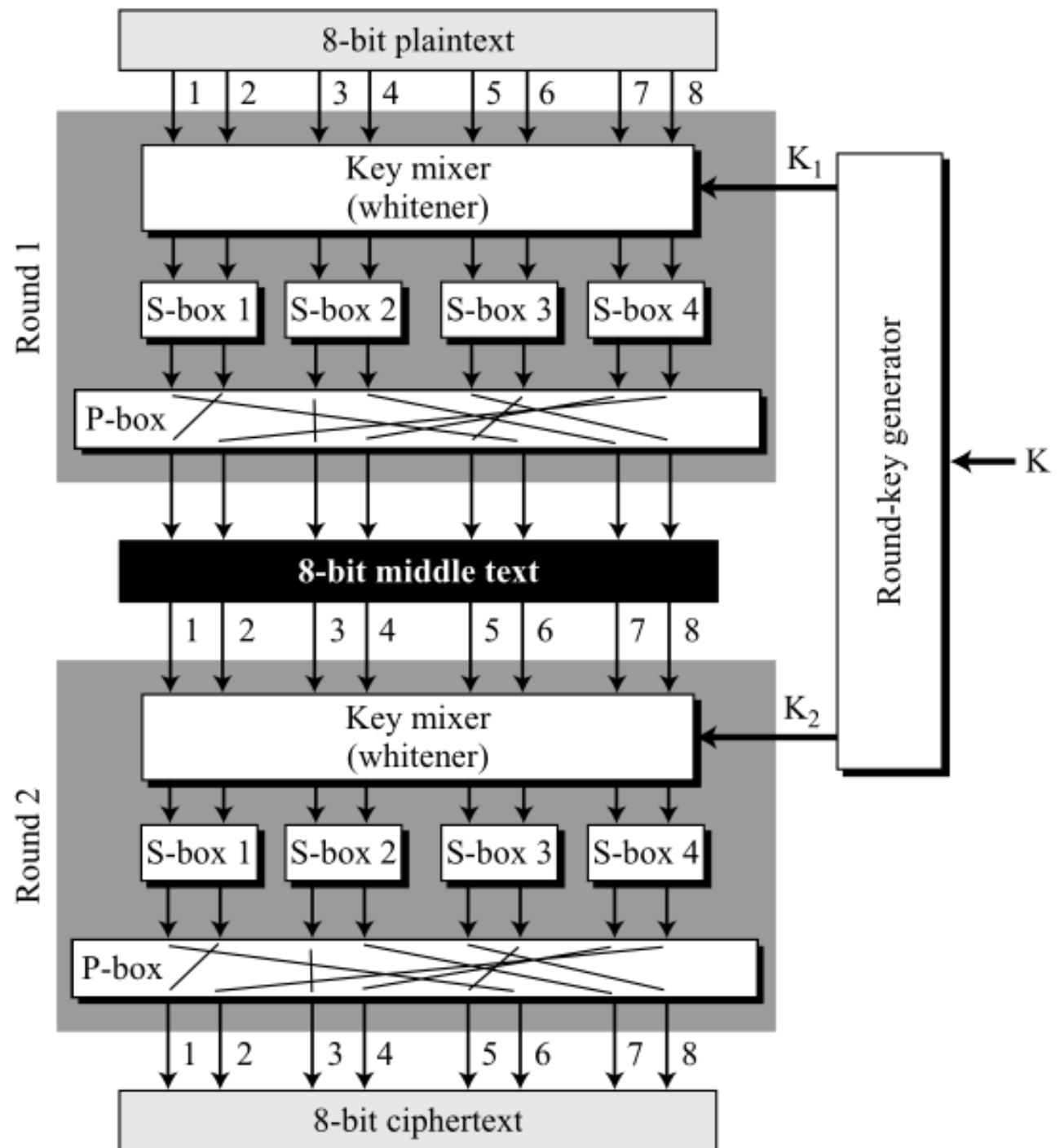
Substitution Permutation Network

- A non-Feistel cipher uses only invertible components.
- It is also known as SPN.
- A component in the encryption cipher has the corresponding component in the decryption cipher.
- For example, S-boxes need to have an equal number of inputs and outputs to be compatible.
- No compression or expansion P-boxes are allowed, because they are not invertible.

SPN

- In SPN, there is no need to divide the plaintext into two halves as we saw in the Feistel ciphers.
- AES is an example of SPN.
- Figure below is a SPN

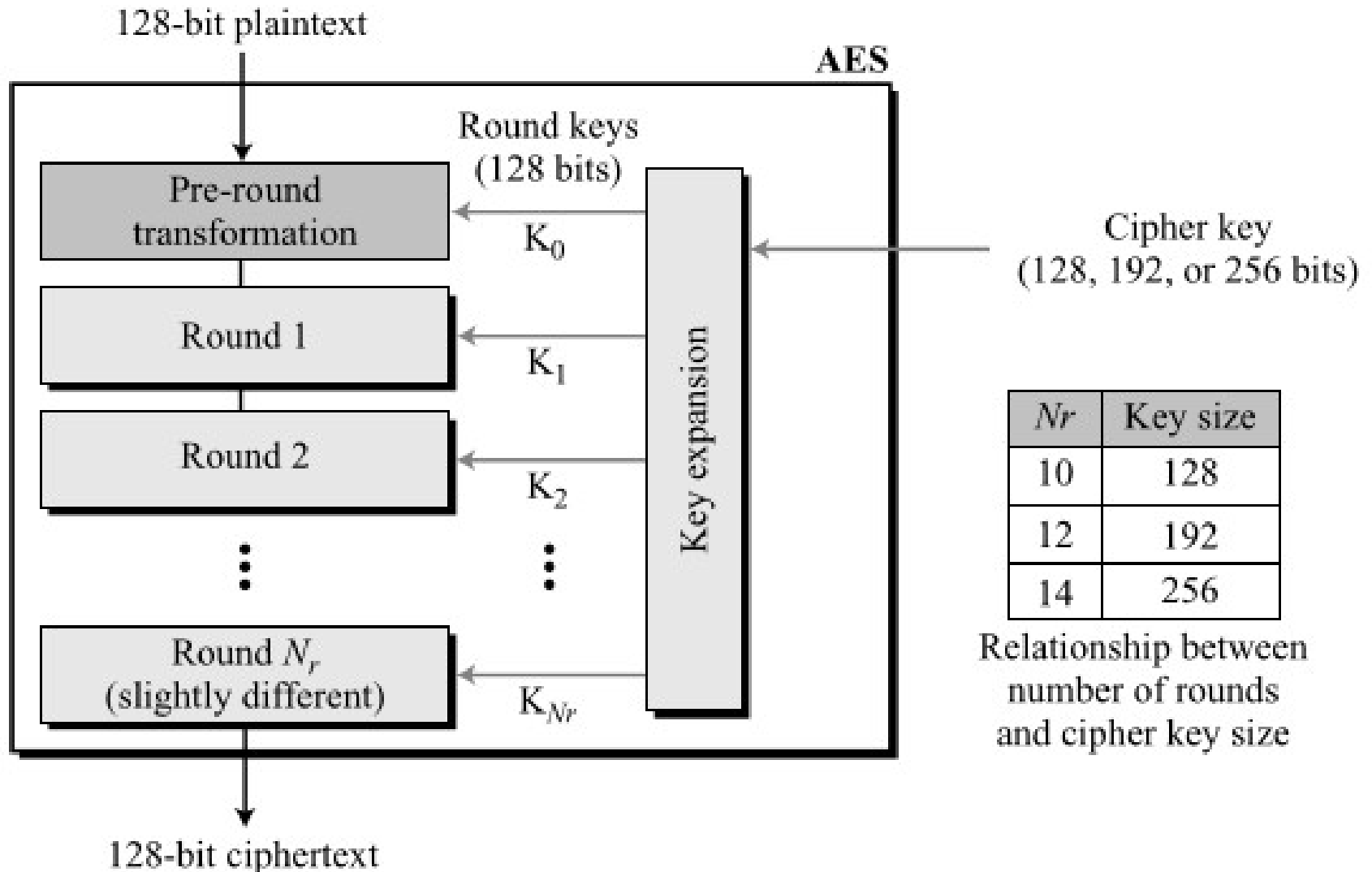
SPN



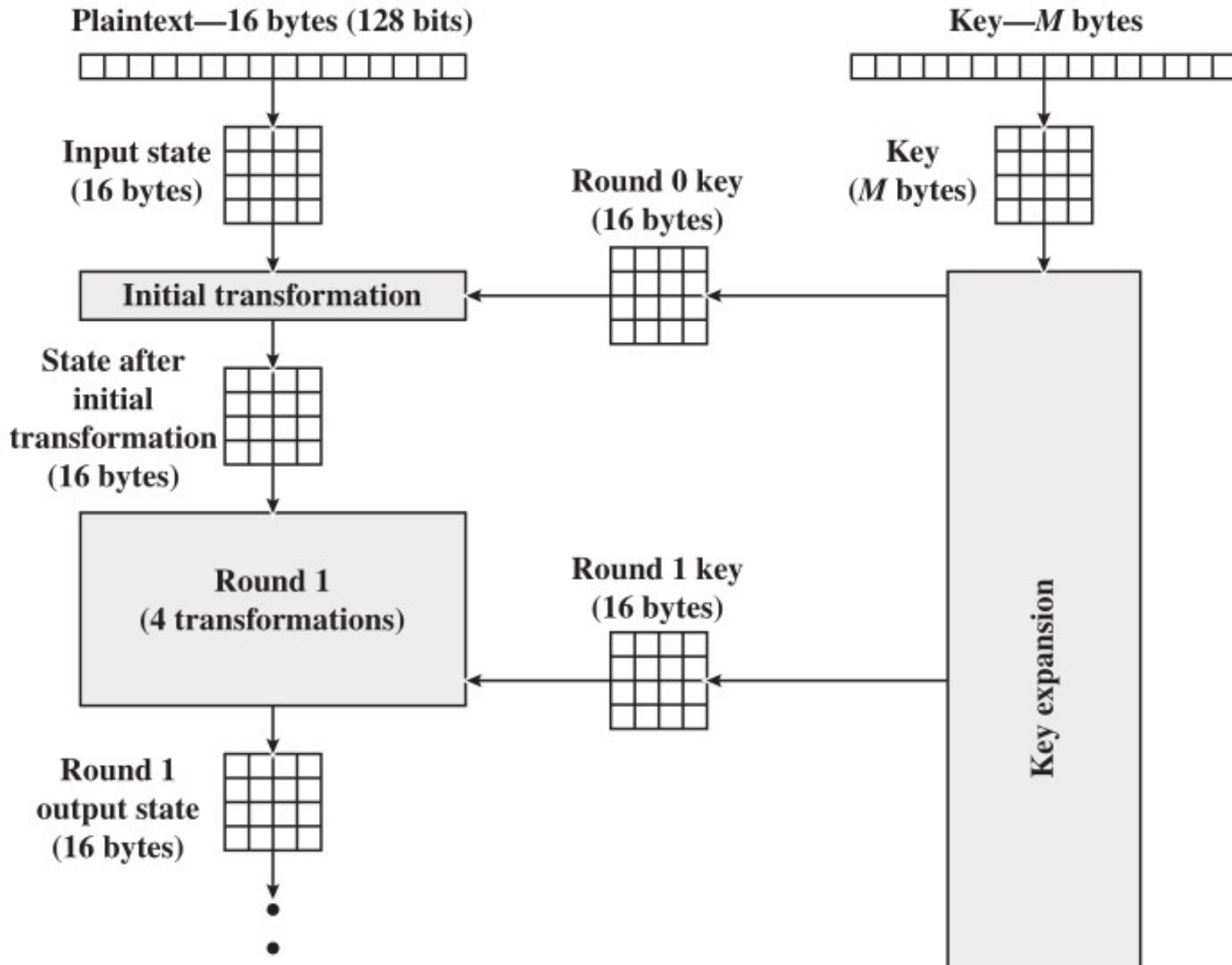
Advanced Encryption Standard

- AES was published by NIST in Dec, 2001.
- AES is a non-Feistel cipher that encrypts and decrypts a data block of 128 bits.
- It can use 10, 12, or 14 rounds with respective key sizes 128, 192, or 256 bits.
- Figure below shows the general design for the encryption algorithm
- The decryption algorithm (called inverse cipher) is similar,
- the round keys are applied in the reverse order

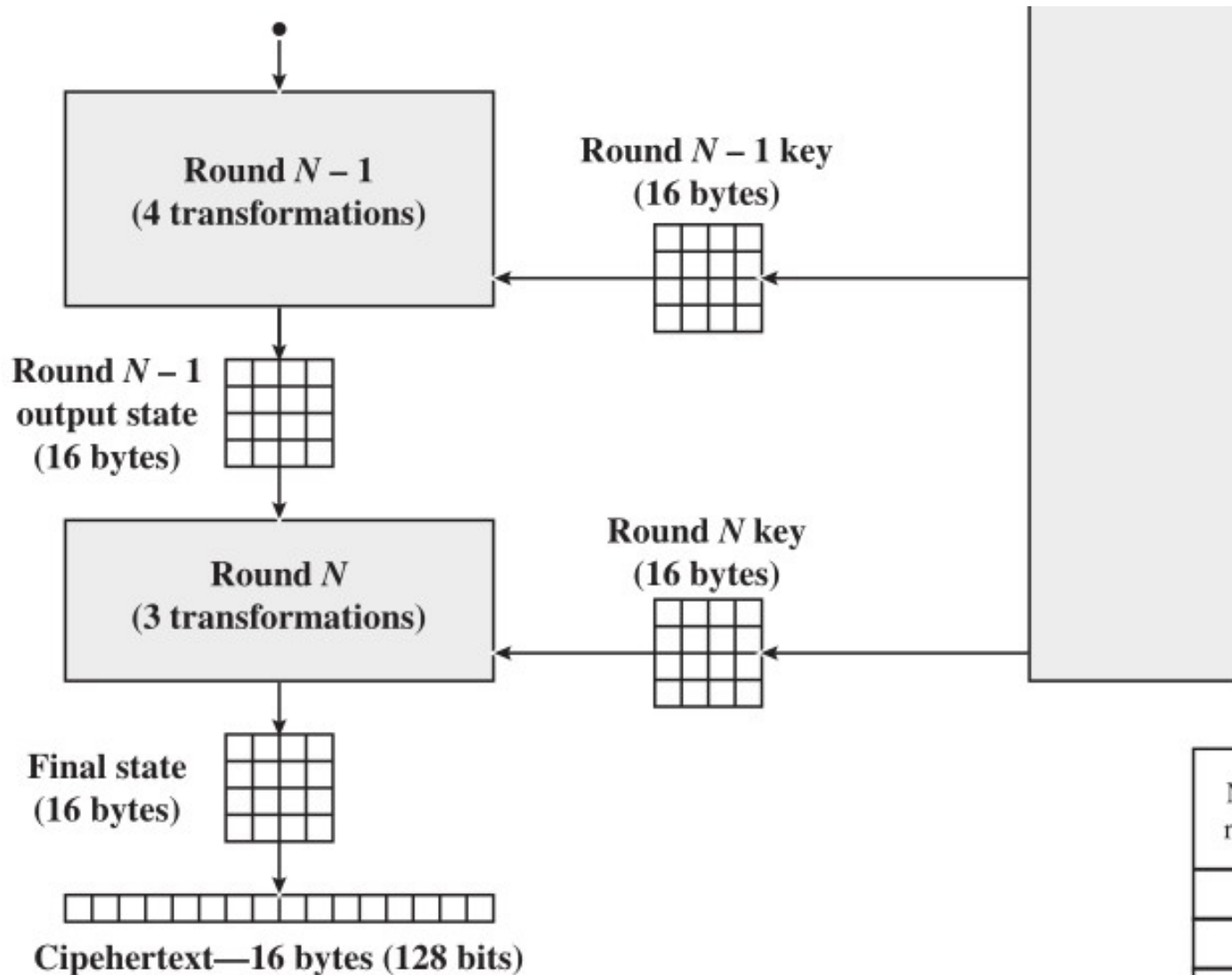
Advanced Encryption Standard



Advanced Encryption Standard



Advanced Encryption Standard



No. of rounds	Key Length (bytes)
10	16
12	24
14	32

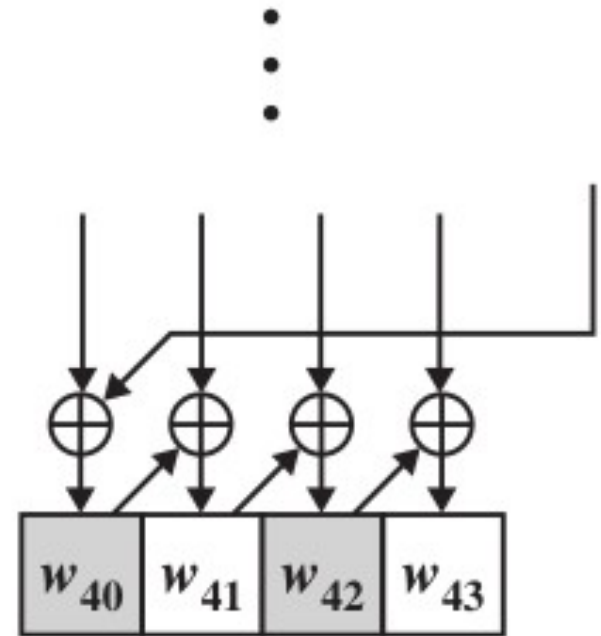
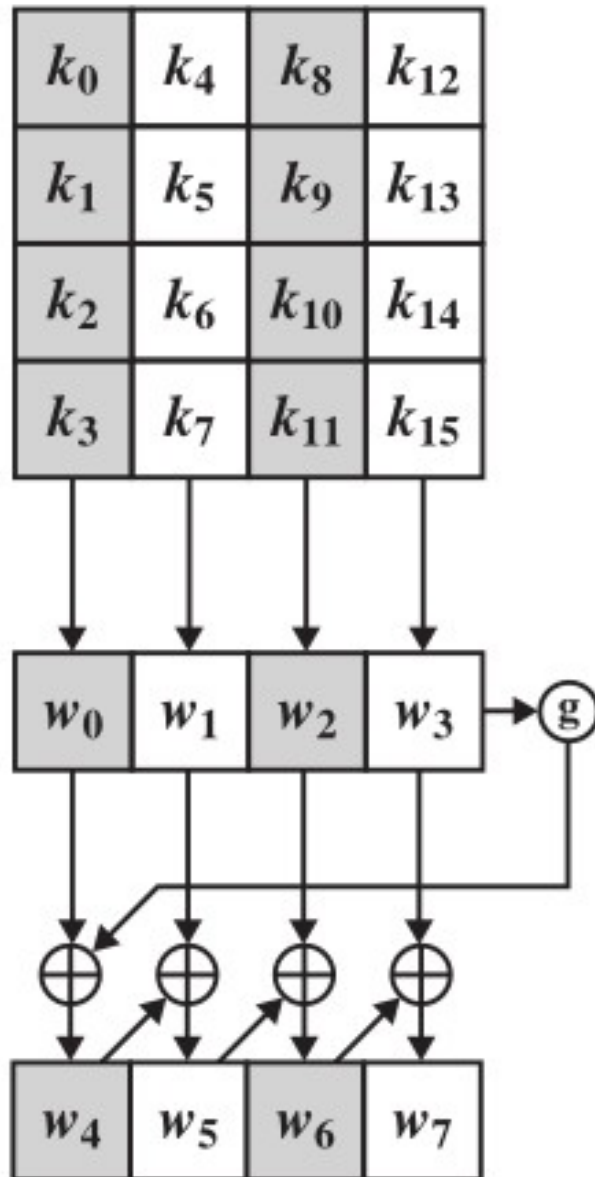
Advanced Encryption Standard

- 1. Generate round keys using key expansion algorithm
- 2. Perform initial transformation of plaintext using K_0
- 3. Perform round transformation
- The output of final round is ciphertext.
- It maintains state matrix to store different intermediate results.
- Initially state matrix consists of plaintext.
- State matrix consists of 16 bytes.

Key Expansion

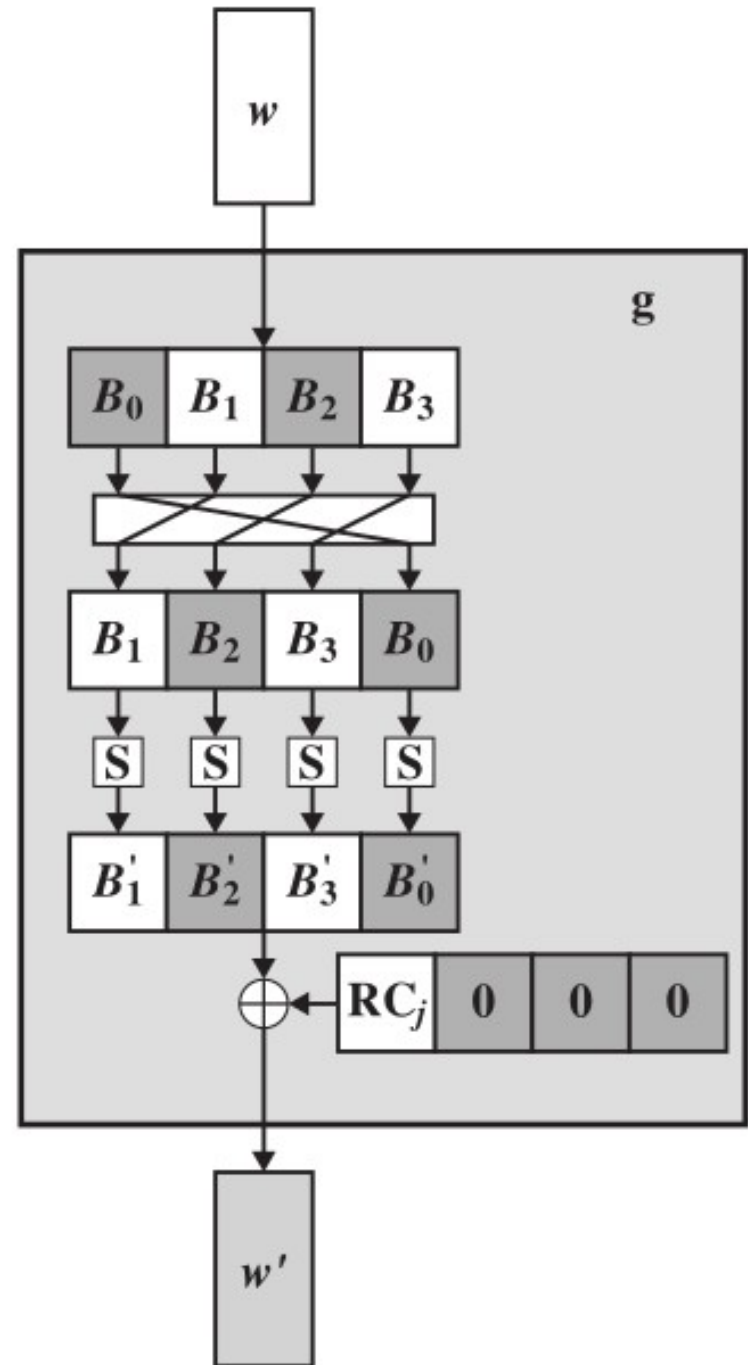
- Key expansion algorithm generates round keys from K_1 to K_{10} if $n=10$.
- It generates these keys by transforming the given key, K_0 .
- Each round keys are 128 bits or four words.
- $K_0 = (k_0, k_1, k_2, k_3, k_4, k_5, k_6, k_7, k_8, k_9, k_{10}, k_{11}, k_{12}, k_{13}, k_{14}, k_{15})$
- Each k_i is two hexadecimal number which is 8bit
- $K_0 = (w_0, w_1, w_2, w_3)$
- Where, $w_0 = (k_0, k_1, k_2, k_3)$, $w_1 = (k_4, k_5, k_6, k_7)$
- $w_2 = (k_8, k_9, k_{10}, k_{11})$, $w_3 = (k_{12}, k_{13}, k_{14}, k_{15})$

Key Expansion



Key Expansion: g function

- Last word is circular left shifted
- Each byte of the word is substituted with the bytes in S-box given below.
- The location of byte is found with the byte in word
- The first nibble in each byte represents row (x) of S-box and second nibble represents column (y)



Key Expansion: g function

Table 5.2 AES S-Boxes

		y															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
	1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
	2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
	3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
	4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
	5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
	6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
	7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
	8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
	9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
	A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
	B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
	C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
	D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
	E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
	F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

Key Expansion: g

- Let given key be
- $K_0 = 0f1571c947d9e8590cb7add6af7f6798$

Key Words	Auxiliary Function
$w_0 = 0f \ 15 \ 71 \ c9$ $w_1 = 47 \ d9 \ e8 \ 59$ $w_2 = 0c \ b7 \ ad \ d6$ $w_3 = af \ 7f \ 67 \ 98$	$RotWord(w_3) = 7f \ 67 \ 98 \ af = x_1$ $SubWord(x_1) = d2 \ 85 \ 46 \ 79 = y_1$ $Rcon(1) = 01 \ 00 \ 00 \ 00$ $y_1 \oplus Rcon(1) = d3 \ 85 \ 46 \ 79 = z_1$
$w_4 = w_0 \oplus z_1 = dc \ 90 \ 37 \ b0$ $w_5 = w_4 \oplus w_1 = 9b \ 49 \ df \ e9$ $w_6 = w_5 \oplus w_2 = 97 \ fe \ 72 \ 3f$ $w_7 = w_6 \oplus w_3 = 38 \ 81 \ 15 \ a7$	$RotWord(w_7) = 81 \ 15 \ a7 \ 38 = x_2$ $SubWord(x_2) = 0c \ 59 \ 5c \ 07 = y_2$ $Rcon(2) = 02 \ 00 \ 00 \ 00$ $y_2 \oplus Rcon(2) = 0e \ 59 \ 5c \ 07 = z_2$

- Rotword, x_1 is last word (w_3) of K_0
- Subword, y_1 is substituted word of x_1 from s-box
- Rcon is round constant word

Key Expansion: g function

- Different round constant words are given below.

Table 7.4 *RCon constants*

<i>Round</i>	<i>Constant (RCon)</i>	<i>Round</i>	<i>Constant (RCon)</i>
1	(<u>01</u> 00 00 00) ₁₆	6	(<u>20</u> 00 00 00) ₁₆
2	(<u>02</u> 00 00 00) ₁₆	7	(<u>40</u> 00 00 00) ₁₆
3	(<u>04</u> 00 00 00) ₁₆	8	(<u>80</u> 00 00 00) ₁₆
4	(<u>08</u> 00 00 00) ₁₆	9	(<u>1B</u> 00 00 00) ₁₆
5	(<u>10</u> 00 00 00) ₁₆	10	(<u>36</u> 00 00 00) ₁₆

Key Expansion

- To generate the round keys, the first word of previous round is xored with output of g function.
- This give first word of the round key.
- For second word, the second word of previous round key and first word of the current key is xored.
- In this way, all round keys are found.
- The round keys are given below

Key Expansion

Key Words	Auxiliary Function
$w_0 = 0f \ 15 \ 71 \ c9$ $w_1 = 47 \ d9 \ e8 \ 59$ $w_2 = 0c \ b7 \ ad \ d6$ $w_3 = af \ 7f \ 67 \ 98$	$RotWord(w_3) = 7f \ 67 \ 98 \ af = x_1$ $SubWord(x_1) = d2 \ 85 \ 46 \ 79 = y_1$ $Rcon(1) = 01 \ 00 \ 00 \ 00$ $y_1 \oplus Rcon(1) = d3 \ 85 \ 46 \ 79 = z_1$
$w_4 = w_0 \oplus z_1 = dc \ 90 \ 37 \ b0$ $w_5 = w_4 \oplus w_1 = 9b \ 49 \ df \ e9$ $w_6 = w_5 \oplus w_2 = 97 \ fe \ 72 \ 3f$ $w_7 = w_6 \oplus w_3 = 38 \ 81 \ 15 \ a7$	$RotWord(w_7) = 81 \ 15 \ a7 \ 38 = x_2$ $SubWord(x_2) = 0c \ 59 \ 5c \ 07 = y_2$ $Rcon(2) = 02 \ 00 \ 00 \ 00$ $y_2 \oplus Rcon(2) = 0e \ 59 \ 5c \ 07 = z_2$
$w_8 = w_4 \oplus z_2 = d2 \ c9 \ 6b \ b7$ $w_9 = w_8 \oplus w_5 = 49 \ 80 \ b4 \ 5e$ $w_{10} = w_9 \oplus w_6 = de \ 7e \ c6 \ 61$ $w_{11} = w_{10} \oplus w_7 = e6 \ ff \ d3 \ c6$	$RotWord(w_{11}) = ff \ d3 \ c6 \ e6 = x_3$ $SubWord(x_3) = 16 \ 66 \ b4 \ 83 = y_3$ $Rcon(3) = 04 \ 00 \ 00 \ 00$ $y_3 \oplus Rcon(3) = 12 \ 66 \ b4 \ 8e = z_3$
$w_{12} = w_8 \oplus z_3 = c0 \ af \ df \ 39$ $w_{13} = w_{12} \oplus w_9 = 89 \ 2f \ 6b \ 67$ $w_{14} = w_{13} \oplus w_{10} = 57 \ 51 \ ad \ 06$ $w_{15} = w_{14} \oplus w_{11} = b1 \ ae \ 7e \ c0$	$RotWord(w_{15}) = ae \ 7e \ c0 \ b1 = x_4$ $SubWord(x_4) = e4 \ f3 \ ba \ c8 = y_4$ $Rcon(4) = 08 \ 00 \ 00 \ 00$ $y_4 \oplus Rcon(4) = ec \ f3 \ ba \ c8 = 4$

Key Expansion

Key Words	Auxiliary Function
$w_{16} = w_{12} \oplus z_4 = 2c\ 5c\ 65\ f1$ $w_{17} = w_{16} \oplus w_{13} = a5\ 73\ 0e\ 96$ $w_{18} = w_{17} \oplus w_{14} = f2\ 22\ a3\ 90$ $w_{19} = w_{18} \oplus w_{15} = 43\ 8c\ dd\ 50$	$RotWord(w_{19}) = 8c\ dd\ 50\ 43 = x_5$ $SubWord(x_5) = 64\ c1\ 53\ 1a = y_5$ $Rcon(5) = 10\ 00\ 00\ 00$ $y_5 \oplus Rcon(5) = 74\ c1\ 53\ 1a = z_5$
$w_{20} = w_{16} \oplus z_5 = 58\ 9d\ 36\ eb$ $w_{21} = w_{20} \oplus w_{17} = fd\ ee\ 38\ 7d$ $w_{22} = w_{21} \oplus w_{18} = 0f\ cc\ 9b\ ed$ $w_{23} = w_{22} \oplus w_{19} = 4c\ 40\ 46\ bd$	$RotWord(w_{23}) = 40\ 46\ bd\ 4c = x_6$ $SubWord(x_6) = 09\ 5a\ 7a\ 29 = y_6$ $Rcon(6) = 20\ 00\ 00\ 00$ $y_6 \oplus Rcon(6) = 29\ 5a\ 7a\ 29 = z_6$
$w_{24} = w_{20} \oplus z_6 = 71\ c7\ 4c\ c2$ $w_{25} = w_{24} \oplus w_{21} = 8c\ 29\ 74\ bf$ $w_{26} = w_{25} \oplus w_{22} = 83\ e5\ ef\ 52$ $w_{27} = w_{26} \oplus w_{23} = cf\ a5\ a9\ ef$	$RotWord(w_{27}) = a5\ a9\ ef\ cf = x_7$ $SubWord(x_7) = 06\ d3\ bf\ 8a = y_7$ $Rcon(7) = 40\ 00\ 00\ 00$ $y_7 \oplus Rcon(7) = 46\ d3\ df\ 8a = z_7$
$w_{28} = w_{24} \oplus z_7 = 37\ 14\ 93\ 48$ $w_{29} = w_{28} \oplus w_{25} = bb\ 3d\ e7\ f7$ $w_{30} = w_{29} \oplus w_{26} = 38\ d8\ 08\ a5$ $w_{31} = w_{30} \oplus w_{27} = f7\ 7d\ a1\ 4a$	$RotWord(w_{31}) = 7d\ a1\ 4a\ f7 = x_8$ $SubWord(x_8) = ff\ 32\ d6\ 68 = y_8$ $Rcon(8) = 80\ 00\ 00\ 00$ $y_8 \oplus Rcon(8) = 7f\ 32\ d6\ 68 = z_8$

Key Expansion

$w_{32} = w_{28} \oplus z_8 = 48\ 26\ 45\ 20$ $w_{33} = w_{32} \oplus w_{29} = f3\ 1b\ a2\ d7$ $w_{34} = w_{33} \oplus w_{30} = cb\ c3\ aa\ 72$ $w_{35} = w_{34} \oplus w_{32} = 3c\ be\ 0b\ 3$	$RotWord(w_{35}) = be\ 0b\ 38\ 3c = x_9$ $SubWord(x_9) = ae\ 2b\ 07\ eb = y_9$ $Rcon(9) = 1B\ 00\ 00\ 00$ $y_9 \oplus Rcon(9) = b5\ 2b\ 07\ eb = z_9$
$w_{36} = w_{32} \oplus z_9 = fd\ 0d\ 42\ cb$ $w_{37} = w_{36} \oplus w_{33} = 0e\ 16\ e0\ 1c$ $w_{38} = w_{37} \oplus w_{34} = c5\ d5\ 4a\ 6e$ $w_{39} = w_{38} \oplus w_{35} = f9\ 6b\ 41\ 56$	$RotWord(w_{39}) = 6b\ 41\ 56\ f9 = x_{10}$ $SubWord(x_{10}) = 7f\ 83\ b1\ 99 = y_{10}$ $Rcon(10) = 36\ 00\ 00\ 00$ $y_{10} \oplus Rcon(10) = 49\ 83\ b1\ 99 = z_{10}$
$w_{40} = w_{36} \oplus z_{10} = b4\ 8e\ f3\ 52$ $w_{41} = w_{40} \oplus w_{37} = ba\ 98\ 13\ 4e$ $w_{42} = w_{41} \oplus w_{38} = 7f\ 4d\ 59\ 20$ $w_{43} = w_{42} \oplus w_{39} = 86\ 26\ 18\ 76$	

Initial Transformation

- This step is also known as add round key.
- Block size of 128 bits are created from plaintext.
- $P = (p_0, p_1, p_2, p_3, p_4, p_5, p_6, p_7, p_8, p_9, p_{10}, p_{11}, p_{12}, p_{13}, p_{14}, p_{15})$
- $K_0 = (k_0, k_1, k_2, k_3, k_4, k_5, k_6, k_7, k_8, k_9, k_{10}, k_{11}, k_{12}, k_{13}, k_{14}, k_{15})$
- $P + K_0 = p_0 + k_0, p_1 + k_1, \dots, p_{15} + k_{15}$
- 0123456789abcdeffedcba9876543210 (plaintext)

P					K ₀					Result			
01	89	fe	76	+	0f	47	0c	af	=	0e	ce	f2	d9
23	ab	dc	54		15	d9	b7	7f		36	72	6b	2b
45	cd	ba	32		71	e8	ad	67		34	25	17	55
67	ef	98	10		c9	59	d6	98		ae	b6	4e	88

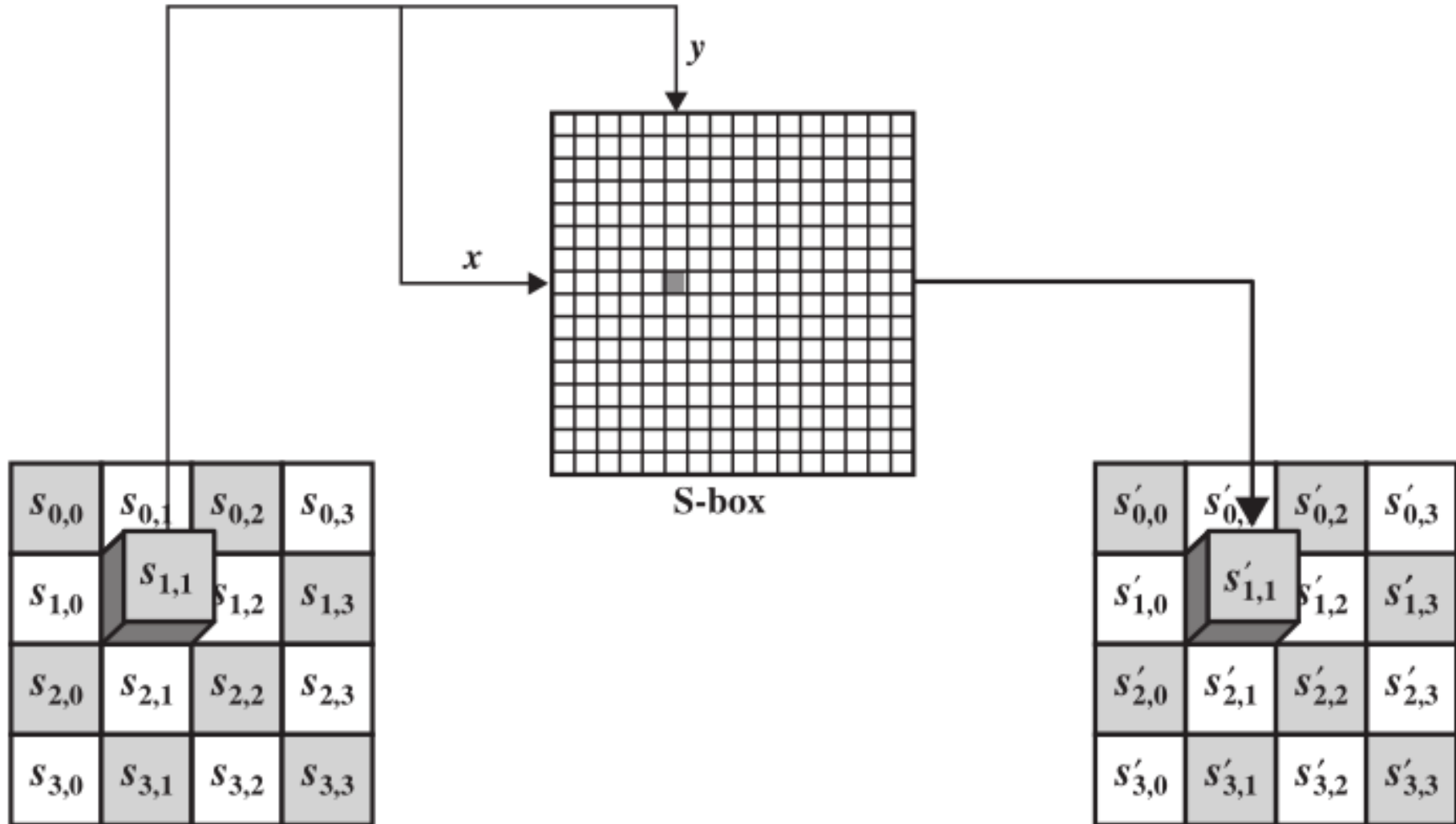
Round Transformation

- AES can be implemented in 10, 12 or 14 rounds.
- Each round consists of following transformations for generating strong ciphers.
- SubBytes
- ShiftRows
- MixColumns
- AddRoundKey
- Last round does not have mix column transformation.

Round Transformation

- **SubBytes**
- This operations is also called substitute byte transformation.
- The each byte in the state is mapped to new byte from the S-box.
- AES defines (16,16) S-box matrix of bytes.
- The first value in state byte is x values and second is y value.
- Now corresponding byte in S-box located at x row and y column is substituted.

Round Transformation: SubBytes



Round Transformation: SubBytes

- The state after initial transformation is

0e	ce	f2	d9
36	72	6b	2b
34	25	17	55
ae	b6	4e	88

- The first byte in this state is 0e
- It means row is 0 and column is e
- Now the byte is 0 row and e column in S-box is ab.
- So, 0e is mapped to ab.
- Similarly, we can map other 15 bytes.

Round Transformation: SubBytes

After initial
transformation

0e	ce	f2	d9
36	72	6b	2b
34	25	17	55
ae	b6	4e	88



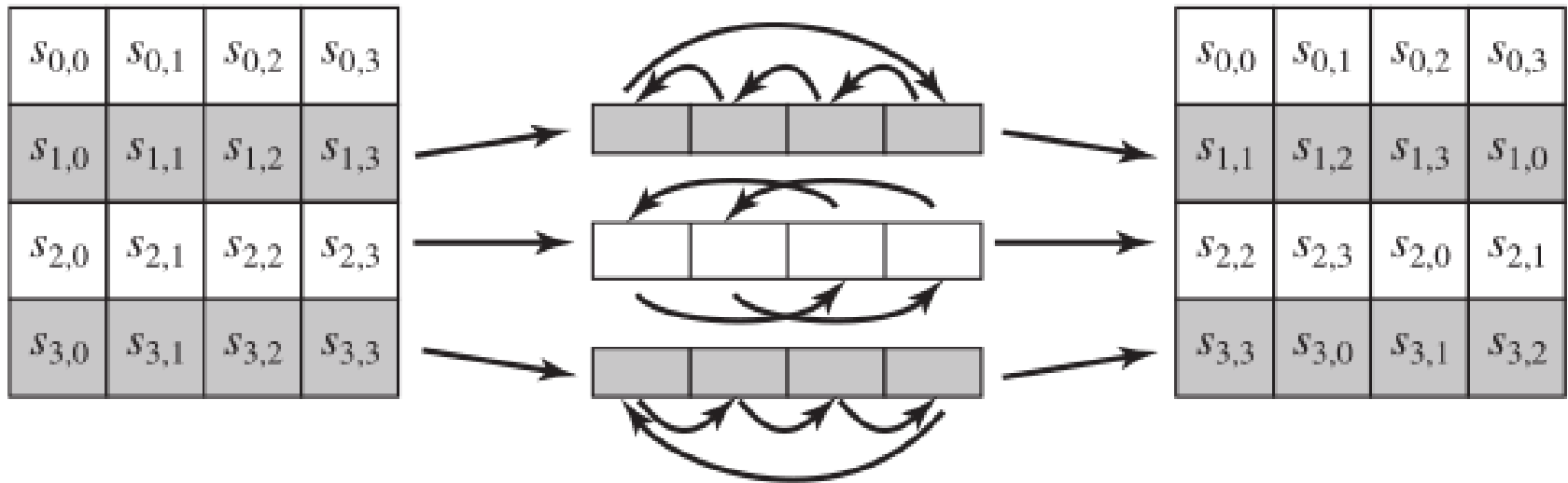
After SubBytes

ab	8b	89	35
05	40	7f	f1
18	3f	f0	fc
e4	4e	2f	c4

Round Transformation: ShiftRows

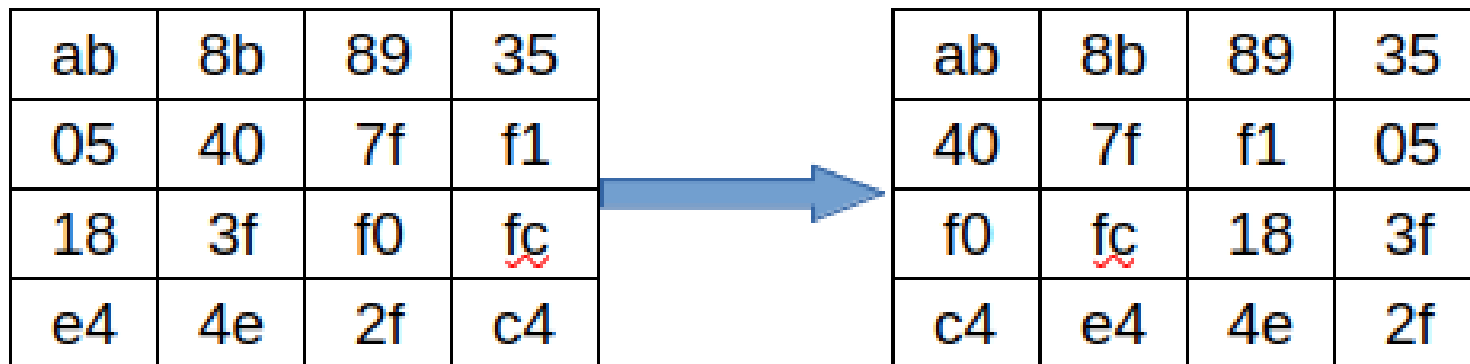
- Note that for convenience, the state is arranged in (4,4) byte matrix.
- The first column is first word, second column is second word and so on.
- In ShiftRow transformation, first row of the state is not altered.
- Second row is circular left shifted by 1 byte.
- Third row is circular left shifted by 2 bytes.
- Fourth row is left shifted by 3 bytes.

Round Transformation: ShiftRows



After SubBytes

After ShiftRows



Round Transformation MixColumn

- At this stage, the state matrix consists of the result of RowShift.
- Each byte of a column in state matrix is mapped into a new value that is a function of all four bytes in that column.
- The transformation can be defined by the following matrix multiplication on State

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} = \begin{bmatrix} s'_{0,0} & s'_{0,1} & s'_{0,2} & s'_{0,3} \\ s'_{1,0} & s'_{1,1} & s'_{1,2} & s'_{1,3} \\ s'_{2,0} & s'_{2,1} & s'_{2,2} & s'_{2,3} \\ s'_{3,0} & s'_{3,1} & s'_{3,2} & s'_{3,3} \end{bmatrix}$$

Round Transformation MixColumn

- First matrix is constant matrix and second is state matrix.
- Note that for convenience, the state is arranged in (4,4) byte matrix.
- The first column is first word, second column is second word and so on.
- Constant matrix is designed in such a way that the each column of state matrix is transformed to a new value that is a function of all four bytes in that column.
- Here matrix multiplication is a field operation in $GF(2^8)$ with prime polynomial $x^8 + x^4 + x^3 + x + 1$.⁴⁵

Round Transformation MixColumn

- Multiplication of bytes is done in $GF(2^8)$ with modulus $x^8 + x^4 + x^3 + x + 1$.
- Addition is the same as XORing of 8-bit words.
- So, the numbers in constant matrix are polynomials.
- It means 03 in constant matrix is 00000011 which is $x+1$ and so on.
- After matrix multiplication, we get following equations

Round Transformation MixColumn

$$s'_{0,j} = (2 \cdot s_{0,j}) \oplus (3 \cdot s_{1,j}) \oplus s_{2,j} \oplus s_{3,j}$$

$$s'_{1,j} = s_{0,j} \oplus (2 \cdot s_{1,j}) \oplus (3 \cdot s_{2,j}) \oplus s_{3,j}$$

$$s'_{2,j} = s_{0,j} \oplus s_{1,j} \oplus (2 \cdot s_{2,j}) \oplus (3 \cdot s_{3,j})$$

$$s'_{3,j} = (3 \cdot s_{0,j}) \oplus s_{1,j} \oplus s_{2,j} \oplus (2 \cdot s_{3,j})$$

- We can compute transformed columns using above equations.
- Note that we can perform the multiplication of polynomials as xor operations in $\text{GF}(2^8)$ field.

Round Transformation MixColumn

- It follows that multiplication by x (i.e., 00000010 or 2) can be implemented as a 1-bit left shift followed by a conditional bitwise XOR with (00011011), which represents $(x^4 + x^3 + x + 1)$

$$x \times f(x) = \begin{cases} (b_6b_5b_4b_3b_2b_1b_00) & \text{if } b_7 = 0 \\ (b_6b_5b_4b_3b_2b_1b_00) \oplus (00011011) & \text{if } b_7 = 1 \end{cases}$$

- So, we need to convert $f(x)$ into binary representation.
- If its b_7 is 0, the left shift.
- Otherwise, left shift and XOR with 00011011.

Round Transformation MixColumn

- Let us see example.

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

ab	8b	89	35
40	7f	f1	05
f0	fc	18	3f
c4	e4	4e	2f

- We get following first column after matrix multiplication.
- $S'_{0,0} = 2 s_{0,0} + 3 s_{1,0} + s_{2,0} + s_{3,0}$
- $= 2. ab + 3. 40 + f0 + c4$
- $2. 10101011 + 3. 01000000 + 11110000 + 11000100$

Round Transformation MixColumn

- Let us see compute.
- 2. $10101011 \quad 01010110$
- $= 01010110 + 00011011 \quad 00011011$
- $= 01001101 \quad 01001101$
- Let us compute
- 3. $01000000 = 00000011 \times 01000000$
- $= 01000000 \times [00000001 + 00000010]$
- $= 01000000 \times 00000001 + 01000000 \times 00000010$
- $= 01000000 + 10000000 = 11000000$

Round Transformation MixColumn

- Now, let come back to
- 2. 10101011 + 3. 01000000 + 11110000 + 11000100
- = 01001101 + 11000000 + 11110000 + 11000100
- = 10111001
- = b9

01001101

11000000

11110000

11000100

10111001

Round Transformation MixColumn

- After matrix multiplication

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

After RowShift

ab	8b	89	35
40	7f	f1	05
f0	fc	18	3f
c4	e4	4e	2f



After MixColumn

b9	94	57	75
e4	8e	16	51
47	20	9a	3f
c5	d6	f5	3b

Round Transformation

AddRoundKey

- In this transformation, the 128 bits of State are bitwise XORed with the 128 bits of the round key.
- $B9 + dc = 10111001 + 11011100 = 01100101 = 65$

After MixColumn

K_1

10111001

11011100

01100101

b9	94	57	75
e4	8e	16	51
47	20	9a	3f
c5	d6	f5	3b

+

dc	9b	97	38
90	49	<u>fe</u>	81
37	<u>df</u>	72	15
b0	e9	3f	a7



65	0f	c0	4d
74	cf	e8	d0
70	ff	e8	2a
75	3f	ca	9c

AES Encryption Example

Plaintext:	0123456789abcdef fedcba9876543210
Key:	0f1571c947d9e8590cb7add6af7f6798
Ciphertext:	ff0b844a0853bf7c6934ab4364148fb9

Start of Round	After SubBytes	After ShiftRows	After MixColumns	Round Key
01 89 fe 76 23 ab dc 54 45 cd ba 32 67 ef 98 10				0f 47 0c af 15 d9 b7 7f 71 e8 ad 67 c9 59 d6 98
0e ce f2 d9 36 72 6b 2b 34 25 17 55 ae b6 4e 88	ab 8b 89 35 05 40 7f f1 18 3f f0 fc e4 4e 2f c4	ab 8b 89 35 40 7f f1 05 f0 fc 18 3f c4 e4 4e 2f	b9 94 57 75 e4 8e 16 51 47 20 9a 3f c5 d6 f5 3b	dc 9b 97 38 90 49 fe 81 37 df 72 15 b0 e9 3f a7
65 0f c0 4d 74 c7 e8 d0 70 ff e8 2a 75 3f ca 9c	4d 76 ba e3 92 c6 9b 70 51 16 9b e5 9d 75 74 de	4d 76 ba e3 c6 9b 70 92 9b e5 51 16 de 9d 75 74	8e 22 db 12 b2 f2 dc 92 df 80 f7 c1 2d c5 1e 52	d2 49 de e6 c9 80 7e ff 6b b4 c6 d3 b7 5e 61 c6
5c 6b 05 f4 7b 72 a2 6d b4 34 31 12 9a 9b 7f 94	4a 7f 6b bf 21 40 3a 3c 8d 18 c7 c9 b8 14 d2 22	4a 7f 6b bf 40 3a 3c 21 c7 c9 8d 18 22 b8 14 d2	b1 c1 0b cc ba f3 8b 07 f9 1f 6a c3 1d 19 24 5c	c0 89 57 b1 af 2f 51 ae df 6b ad 7e 39 67 06 c0

AES Encryption Example

71 48 5c 7d 15 dc da a9 26 74 c7 bd 24 7e 22 9c	a3 52 4a ff 59 86 57 d3 f7 92 c6 7a 36 f3 93 de	a3 52 4a ff 86 57 d3 59 c6 7a f7 92 de 36 f3 93	d4 11 fe 0f 3b 44 06 73 cb ab 62 37 19 b7 07 ec	2c a5 f2 43 5c 73 22 8c 65 0e a3 dd f1 96 90 50
f8 b4 0c 4c 67 37 24 ff ae a5 c1 ea e8 21 97 bc	41 8d fe 29 85 9a 36 16 e4 06 78 87 9b fd 88 65	41 8d fe 29 9a 36 16 85 78 87 e4 06 65 9b fd 88	2a 47 c4 48 83 e8 18 ba 84 18 27 23 eb 10 0a f3	58 fd 0f 4c 9d ee cc 40 36 38 9b 46 eb 7d ed bd
72 ba cb 04 1e 06 d4 fa b2 20 bc 65 00 6d e7 4e	40 f4 1f f2 72 6f 48 2d 37 b7 65 4d 63 3c 94 2f	40 f4 1f f2 6f 48 2d 72 65 4d 37 b7 2f 63 3c 94	7b 05 42 4a 1e d0 20 40 94 83 18 52 94 c4 43 fb	71 8c 83 cf c7 29 e5 a5 4c 74 ef a9 c2 bf 52 ef
0a 89 c1 85 d9 f9 c5 e5 d8 f7 f7 fb 56 7b 11 14	67 a7 78 97 35 99 a6 d9 61 68 68 0f b1 21 82 fa	67 a7 78 97 99 a6 d9 35 68 0f 61 68 fa b1 21 82	ec 1a c0 80 0c 50 53 c7 3b d7 00 ef b7 22 72 e0	37 bb 38 f7 14 3d d8 7d 93 e7 08 a1 48 f7 a5 4a

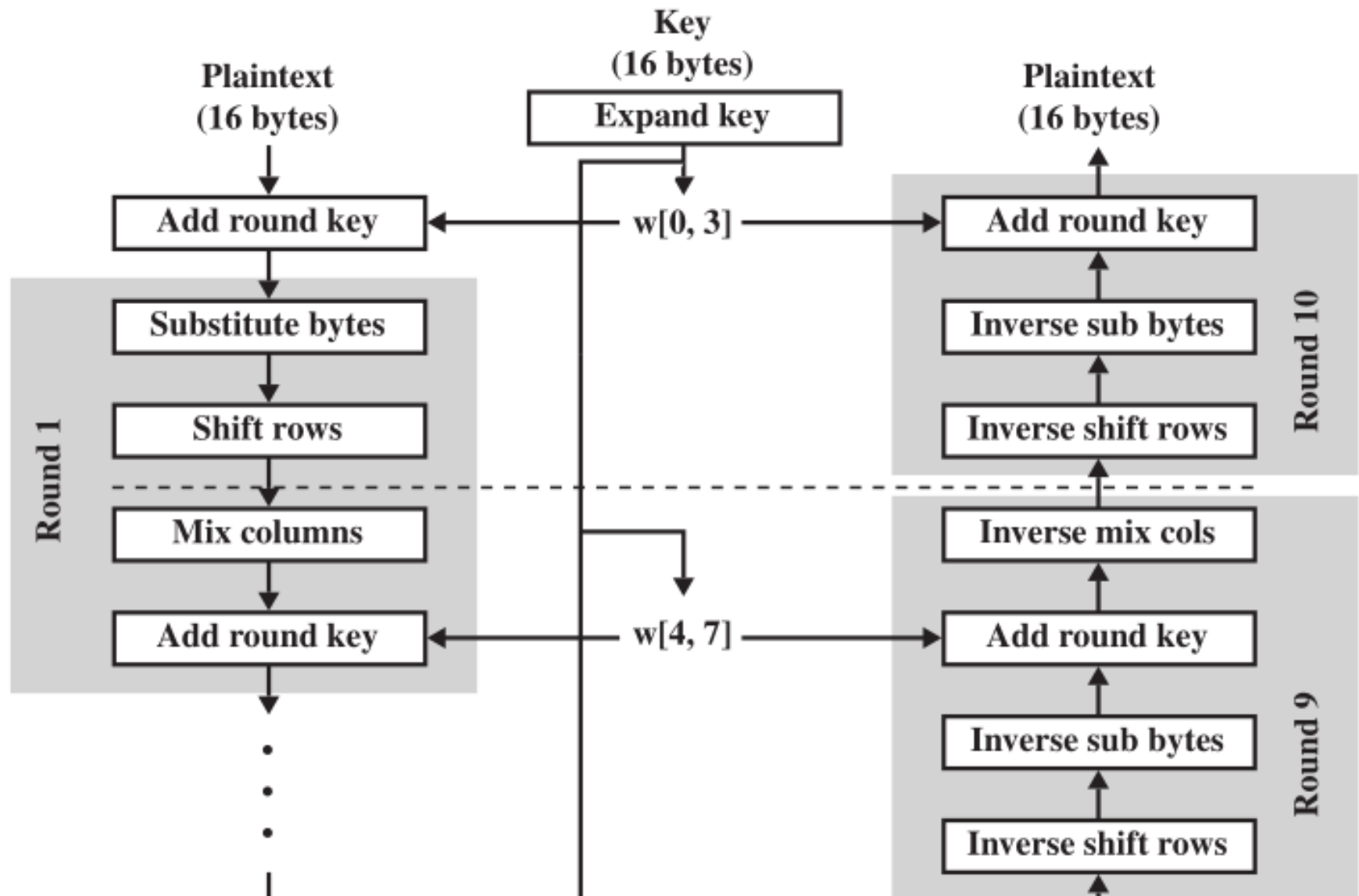
AES Encryption Example

db a1 f8 77 18 6d 8b ba a8 30 08 4e ff d5 d7 aa	b9 32 41 f5 ad 3c 3d f4 c2 04 30 2f 16 03 0e ac	b9 32 41 f5 3c 3d f4 ad 30 2f c2 04 ac 16 03 0e	b1 1a 44 17 3d 2f ec b6 0a 6b 2f 42 9f 68 f3 b1	48 f3 cb 3c 26 1b c3 be 45 a2 aa 0b 20 d7 72 38
f9 e9 8f 2b 1b 34 2f 08 4f c9 85 49 bf bf 81 89	99 1e 73 f1 af 18 15 30 84 dd 97 3b 08 08 0c a7	99 1e 73 f1 18 15 30 af 97 3b 84 dd a7 08 08 0c	31 30 3a c2 ac 71 8c c4 46 65 48 eb 6a 1c 31 62	fd 0e c5 f9 0d 16 d5 6b 42 e0 4a 41 cb 1c 6e 56
cc 3e ff 3b a1 67 59 af 04 85 02 aa a1 00 5f 34	4b b2 16 e2 32 85 cb 79 f2 97 77 ac 32 63 cf 18	4b b2 16 e2 85 cb 79 32 77 ac f2 97 18 32 63 cf	4b 86 8a 36 b1 cb 27 5a fb f2 f2 af cc 5a 5b cf	b4 ba 7f 86 8e 98 4d 26 f3 13 59 18 52 4e 20 76
ff 08 69 64 0b 53 34 14 84 bf ab 8f 4a 7c 43 b9				

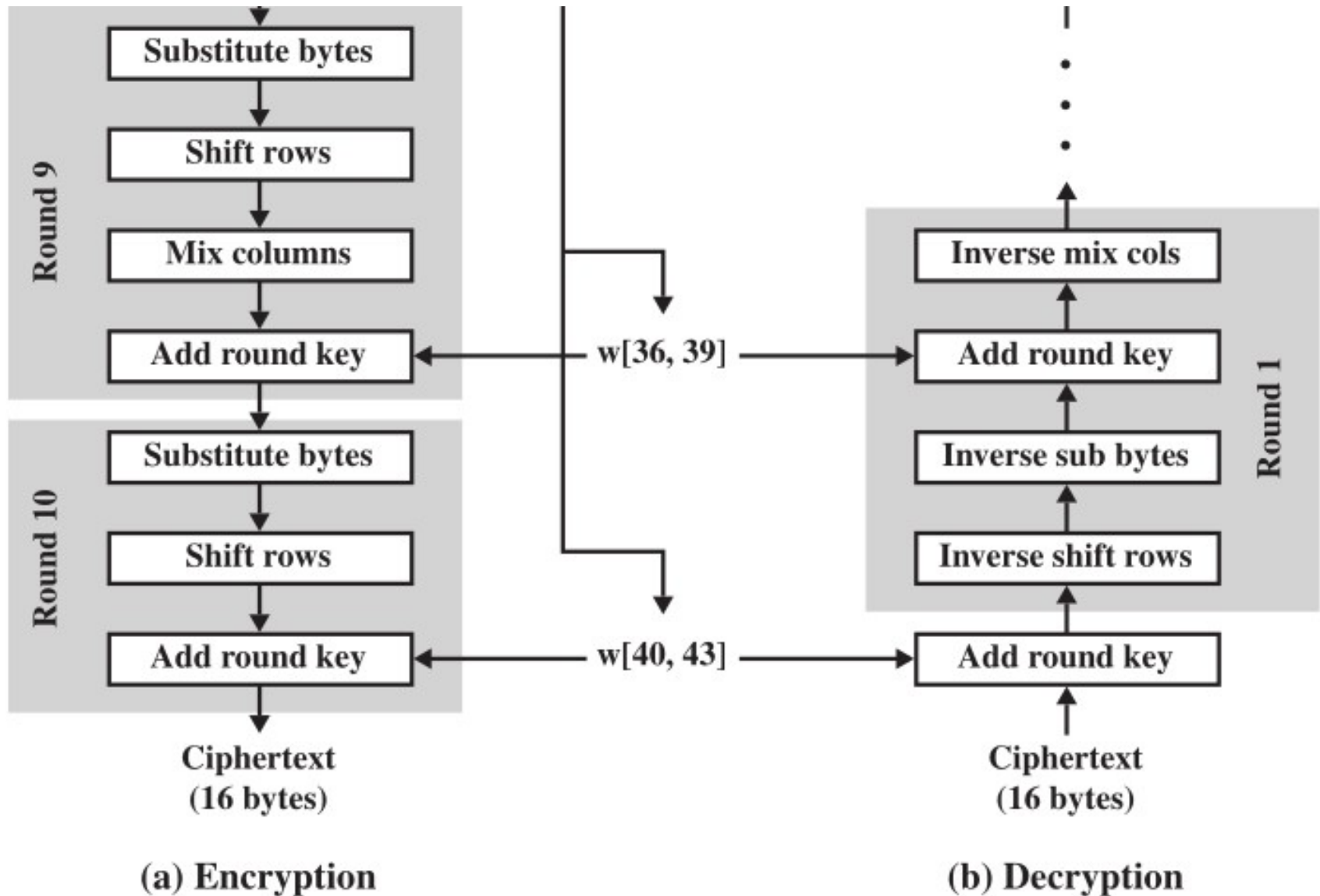
AES Decryption

- Add K_{10} round key with ciphertext
- Perform inverse round transformation
- After last round transformation, we will get plaintext.

AES Decryption



AES Decryption



AES Decryption: AddRoundKey

- AddRoundKey is same to AddRoundKey to that of Encryption.
- But in this stage, last round key (K_{10}) is XORed with cipher text.
- The result is stored in state matrix.

AES Decryption: Round Transformation

- In round transformation, following transformations are performed one after another.
- Inverse shift rows
- Inverse sub bytes
- Add round key
- Inverse mix cols
- The last round does not have Inverse mid cols transformation.

AES Decryption: InverseShiftRows

- InvShiftRows performs the circular right shifts in the for each of the last three rows
- Second row of state matrix is circular right shift with 1-byte
- Third row with 2 bytes.
- Fourth row with 3 bytes.

AES Decryption: InverseSubBytes

- InvSubBytes makes use of the inverse S-box for substituting bytes of state matrix.
- For substitution, the location of byte in inverse S-box is found using the byte to be substituted.
- Byte 2A means second row and A column in inverse S-box.
- At this location, byte 95 is present.
- So, 2A is substituted with 95.
- Inverse S-box is given below.

AES Decryption: Inverse Shift Bytes

		y															
		0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
x	0	52	09	6A	D5	30	36	A5	38	BF	40	A3	9E	81	F3	D7	FB
	1	7C	E3	39	82	9B	2F	FF	87	34	8E	43	44	C4	DE	E9	CB
	2	54	7B	94	32	A6	C2	23	3D	EE	4C	95	0B	42	FA	C3	4E
	3	08	2E	A1	66	28	D9	24	B2	76	5B	A2	49	6D	8B	D1	25
	4	72	F8	F6	64	86	68	98	16	D4	A4	5C	CC	5D	65	B6	92
	5	6C	70	48	50	FD	ED	B9	DA	5E	15	46	57	A7	8D	9D	84
	6	90	D8	AB	00	8C	BC	D3	0A	F7	E4	58	05	B8	B3	45	06
	7	D0	2C	1E	8F	CA	3F	0F	02	C1	AF	BD	03	01	13	8A	6B
	8	3A	91	11	41	4F	67	DC	EA	97	F2	CF	CE	F0	B4	E6	73
	9	96	AC	74	22	E7	AD	35	85	E2	F9	37	E8	1C	75	DF	6E
	A	47	F1	1A	71	1D	29	C5	89	6F	B7	62	0E	AA	18	BE	1B
	B	FC	56	3E	4B	C6	D2	79	20	9A	DB	C0	FE	78	CD	5A	F4
	C	1F	DD	A8	33	88	07	C7	31	B1	12	10	59	27	80	EC	5F
	D	60	51	7F	A9	19	B5	4A	0D	2D	E5	7A	9F	93	C9	9C	EF
	E	A0	E0	3B	4D	AE	2A	F5	B0	C8	EB	BB	3C	83	53	99	61
	F	17	2B	04	7E	BA	77	D6	26	E1	69	14	63	55	21	0C	7D

(b) Inverse S-box

AES Decryption: InverseSubBytes

- InvSubBytes makes use of the inverse S-box for substituting bytes of state matrix.
- For substitution, the location of byte in inverse S-box is found using the byte to be substituted.
- Byte 2A means second row and A column in inverse S-box.
- At this location, byte 95 is present.
- So, 2A is substituted with 95.
- Inverse S-box is given below.

AES Decryption: AddRoundKey

- Inverse add round key is same to decryption add round key.
- It is because xor operation is its own inverse.

AES Decryption: InverseMixCols

- InvMixColumns is defined by the following matrix multiplication:

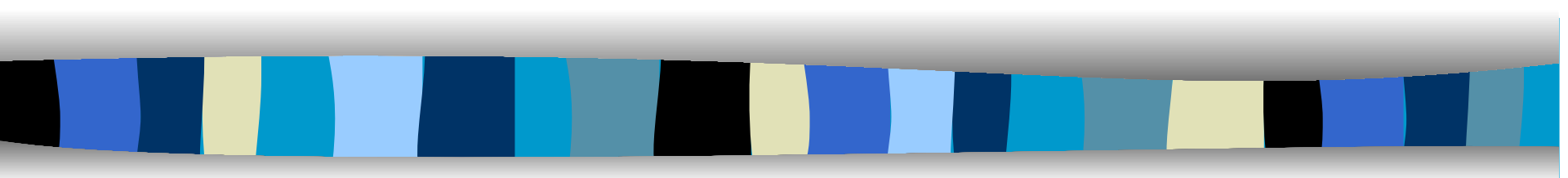
$$\begin{bmatrix} 0E & 0B & 0D & 09 \\ 09 & 0E & 0B & 0D \\ 0D & 09 & 0E & 0B \\ 0B & 0D & 09 & 0E \end{bmatrix} \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix} = \begin{bmatrix} s'_{0,0} & s'_{0,1} & s'_{0,2} & s'_{0,3} \\ s'_{1,0} & s'_{1,1} & s'_{1,2} & s'_{1,3} \\ s'_{2,0} & s'_{2,1} & s'_{2,2} & s'_{2,3} \\ s'_{3,0} & s'_{3,1} & s'_{3,2} & s'_{3,3} \end{bmatrix}$$

AES Properties

- **Avalanche Effect:**
- A small change in plaintext or key produces a large change in the ciphertext.

Round		Number of Bits that Differ
	0123456789abcdef fedcba9876543210 0023456789abcdef fedcba9876543210	1
0	0e3634aece7225b6f26b174ed92b5588 0f3634aece7225b6f26b174ed92b5588	1
10	ff0b844a0853bf7c6934ab4364148fb9 612b89398d0600cde116227ce72433f0	58

Symmetric Cryptography



Modes of Operation

Modes of operation

- Block cipher uses same key to different blocks of plaintext.
- This is not good for security.
- So, to generate strong and secure ciphertext NIST defines five modes of operations of block ciphers.
- A mode of operation is a technique for enhancing the effect of a cryptographic algorithm.

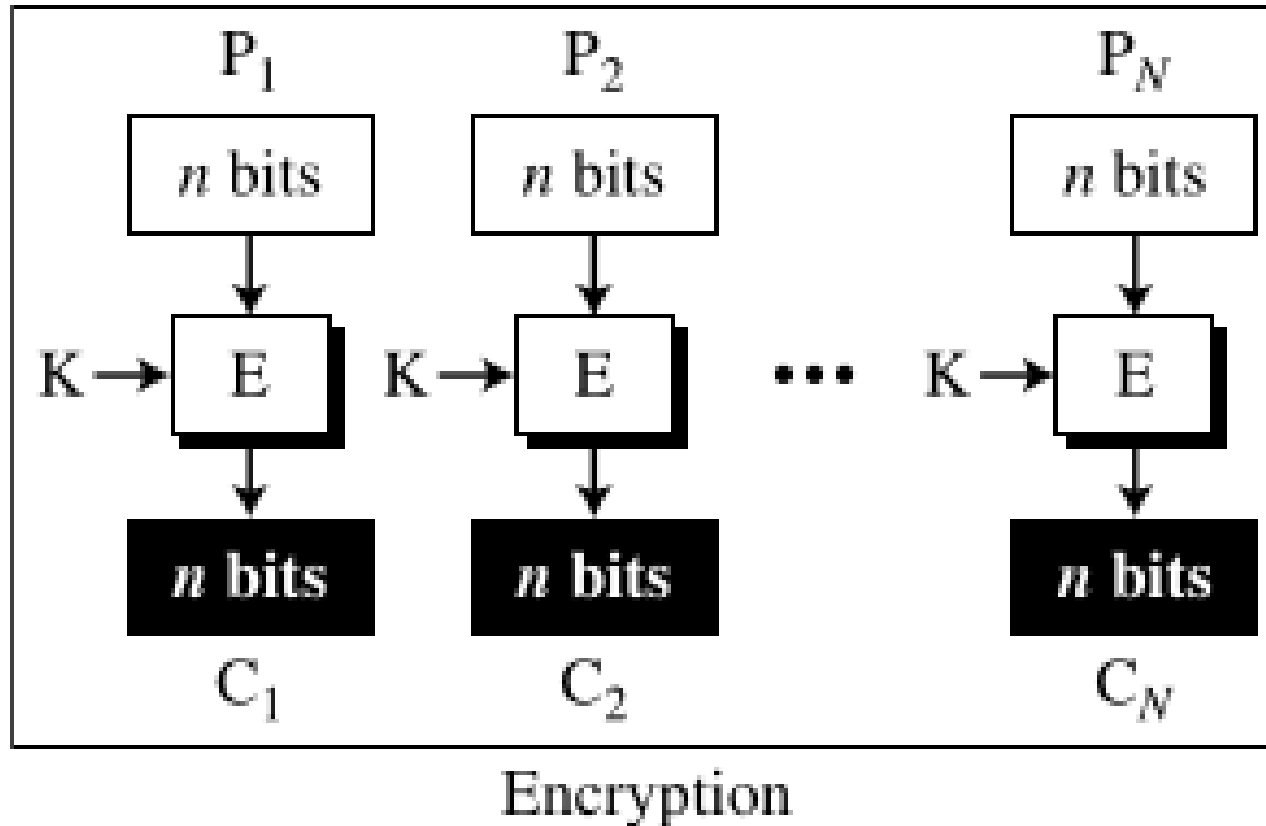
Modes of operation

- Five modes of operations are
- Electronic Codebook (ECB)
- Cipher Block Chaining (CBC)
- Cipher Feedback (CFB)
- Output Feedback (OFB)
- Counter Mode (CTR)

Electronic Codebook

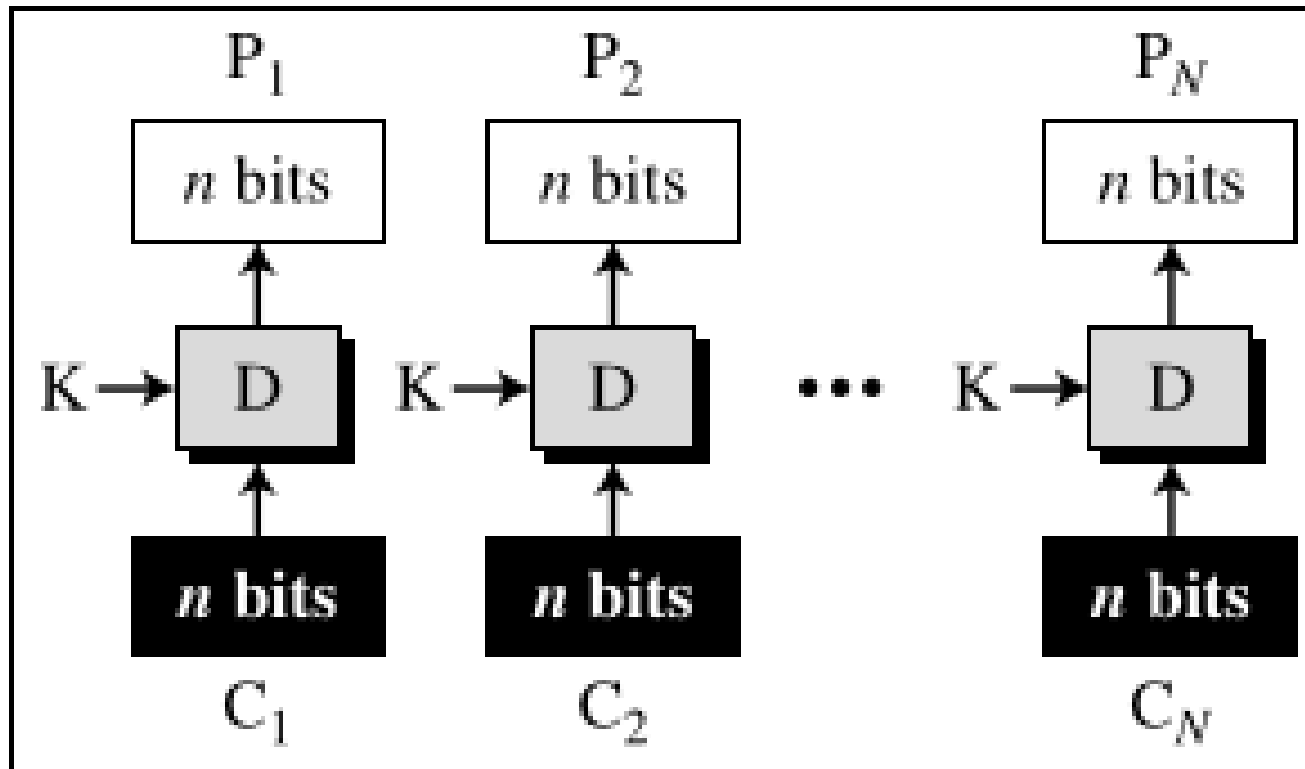
- The simplest mode of operation is called the electronic codebook (ECB) mode.
- The plaintext is divided into different blocks of b -bits.
- If the last block is not of b -bits, then block is padded
- The same key is used to encrypt and decrypt each block.

Electronic Codebook



- E is block encryption algorithm.
- E can be either DES or AES

Electronic Codebook



Decryption

- D is block decryption algorithm.
- D can be either DES or AES

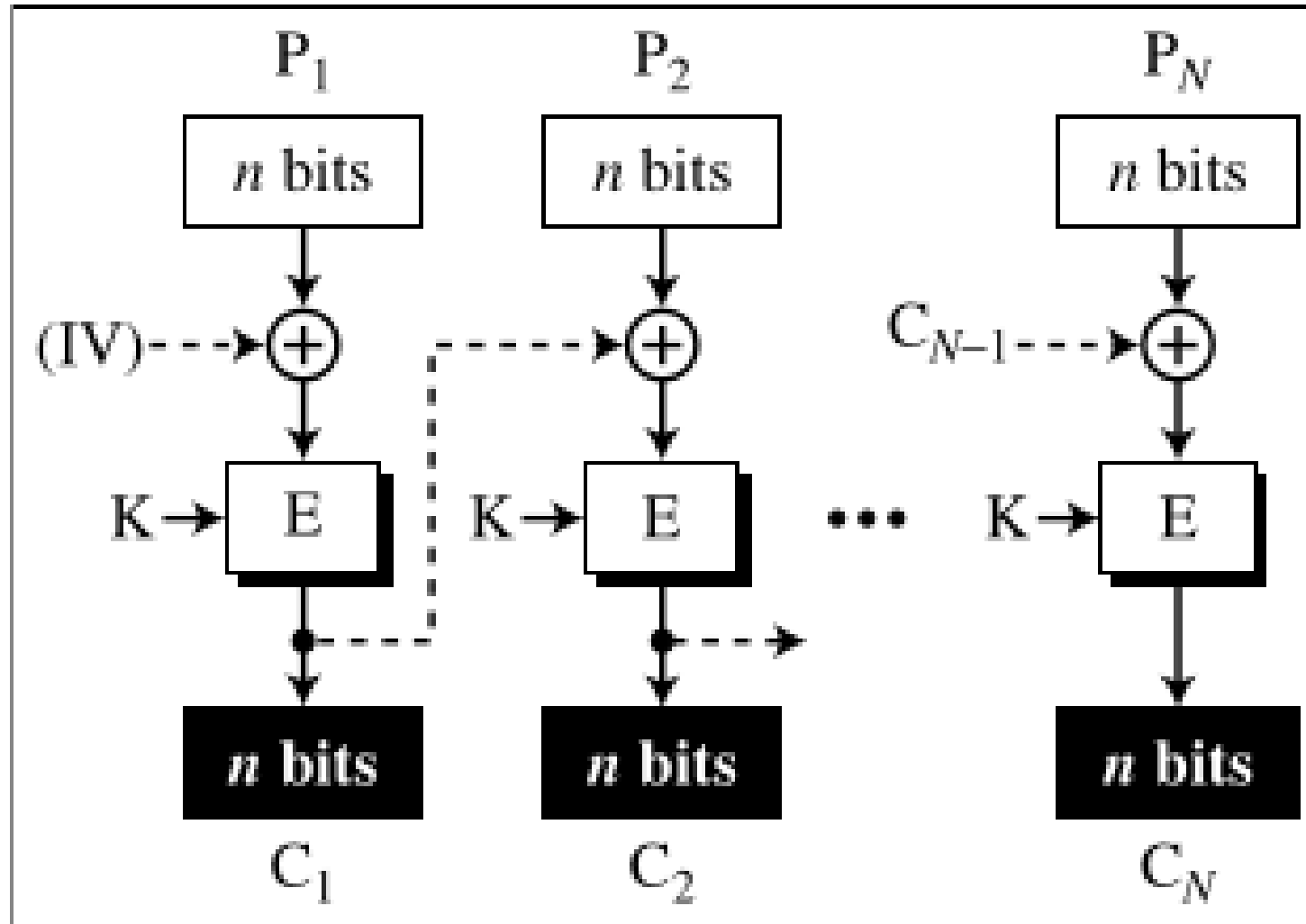
ECB limitations

- If a message has two identical blocks, the ciphertext will be two identical blocks
- Blocks can be rearranged by an adversary to his advantage
- Message information is not sufficiently diffused
- A single bit error can create errors in several bits in corresponding block.
- Other blocks are not affected.
- Using the same key on multiple blocks makes it easier to break

Cipher Block Chaining

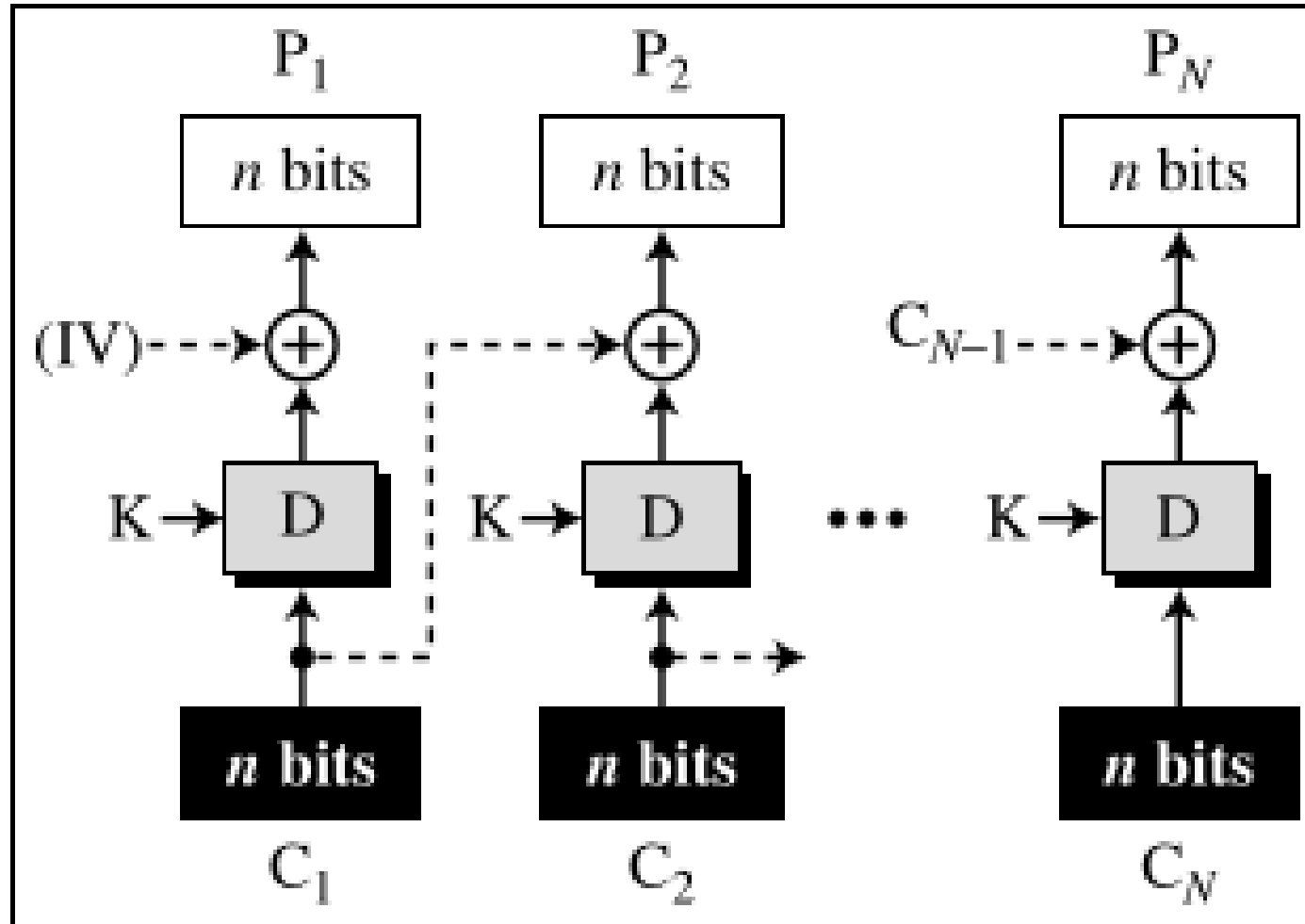
- In CBC mode, each plaintext block is XORed with the previous ciphertext block before being encrypted.
- For the first block, a phony block called the **initialization vector (IV)** is used.
- The sender and receiver agree upon a specific predetermined **IV**.
- IV can be shared as long as new IV is used for each communication.

Cipher Block Chaining



Encryption

Cipher Block Chaining



Decryption

CBC advantages and limitations

- Error in one block propagates to all other blocks.
- IV vector must be known to both sender and receiver.
- Sequential implementation.
- Cannot be parallelized.

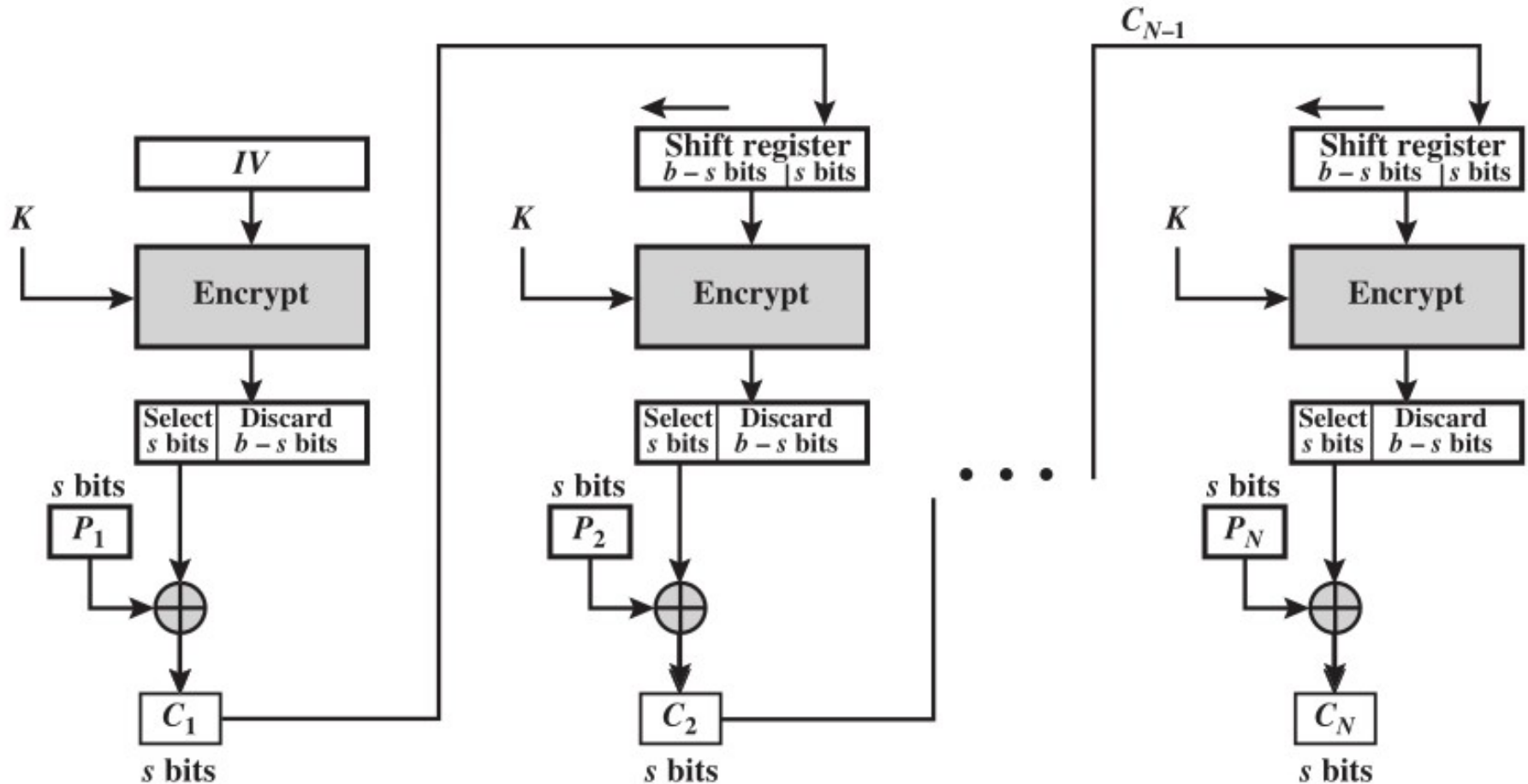
Cipher Feedback

- Cipher feedback (CFB) is one of the stream cipher.
- In stream cipher none need to pad a message.
- It also can operate in real time.
- Thus, if a character stream is being transmitted, each character can be encrypted and transmitted immediately using a character-oriented stream cipher.
- One desirable property of a stream cipher is that the ciphertext be of the same length as the plaintext.

Cipher Feedback

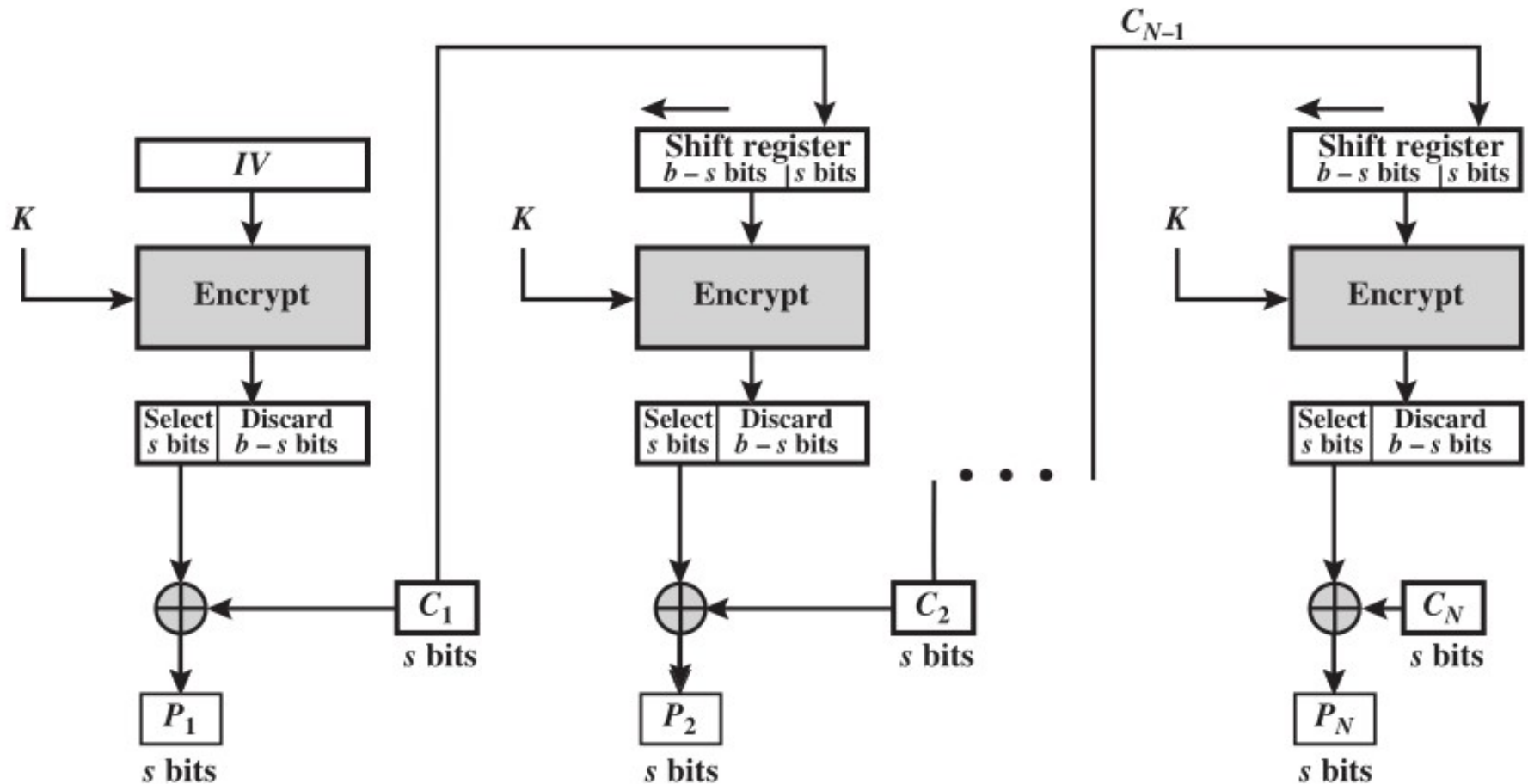
- The encryption function of the first block takes b bit IV and Key K .
- The most significant s -bits of the result is XORed with s -bit plaintext to generate s -bit ciphertext.
- The ciphertext is fed to next block.
- The input data of previous block is copied to shift register.
- The shift register is shifted by s -bits.
- Then s -bit ciphertext is appended with shift register.
- This process continues until all plaintext units have been encrypted.

Cipher Feedback



(a) Encryption

Cipher Feedback

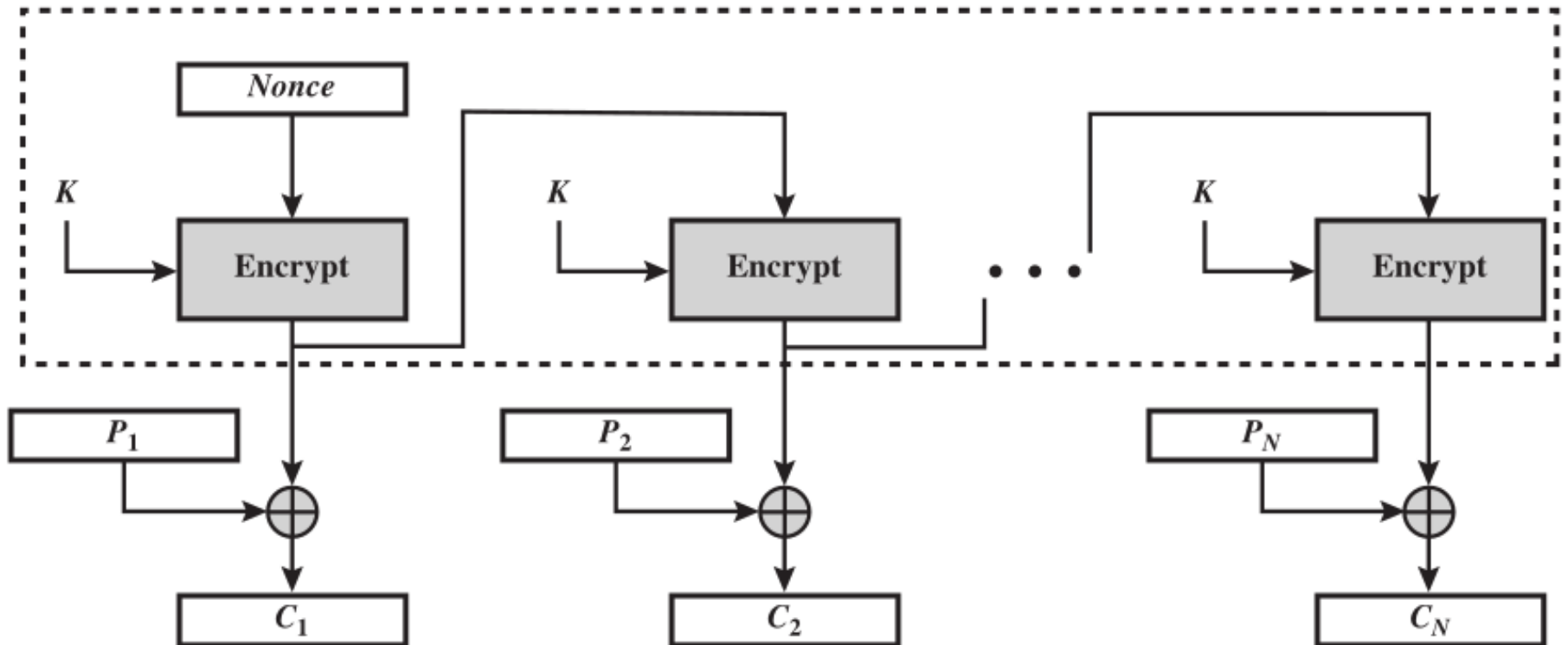


(b) Decryption

Output Feedback

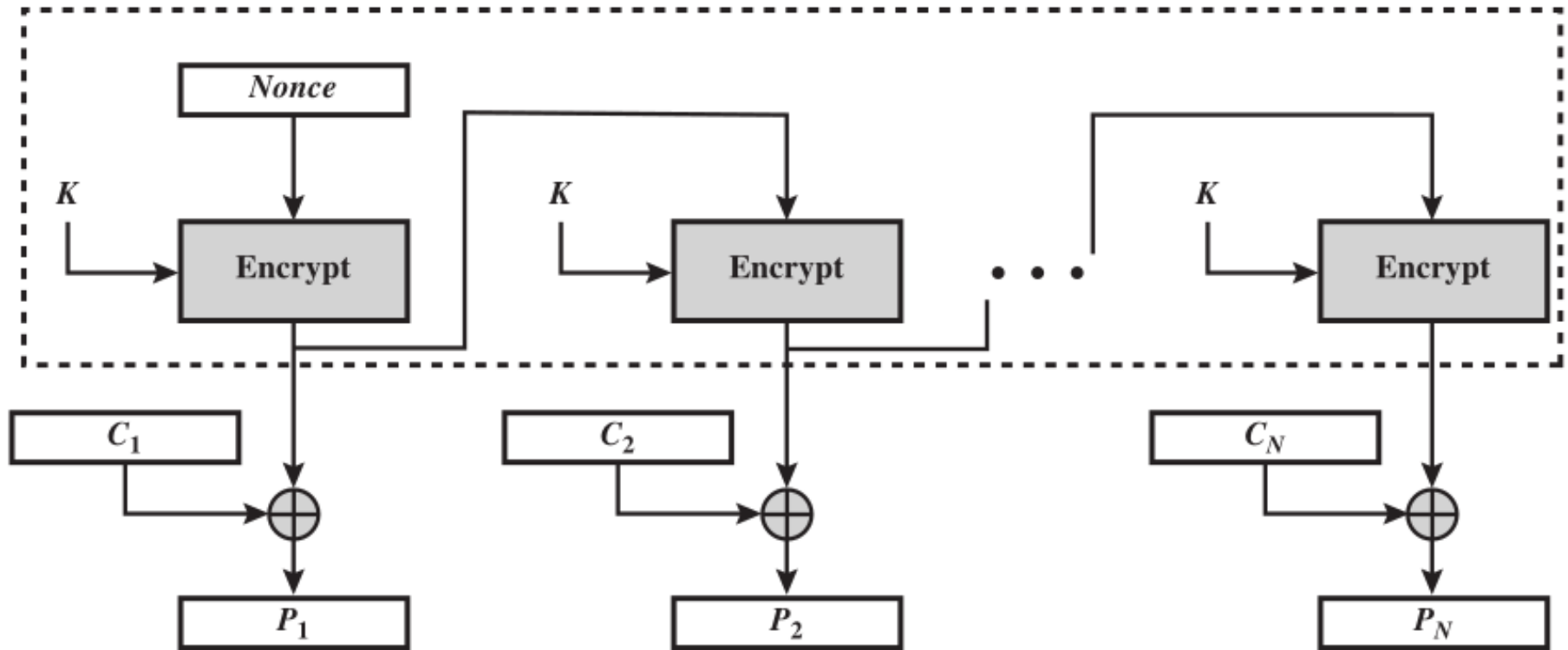
- It is similar to the Cipher feedback with simple modification.
- The output of encryption function is feed back to the next block not the cipher.
- So, next block ciphertext is independent of previous block plaintext or ciphertext.
- So, error in plaintext or ciphertext does not propagate to next block.
- One more advantage is the feedback can be precomputed.
- CS526 So, this mode works as Topic 1 stream cipher perfectly.

Output Feedback



(a) Encryption

Output Feedback



(b) Decryption

Output Feedback

- IV must be nonce means IV must be unique to each execution of the encryption operation.
- Because output of the encryption function depends only on IV and the key.
- This generates fixed output for fixed plaintext.
- So, it will be easy for attacker to determine the output of encryption function.
- One advantage is bit errors in transmission do not propagate.
- It is more vulnerable to message modification.

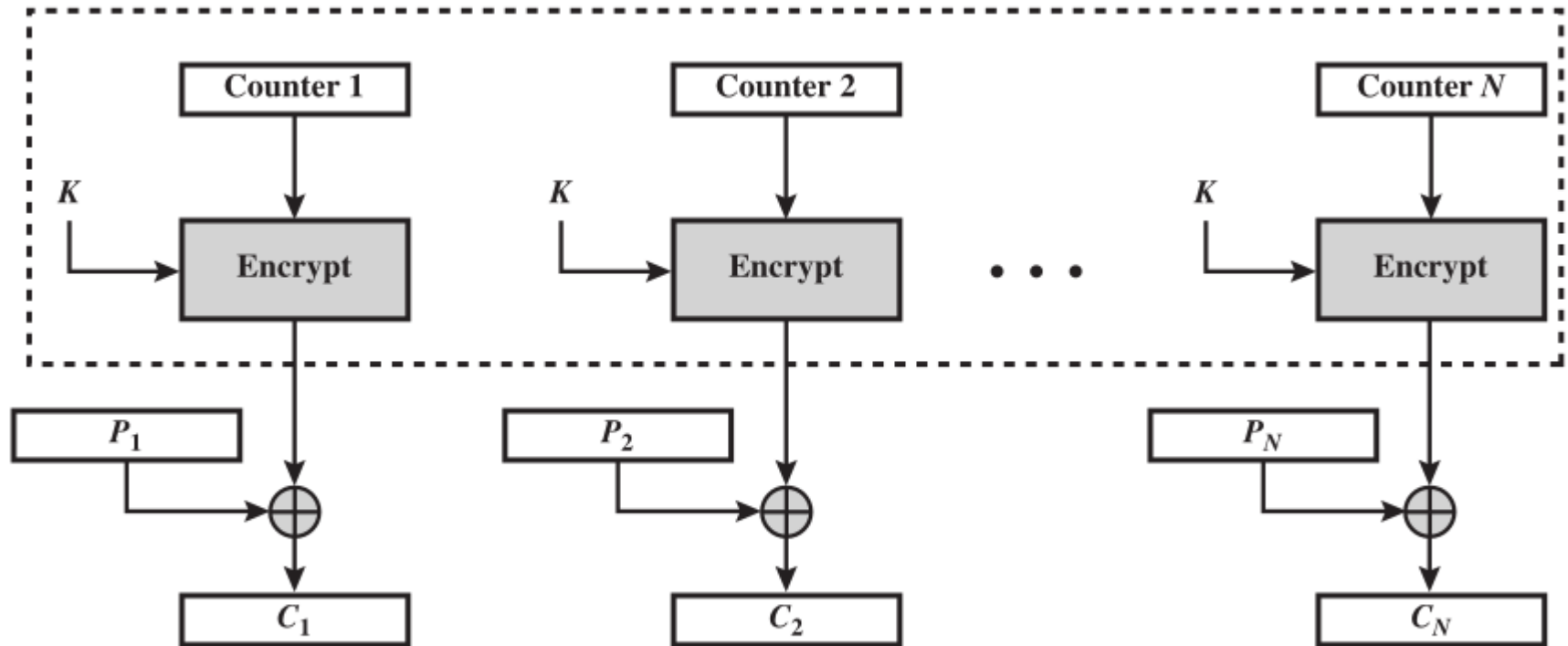
Counter Mode

- In this mode, counter equal to the plaintext block size is used.
- The counter value must be different for each plaintext block that is encrypted.
- Typically, the counter is initialized to some value and then incremented by 1 for each subsequent block (modulo 2^b , where b is the block size).
- For encryption, the counter is encrypted and then XORed with the plaintext block to produce the ciphertext block;
- there is no chaining.

Counter Mode

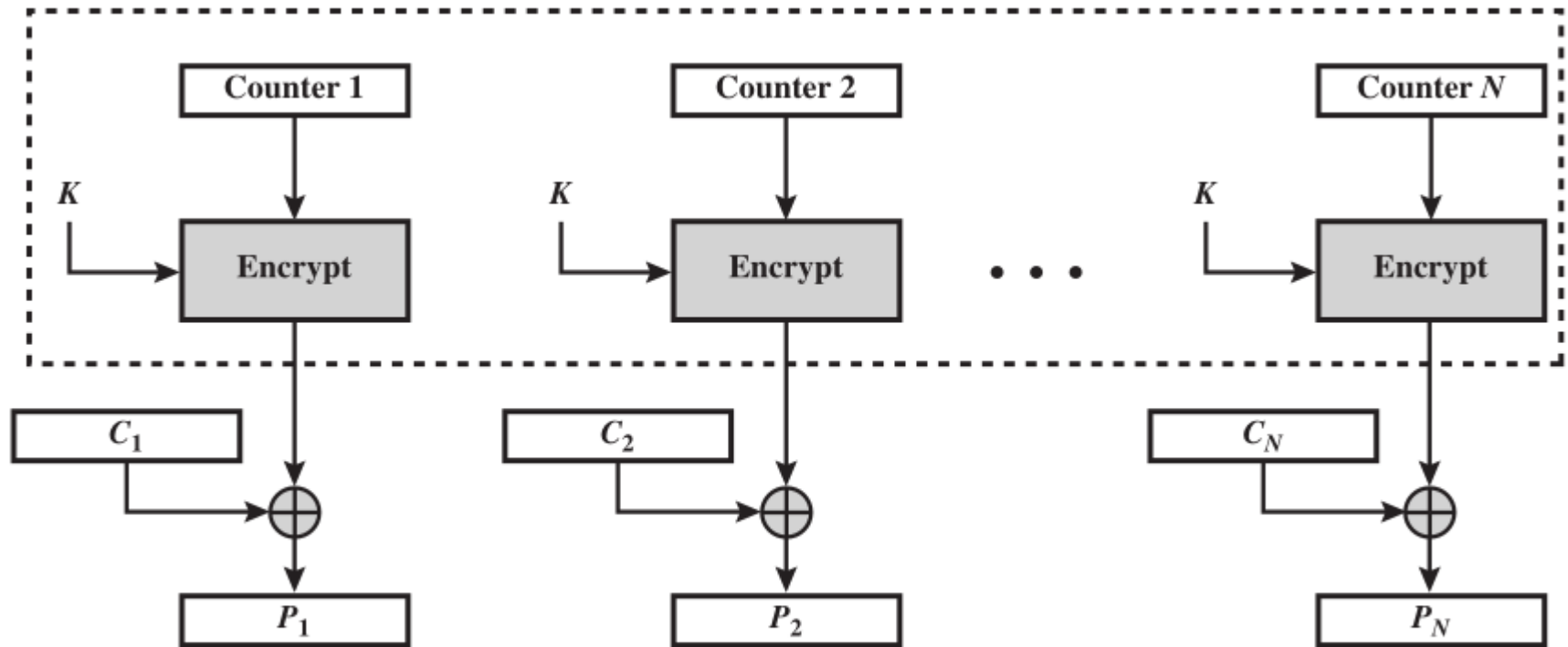
- For decryption, the same sequence of counter values is used, with each encrypted counter XORed with a ciphertext block to recover the corresponding plaintext block.
- It is more vulnerable to message modification.

Counter Mode



(a) Encryption

Counter Mode



(b) Decryption

Counter Mode

- Can do parallel encryptions in h/w or s/w
- Can preprocess in advance of need
- Random access to encrypted data blocks
- But must never reuse key/counter values, otherwise could break